

Next.js — 0 dan Expert darajagacha (o'zbek tilida)

Bu qo'llanma **Next.js 16** (App Router) va **React 19** ni **mutlaqo noldan** professional darajagacha o'rgatadi. Hech qanday oldingi tajriba — JavaScript ham, React ham — talab qilinmaydi: birinchi qism ularni ham noldan tushuntiradi.

Versiya: Next.js **16**, React **19**, Node.js **20.9+**. Kod misollari shu versiyalarga moslangan (async `params`, `"use cache"`, sukut bo'yicha keshlanmaydigan `fetch` /GET va h.k.).

Qanday ishlatish kerak

1. Qismlarni va boblarni **tartib bilan** o'qing (0-BOB → 19-BOB). Har biri oldingisiga tayanadi.
2. Har bobdagi kodni **o'z kompyuteringizda yozib, ishga tushiring**. Faqat o'qib ketsangiz — bilim o'rnashmaydi.
3. Bob oxiridagi **mashqlar (challenge)** ni mustaqil bajaring.
4. "⚠️ Next.js 16" va "Keng tarqalgan xatolar" bloklarini e'tibordan qochirmang — ular real loyihada vaqtingizni tejaydi.

Talablar

Kerak	Daraja
Kompyuter, terminal, brauzer	Asoslar
Node.js 20.9+	O'rnatiladi (0-bobda)
Oldingi dasturlash tajribasi	Shart emas — JS va React noldan o'rgatiladi

To'liq yo'l xaritasi (roadmap)

1-QISM — Asoslar

Bob	Mavzu
0	Boshlashdan oldin (muhit sozlash, Node.js)
1	JavaScript asoslari
2	React asoslari
3	Next.js bilan birinchi qadamlar
4	Routing — sahifalar tizimi

2-QISM — Server, ma'lumot va Actions

Bob	Mavzu
5	Server Components va Client Components
6	Ma'lumot olish (Data Fetching)
7	Server Actions — ma'lumot yuborish va saqlash
8	Keshlash (Caching) va "use cache"

3-QISM — Full-stack: API, baza, auth

Bob	Mavzu
9	Route Handlers — o'z API'ngiz
10	Ma'lumotlar bazasi (Prisma)
11	Validatsiya (Zod)
12	Autentifikatsiya (Better Auth) va xavfsizlik

4-QISM — Production (yakuniy)

Bob	Mavzu
13	Performance (tezlik)
14	Test yozish
15	Deployment — saytni internetga chiqarish
16	Monitoring, ilg'or patternlar va yakuniy loyiha

BONUS — Ilg'or mavzular

Bob	Mavzu
17	Ko'p tillik (i18n)
18	To'lov qabul qilish (Stripe va mahalliy provayderlar)
19	Real-time (jonli yangilanish / WebSocket)

Jami: **20 bob**, 4 asosiy qism + bonus. 0 dan to'liq full-stack production ilovagacha.

Boshlashdan oldin

nodejs.org dan **Node.js 20.9+** ni o'rnating, keyin:

```
npx create-next-app@latest
```



Tayyor bo'lsang — **1-QISM: Asoslar** → dan boshla.

Next.js — 0 dan Expert darajagacha (1-QISM)

 [README](#) / [Mundarija](#) · [Keyingi: 2-qism — Server & Actions](#) →

Kim uchun? Bu qo'llanma hech qachon dasturlash qilmagan, JavaScript ham, React ham bilmaydigan odam uchun yozilgan. Hech narsani "bilasiz" deb taxmin qilmaymiz. Har bir tushunchani oddiy tildan, real hayotdan misollar bilan tushuntiramiz.

Versiya: Next.js **16** (2026-yil holatiga ko'ra eng so'nggi LTS), React 19. Node.js **20.9+** kerak bo'ladi.

To'liq kursning yo'l xaritasi

Bu — katta safarning birinchi qismi. Mana butun yo'l qanday ko'rinadi:

1-QISM (shu hujjat) — Poydevor va birinchi qadamlar - 0-bob. Boshlashdan oldin: web, dasturlash, Next.js nima - 1-bob. JavaScript asoslari - 2-bob. React asoslari - 3-bob. Next.js bilan birinchi qadamlar - 4-bob. Routing (sahifalar tizimi)

2-QISM — Server, ma'lumotlar va dinamik sahifalar - Server Components va Client Components - Ma'lumot olish (data fetching) - Server Actions (formalar, ma'lumot saqlash) - Caching (keshlash) va `"use cache"`

3-QISM — Haqiqiy ilovalar - API yo'naklari (Route Handlers) - Ma'lumotlar bazasi (database) bilan ishlash - Autentifikatsiya (login/register) - Fayl yuklash, validatsiya, xatolarni boshqarish

4-QISM — Expert daraja - Tezlikni optimallashtirish (performance) - Test yozish - Deployment (internetga chiqarish) - Monitoring, ilg'or patternlar, real loyiha

Bu qo'llanmadan qanday foydalanish kerak

- 1. Tartib bilan o'qing.** Boblar bir-birining ustiga quriladi. 1-bobni tushunmasdan 3-bobga o'tmang.
- 2. O'qiganingizni yozib ko'ring.** Dasturlashni o'qib emas, **yozib** o'rganadilar. Har bir kod misolini o'z qo'lingiz bilan yozing.
- 3. Masalalarni albatta yeching.** Har bobdan keyin 20+ masala bor. Ularni yechmasangiz, mavzu miyada qolmaydi. Eng kam — yarmini yeching.

4. **Shoshilmang.** Bir bobni 1–2 kunda hazm qilsangiz ham yetadi. Asosiysi — tushunish.

Maslahat: Yonida daftar tuting. Tushunmagan so'zni yozib qo'ying, keyin qaytib qarang.

0-BOB. Boshlashdan oldin

Next.js'ga kirishdan oldin, "bu narsa umuman nima va nega kerak" degan savolga javob beraylik. Asos mustahkam bo'lsa, qolgani oson.

0.1. Internet va web qanday ishlaydi?

Tasavvur qiling, siz restoranda o'tiribsiz.

- **Siz** — mijoz (customer).
- **Ofitsiant** — siz bilan gaplashadigan odam.
- **Oshxona** — taom tayyorlanadigan joy.

Siz menyuga qaraysiz, ofitsiantga "lag'mon" deysiz, ofitsiant buyurtmani oshxonaga olib boradi, oshxona taom tayyorlaydi, ofitsiant uni sizga olib keladi.

Internet ham xuddi shunday ishlaydi:

Restoran	Internet
Siz (mijoz)	Browser (Chrome, Safari) — sahifani ko'rsatadigan dastur
Ofitsiant	HTTP so'rovi (request) — browser server bilan gaplashish usuli
Oshxona	Server — internetdagi kuchli kompyuter
Tayyorlangan taom	Response (javob) — server qaytaradigan ma'lumot (HTML sahifa, rasm, ma'lumot)

Jarayon: 1. Siz brauzerga `google.com` deb yozasiz. 2. Brauzer Google serveriga **so'rov (request)** yuboradi: "Menga bosh sahifani ber". 3. Server javob (response) qaytaradi — bu HTML sahifa. 4. Brauzer o'sha HTMLni ekranda chiroyli qilib ko'rsatadi.

Frontend va Backend — bu ikki so'zni doim eshitasiz: - **Frontend** = "old tomon" = foydalanuvchi **ko'radigan** qism. Tugmalar, ranglar, matnlar, animatsiyalar. Bu — brauzerda ishlaydi. - **Backend** = "orqa tomon" = foydalanuvchi **ko'rmaydigan** qism. Ma'lumotlarni saqlash, parolni tekshirish, hisob-kitoblar. Bu — serverda ishlaydi.

Restoranda: zal va menyu — frontend. Oshxona va omborxona — backend.

Next.js'ning eng kuchli tomoni shundaki, u ham frontend, ham backendni bitta joyda yozish imkonini beradi. Buni keyinroq ko'ramiz.

0.2. Dasturlash nima va kompyuter qanday "tushunadi"?

Dasturlash — bu kompyuterga **aniq buyruqlar** berish san'ati. Kompyuter juda tez, lekin juda "ahmoq" — u faqat aniq aytilgan narsani bajaradi. "Choy damla" desangiz tushunmaydi, lekin "suvni qaynat → choy sol → 5 daqiqa kut" desangiz bajaradi.

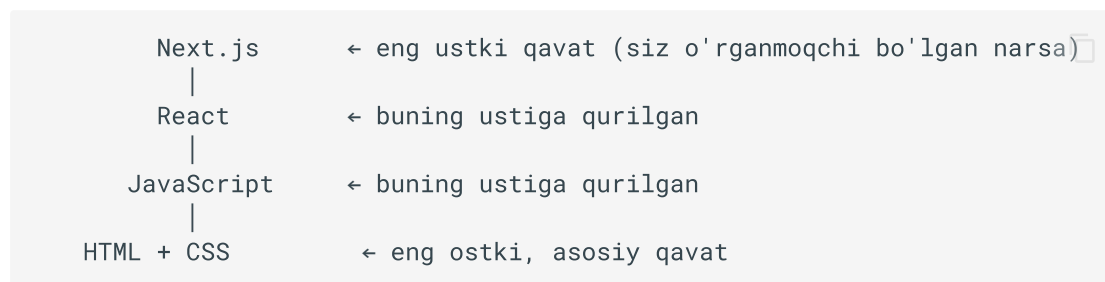
Webda 3 ta asosiy "til" bor. Ularni uyga o'xshatsak:

- **HTML** — uyning **skeleti** (devorlar, eshiklar, derazalar). Sahifada nima borligini belgilaydi: sarlavha, matn, rasm, tugma.
- **CSS** — uyning **bezagi** (rang, oboy, mebel). Sahifa qanday **ko'rinishini** belgilaydi.
- **JavaScript** — uyning **elektr va mexanikasi** (lampa yonadi, eshik ochiladi). Sahifani **harakatga keltiradi**: tugma bosilganda nimadir bo'lishi.

React — JavaScript ustiga qurilgan **vosita (kutubxona)**. U murakkab sahifalarni oson yasashga yordam beradi.

Next.js — React ustiga qurilgan **freymvork**. "Freymvork" — bu tayyor qoidalar va vositalar to'plami, ular bilan ish ancha tezlashadi.

Piramida shunday:



Shuning uchun biz **pastdan yuqoriga** o'rganamiz: avval bir oz JavaScript, keyin React, keyin Next.js.

HTML va CSS haqida: Ular nisbatan oson va Next.js'ni o'rganishda yo'l-yo'lakay o'rganib ketasiz. Bu qo'llanmada JavaScript'ga ko'proq e'tibor beramiz, chunki u

qiyinroq va muhimroq.

0.3. Next.js nima va nega aynan u?

Next.js — bu zamonaviy veb-saytlar va veb-ilovalar yasash uchun eng mashhur freymvorklardan biri. Uni Vercel kompaniyasi yaratgan.

Nega faqat React'ning o'zi yetmaydi? Sof React bilan sayt yasaganda quyidagi muammolar chiqadi: - Sahifadan sahifaga o'tish (routing) tizimini o'zingiz qurishingiz kerak. - Sayt qidiruv tizimlarida (Google) yaxshi chiqmaydi (SEO muammosi). - Serverdan ma'lumot olish murakkab. - Tezlikni optimallashtirish qiyin.

Next.js bularning hammasini sizning o'rningizga hal qiladi: -  **Routing tayyor** — papka yaratmasiz, sahifa paydo bo'ladi (keyin ko'ramiz). -  **SEO yaxshi** — sahifalar serverda tayyorlanadi, Google ularni oson o'qiydi. -  **Tez** — rasm, shrift, kodni avtomatik optimallashtiradi. -  **Frontend + Backend bitta loyihada** — alohida server kerak emas.

Kim ishlatadi? Netflix, TikTok, Twitch, Nike, Notion va minglab kompaniyalar. Ish bozorida Next.js'ni biladiganlar juda qadrlanadi.

0.4. Kerakli vositalarni o'rnatish

Yozishni boshlashdan oldin kompyuteringizga 3 ta narsa o'rnatamiz.

1) Node.js (versiya 20.9 yoki yuqori)

Node.js nima? Odatda JavaScript faqat brauzerda ishlaydi. Node.js — JavaScript'ni brauzersiz, to'g'ridan-to'g'ri kompyuterda ishlatish imkonini beradigan dastur. Next.js ishlashi uchun shart.

O'rnatish: 1. nodejs.org saytiga kiring. 2. **LTS** (Long Term Support — barqaror) versiyasini yuklab oling. Versiya 20.9 dan yuqori bo'lsin. 3. Yuklangan faylni ochib, "Next → Next → Install" qilib o'rnatish.

Tekshirish: Terminal (pastda tushuntiramiz) ochib, quyidagini yozing:

```
node --version
```



Agar `v20.x.x` yoki `v22.x.x` chiqsa — hammasi joyida. Node.js bilan birga **npm** ham o'rnatiladi (bu — boshqalar yozgan tayyor kodlarni yuklab oladigan vosita).

2) VS Code (kod yozadigan dastur)

Kod yozadigan dastur (muharrir) kerak. Eng mashhuri — **Visual Studio Code (VS Code)**, bepul.

1. code.visualstudio.com dan yuklab oling.
2. O'rnatish.
3. Foydali kengaytmalar (Extensions) o'rnatib oling (chap tomondagi kvadratchalar ikonkasi):
4. **ESLint** — xatolarni topadi.
5. **Prettier** — kodni chiroyli formatlaydi.
6. **Tailwind CSS IntelliSense** — keyinroq kerak bo'ladi.

3) Terminal (kompyuterga buyruq beradigan oyna)

Terminal — bu sichqonchasiz, faqat matn bilan kompyuterga buyruq beradigan oyna. Boshida qo'rqinchli ko'rinadi, lekin tez o'rganasiz.

- **VS Code ichida ochish:** Yuqori menyudan `Terminal → New Terminal`.

Eng kerakli buyruqlar:

Buyruq	Nima qiladi
<code>pwd</code>	Hozir qaysi papkadaman? (joriy joylashuv)
<code>ls</code>	Shu papkada qanday fayllar bor?
<code>cd papka-nomi</code>	Papka ichiga kirish
<code>cd ..</code>	Bir qadam orqaga (chiqish)
<code>mkdir papka-nomi</code>	Yangi papka yaratish
<code>clear</code>	Oynani tozalash

Eslatma: Buyruqlarni hozir yodlash shart emas. Kerak bo'lganda qaytib qarang. Bir necha kunda o'zi yodlanib ketadi.



0-BOB MASALALARI (tushunish savollari)

Bu bob nazariy bo'lgani uchun masalalar — fikrlash va o'rnatish bo'yicha. Javoblarni daftaringizga yozing.

1. O'z so'zlaringiz bilan tushuntiring: "browser", "server", "request", "response" nima?
2. Restoran misolida ofitsiant — bu nima edi? Oshxona-chi?
3. Frontend bilan backend o'rtasidagi farqni misol bilan ayting.
4. HTML, CSS, JavaScript — har biri uying qaysi qismiga o'xshaydi?
5. React bilan Next.js o'rtasida qanday bog'liqlik bor? Qaysi biri "ustida" qurilgan?
6. Nega sof React yetmaydi va Next.js kerak bo'ladi? Kamida 2 ta sabab ayting.
7. Node.js nima va nega kerak?
8. npm nima ish qiladi?
9. Node.js'ni o'rnatish va terminalda `node --version` natijasini yozib oling.
10. VS Code'ni o'rnatish va undagi "Extensions" oynasini toping.
11. VS Code'da ESLint va Prettier kengaytmalarini o'rnatish.
12. Terminalni VS Code ichida oching.
13. Terminalda `pwd` yozing — qanday natija chiqdi? Bu nimani anglatadi?
14. Terminalda `mkdir mening-birinchi-papkam` yozib, yangi papka yarating.
15. `cd mening-birinchi-papkam` bilan papka ichiga kiring, keyin `cd ..` bilan chiqing.
16. `ls` buyrug'i nima ko'rsatadi? Sinab ko'ring.
17. SEO nima degani? Nega Next.js bunda yaxshi?
18. "Freymvork" so'zining ma'nosini o'z so'zingiz bilan tushuntiring.
19. Internetda eng kamida 5 ta Next.js bilan qurilgan sayt nomini toping (internetdan qidiring).
20. Kompyuter nega "ahmoq" deyiladi? Choy damlash misolida tushuntiring.
21. `clear` buyrug'i nima qiladi?
22. Next.js'ni kim yaratgan?

1-BOB. JavaScript asoslari

JavaScript — bizning butun safarning poydevori. React va Next.js — bu JavaScript. Shuning uchun bu bobni puxta o'zlashtirish juda muhim. Shoshilmang.

Qayerda yozaman? Tez sinash uchun: brauzeringizda `F12` tugmasini bosib → `Console` bo'limini oching → o'sha yerga JavaScript yozib, Enter bosib. Yoki playcode.io kabi onlayn muharrirdan foydalaning.

1.1. O'zgaruvchilar (Variables) — ma'lumot saqlash qutilari

O'zgaruvchi — bu nomlangan **quti**. Unga biror qiymat (son, matn) solib qo'yamiz, keyin nomi orqali ishlatamiz.

```
let ism = "Oqil";  
const tugilganYil = 1998;
```

- `let` — qiymati **keyin o'zgarishi** mumkin bo'lgan quti.
- `const` — qiymati **o'zgarmaydigan** quti (constant — doimiy).

```
let yosh = 25;  
yosh = 26;           // ✅ Mumkin, chunki "let"  
console.log(yosh); // 26  
  
const pi = 3.14;  
pi = 3.15;           // ❌ XATO! "const"ni o'zgartirib bo'lmaydi
```

`console.log(...)` — bu eng ko'p ishlatadigan buyrug'ingiz. U qavs ichidagi narsani ekranga (konsolga) chiqaradi. Dasturchining "ko'zi" — kod nima qilayotganini ko'rish uchun.

Qoida: Doim `const` ishlatib. Faqat qiymat o'zgarishi kerak bo'lsa `let` ga o'ting. Bu kodni xavfsizroq qiladi.

var haqida: Eski kodlarda `var` ko'rasiz. Uni ishlatmang — eskirgan va muammoli.

Nomlash qoidalari: - Lotin harf bilan boshlanadi: `ism`, `userName`, `_temp`. - Bo'sh joy yo'q. Ko'p so'zli nom uchun **camelCase**: `firstName`, `totalPrice`. - Mazmunli nom bering: `x` emas, `userAge`.

1.2. Ma'lumot turlari (Data Types)

JavaScript'da bir necha xil ma'lumot bor:

```
// 1. String (matn) — qo'shtirnoq yoki bittirnoq ichida  
let ism = "Aziza";
```

```
let shahar = 'Toshkent';

// 2. Number (son) – butun yoki kasr
let yosh = 30;
let narx = 19.99;

// 3. Boolean (ha/yo'q) – faqat true yoki false
let tirikmi = true;
let uxlayaptimi = false;

// 4. null – "ataylab bo'sh" qiymat
let mukofot = null;

// 5. undefined – "hali qiymat berilmagan"
let sirli;
console.log(sirli); // undefined
```

Turini bilish uchun `typeof` ishlatamiz:

```
console.log(typeof "salom"); // "string"
console.log(typeof 42);      // "number"
console.log(typeof true);    // "boolean"
```

1.3. String'lar bilan ishlash

Matnlarni birlashtirish (ulash):

```
let ism = "Oqil";
let salom = "Salom, " + ism + "!"; // "Salom, Oqil!"
```

Lekin zamonaviy va qulay usul — **template literals** (orqaga egilgan tirnoq ``` ichida, `${}` bilan):

```
let ism = "Oqil";
let yosh = 25;
let xabar = `Mening ismim ${ism}, yoshim ${yosh}.`;
console.log(xabar); // "Mening ismim Oqil, yoshim 25."
```

``` belgisi klaviaturada `Esc` tugmasi tagida (1 chap tomonida) bo'ladi.

Foydali string metodlari:

```
let matn = "Salom Dunyo";
console.log(matn.length); // 11 (uzunligi)
console.log(matn.toUpperCase()); // "SALOM DUNYO"
console.log(matn.toLowerCase()); // "salom dunyo"
```

```
console.log(matn.includes("Dunyo")); // true (ichida bormi?)
console.log(matn.replace("Dunyo", "Olam")); // "Salom Olam"
```

## 1.4. Sonlar va matematik amallar

```
let a = 10;
let b = 3;

console.log(a + b); // 13 (qo'shish)
console.log(a - b); // 7 (ayirish)
console.log(a * b); // 30 (ko'paytirish)
console.log(a / b); // 3.33... (bo'lish)
console.log(a % b); // 1 (qoldiq - 10 ni 3 ga bo'lgandagi qoldiq)
console.log(a ** 2); // 100 (daraja - 10 ning kvadrati)
```

% (qoldiq) juda foydali — masalan, sonning juft yoki toqligini tekshirishda:

```
let son = 8;
console.log(son % 2); // 0 → juft son (qoldiq 0)
```

Qisqartirilgan amallar:

```
let ball = 10;
ball += 5; // ball = ball + 5 → 15
ball -= 3; // 12
ball++; // 13 (1 ga oshirish)
ball--; // 12 (1 ga kamaytirish)
```

## 1.5. Boolean, taqqoslash va mantiq

**Taqqoslash** natijasi har doim `true` yoki `false`:

```
console.log(5 > 3); // true
console.log(5 < 3); // false
console.log(5 >= 5); // true (katta yoki teng)
console.log(5 === 5); // true (TENG)
console.log(5 === "5"); // false (son va matn teng emas!)
console.log(5 !== 3); // true (teng EMAS)
```

**MUHIM:** Tenglikni tekshirishda doim `===` (uchta belgi) ishlatib, `==` (ikkita) emas. `===` turni ham tekshiradi va xatolardan saqlaydi.

**Mantiqiy operatorlar:**

```
let yosh = 20;

// && (VA) – ikkala shart ham true bo'lishi kerak
console.log(yosh > 18 && yosh < 65); // true

// || (YOKI) – kamida bittasi true bo'lsa yetadi
console.log(yosh < 18 || yosh > 65); // false

// ! (EMAS) – teskarisi
console.log(!true); // false
```

## 1.6. Shartlar (if / else) — qaror qabul qilish

Dastur har doim ham bir xil ishlamaydi — ba'zan u **qaror** qabul qilishi kerak.

```
let yosh = 16;

if (yosh >= 18) {
 console.log("Siz voyaga yetgansiz");
} else {
 console.log("Siz hali balog'atga yetmagansiz");
}

// Natija: "Siz hali balog'atga yetmagansiz"
```

Ko'p variantli shart (else if):

```
let ball = 75;

if (ball >= 90) {
 console.log("A'lo");
} else if (ball >= 70) {
 console.log("Yaxshi");
} else if (ball >= 50) {
 console.log("Qoniqarli");
} else {
 console.log("Qoniqarsiz");
}

// Natija: "Yaxshi"
```

**Ternar operator** — qisqa if/else (React'da juda ko'p ishlatiladi):

```
let yosh = 20;
let holat = yosh >= 18 ? "kattalar" : "bolalar";
console.log(holat); // "kattalar"
// Sintaksis: shart ? (true bo'lsa) : (false bo'lsa)
```

## 1.7. Massivlar (Arrays) — ro'yxatlar

**Massiv** — bu **tartiblangan ro'yxat**. Bir nechta qiymatni bitta o'zgaruvchida saqlaydi.

```
let mevalar = ["olma", "banan", "uzum"];
```

Elementga **indeks** orqali murojaat qilamiz. **Indeks 0 dan boshlanadi!**

```
console.log(mevalar[0]); // "olma" (birinchi)
console.log(mevalar[1]); // "banan" (ikkinchi)
console.log(mevalar[2]); // "uzum" (uchinchi)
console.log(mevalar.length); // 3 (nechta element bor)
```

Foydali massiv metodlari:

```
let sonlar = [1, 2, 3];

sonlar.push(4); // oxiriga qo'shish → [1, 2, 3, 4]
sonlar.pop(); // oxirgisini olib tashlash → [1, 2, 3]
sonlar.unshift(0); // boshiga qo'shish → [0, 1, 2, 3]
console.log(sonlar.includes(2)); // true (ichida bormi?)
console.log(sonlar.indexOf(2)); // 2 (qaysi indeksda?)
```

### Massivlar bilan ishlashning eng muhim 3 metodi

Bular React'da har kuni ishlatiladi. Yaxshi tushunib oling.

**.map()** — har bir elementni o'zgartirib, **YANGI** massiv qaytaradi

```
let sonlar = [1, 2, 3];
let kvadratlar = sonlar.map(son => son * son);
console.log(kvadratlar); // [1, 4, 9]
// "Har bir sonni olib, kvadratga ko'tar va yangi ro'yxat tuz"
```

**.filter()** — shartga mos elementlarni saralab, **YANGI** massiv qaytaradi

```
let sonlar = [1, 2, 3, 4, 5, 6];
let juftlar = sonlar.filter(son => son % 2 === 0);
console.log(juftlar); // [2, 4, 6]
// "Faqat juft sonlarni ajratib ol"
```

**.find()** — shartga mos **BIRINCHI** elementni qaytaradi

```
let sonlar = [5, 12, 8, 20];
let birinchiKatta = sonlar.find(son => son > 10);
console.log(birinchiKatta); // 12
```

`.reduce()` — barcha elementlarni bitta qiymatga "yig'adi" (masalan, yig'indi):

```
let sonlar = [10, 20, 30];
let yigindi = sonlar.reduce((jami, son) => jami + son, 0);
console.log(yigindi); // 60
```

## 1.8. Obyektlar (Objects) — bog'liq ma'lumotlar to'plami

Massiv — ro'yxat ( ["olma", "banan"] ). **Obyekt** esa — bir narsaning **xususiyatlari** to'plami. Real hayotdagi "narsa"ni tasvirlaydi.

```
let foydalanuvchi = {
 ism: "Oqil",
 yosh: 25,
 shahar: "Toshkent",
 faolmi: true
};
```

Har bir qism — **kalit: qiymat** ( key: value ) juftligi. Qiymatga **nuqta** orqali murojaat qilamiz:

```
console.log(foydalanuvchi.ism); // "Oqil"
console.log(foydalanuvchi.yosh); // 25

foydalanuvchi.yosh = 26; // o'zgartirish
foydalanuvchi.kasb = "Dasturchi"; // yangi xususiyat qo'shish
```

**Obyektlar massivi** — eng ko'p uchraydigan ma'lumot ko'rinishi (masalan, foydalanuvchilar ro'yxati):

```
let foydalanuvchilar = [
 { id: 1, ism: "Ali", yosh: 20 },
 { id: 2, ism: "Vali", yosh: 25 },
 { id: 3, ism: "Guli", yosh: 30 }
];

// 25 dan katta yoshdagilarni topish:
let kattalar = foydalanuvchilar.filter(u => u.yosh > 22);
console.log(kattalar); // Vali va Guli

// Faqat ismlar ro'yxatini olish:
let ismlar = foydalanuvchilar.map(u => u.ism);
console.log(ismlar); // ["Ali", "Vali", "Guli"]
```

## 1.9. Funktsiyalar (Functions) — qayta ishlatiladigan kod

**Funksiya** — bu biror vazifani bajaradigan **kod bo'lagi**, unga nom beriladi va kerak bo'lganda chaqiriladi. Bir marta yozasiz, ko'p marta ishlatasiz.

Tasavvur qiling, "choy damlash" retsepti. Bir marta yozib qo'yasiz, keyin har gal "choy damla" deysiz.

```
function salomBer(ism) {
 return `Salom, ${ism}!`;
}

console.log(salomBer("Oqil")); // "Salom, Oqil!"
console.log(salomBer("Aziza")); // "Salom, Aziza!"
```

- `ism` — **parametr** (funksiyaga kiradigan ma'lumot).
- `return` — funksiya **qaytaradigan natija**.

Bir necha parametr:

```
function qoshish(a, b) {
 return a + b;
}

console.log(qoshish(3, 5)); // 8
```

## Arrow funksiyalar (zamonaviy yozuv)

React'da ko'proq **arrow** (o'q) ko'rinishi ishlatiladi. Bu xuddi shu narsa, faqat qisqaroq:

```
// Oddiy funksiya:
function qoshish(a, b) {
 return a + b;
}

// Arrow funksiya (aynan shu ish):
const qoshish = (a, b) => {
 return a + b;
};

// Agar faqat return bo'lsa, yanada qisqa:
const qoshish = (a, b) => a + b;

console.log(qoshish(2, 3)); // 5
```

Yuqorida `.map()`, `.filter()` ichida ishlatgan `son => son * son` — bu ham arrow funksiya!

## 1.10. Destructuring va spread — qulay yozuvlar

React'da bularsiz yashab bo'lmaydi. Yaxshi tushunib oling.

**Destructuring (ajratib olish)** — obyekt yoki massivdan qiymatlarni tez ajratib olish:

```
let foydalanuvchi = { ism: "Oqil", yosh: 25 };

// Eski usul:
let ism = foydalanuvchi.ism;
let yosh = foydalanuvchi.yosh;

// Destructuring (qisqa):
let { ism, yosh } = foydalanuvchi;
console.log(ism, yosh); // "Oqil" 25
```

Massiv uchun:

```
let ranglar = ["qizil", "yashil", "ko'k"];
let [birinchi, ikkinchi] = ranglar;
console.log(birinchi); // "qizil"
```

**Spread ( ... )** — massiv/obyektni "yoyish", nusxa olish:

```
// Massivlarni birlashtirish:
let a = [1, 2];
let b = [3, 4];
let hammasi = [...a, ...b]; // [1, 2, 3, 4]

// Obyektga nusxa olib, ustiga qo'shish:
let user = { ism: "Oqil", yosh: 25 };
let yangiUser = { ...user, shahar: "Toshkent" };
// { ism: "Oqil", yosh: 25, shahar: "Toshkent" }
```

**Nega muhim?** React'da ma'lumotni o'zgartirganda, "eskisini buzmasdan nusxa olib o'zgartirish" qoida. Spread aynan shu uchun kerak.

## 1.11. Tsikllar (Loops) — takrorlash

Bir xil ishni ko'p marta qilish kerak bo'lsa:

```
// for tsikli - 5 marta salom aytadi
for (let i = 1; i <= 5; i++) {
 console.log(`${i}-salom`);
}
// 1-salom, 2-salom, ... 5-salom
```

Massiv bo'ylab yurish ( `for...of` ):

```
let mevalar = ["olma", "banan", "uzum"];
for (let meva of mevalar) {
 console.log(meva);
}
// olma, banan, uzum
```

React'da ko'pincha tsikl o'rniga `.map()` ishlatamiz, lekin `for` ni bilish shart.

## 1.12. Maxsus operatorlar (React'da ko'p kerak)

**Optional chaining (?.)** — obyekt bor-yo'qligini tekshirib, xatosiz murojaat qilish:

```
let user = { ism: "Oqil", manzil: { shahar: "Toshkent" } };
console.log(user?.manzil?.shahar); // "Toshkent"

let user2 = {};
console.log(user2?.manzil?.shahar); // undefined (xato BERMAYDI!)
// `?.` bo'lmasa, bu yerda dastur "qulab" tushardi
```

**Nullish operator (??)** — qiymat `null` / `undefined` bo'lsa, zaxira qiymat berish:

```
let ism = null;
let natija = ism ?? "Mehmon";
console.log(natija); // "Mehmon"
```

**&& bilan qisqa ko'rsatish** (React'da elementni shartli ko'rsatishda):

```
let xabarlarSoni = 5;
console.log(xabarlarSoni > 0 && "Sizda yangi xabar bor!");
// "Sizda yangi xabar bor!"
```

## 1.13. Asinxron kod — `async` / `await`

Ba'zi ishlar **vaqt oladi** — masalan, serverdan ma'lumot olish. Dastur javobni kutishi kerak, lekin shu paytda muzlab qolmasligi kerak. Buni **asinxron** kod hal qiladi.

Restoran misolida: ofitsiantga buyurtma berdingiz va kutyapsiz. Lekin shu paytda telefon ko'rib, gaplashib o'tiraverasiz — "muzlab" qolmaysiz. Taom kelganda yeysiz.

```
// Serverdan ma'lumot olish (eng keng tarqalgan misol):
async function malumotOl() {
 const javob = await fetch("https://api.example.com/users");
 const malumot = await javob.json();
}
```

```
console.log(malumot);
}
```

- `async` — "bu funksiya ichida kutish bo'lishi mumkin" degani.
- `await` — "shu ishni kutib tur, tugagach davom et" degani.
- `fetch(...)` — internetdan ma'lumot olib keladigan funksiya.

Hozircha buni to'liq tushunmasangiz ham mayli — 2-qismda ma'lumot olishda chuqur ko'ramiz. Hozir shaklini tanib qo'ying.

## 1.14. Modullar — `import` / `export`

Katta loyihada kodni bir nechta faylga bo'lib yozamiz. Bir fayldagi kodni boshqasida ishlatish uchun **export/import**:

```
// matematika.js fayli
export function qoshish(a, b) {
 return a + b;
}

// app.js fayli
import { qoshish } from "./matematika.js";
console.log(qoshish(2, 3)); // 5
```

- `export` — "bu kodni boshqa fayllar ishlatishi mumkin".
- `import` — "boshqa fayldan kod olib kelaman".

React va Next.js'da har bir fayl yuqorida `import` lar bilan boshlanadi. Bu — odatiy holat.

## 1-BOB MASALALARI (25 ta)

Har bir masalani VS Code yoki brauzer konsolida yozib, natijasini `console.log` bilan tekshiring.

**Oson (1–9):** 1. `ism`, `familiya`, `yosh` o'zgaruvchilarini yarating va `console.log` bilan chiqaring. 2. Doira radiusi `r = 5`. Maydonini hisoblang (`pi * r * r`, `pi = 3.14`). 3. Template literal bilan shunday matn chiqaring: "Mening ismim X, men Y yoshdaman". 4. "javascript" so'zini katta harflarga aylantiring va uzunligini chiqaring. 5. 17 ning juft yoki toqligini `%` bilan aniqlang va `console.log` qiling. 6. `let narx = 100`. Unga 20% chegirma qo'llang va yangi narxni chiqaring. 7. 2 ta sonni taqqoslab, qaysi biri kattaligini `console.log` bilan ayting (`if/else`). 8. `5 === "5"` va `5 == "5"`

natijalarini chiqaring. Farqini izohlang. 9. Ternar operator bilan: agar yosh 18 dan katta bo'lsa "Kattalar", aks holda "Bolalar" chiqaring.

**O'rta (10–18):** 10. `["dushanba", "seshanba", "chorshanba"]` massivi yarating. 2-elementni chiqaring. 11. Massivga `.push()` bilan "payshanba" qo'shing va `length` ni chiqaring. 12. `[1, 2, 3, 4, 5]` massivining har bir elementini 2 ga ko'paytiring (`.map()`). 13. `[10, 25, 3, 40, 7]` dan 20 dan katta sonlarni saralang (`.filter()`). 14. Shu massivdan birinchi 20 dan katta sonni toping (`.find()`). 15. `[100, 200, 300]` ning yig'indisini `.reduce()` bilan hisoblang. 16. Obyekt yarating: `{ ism, yosh, kasb }`. Keyin `kasb` ni o'zgartiring va `shahar` qo'shing. 17. 3 ta foydalanuvchi obyektidan iborat massiv yarating va `.map()` bilan faqat ismlarini ajrating. 18. Shu massivdan 25 yoshdan katta foydalanuvchilarni `.filter()` bilan ajrating.

**Funksiyalar va ilg'or (19–25):** 19. `kvadrat(n)` funksiyasini yozing — sonning kvadratini qaytarsin. `kvadrat(6)` ni sinang. 20. Shu funksiyani **arrow** ko'rinishida qayta yozing. 21. `engKatta(a, b, c)` funksiyasi — 3 sondan eng kattasini qaytarsin. 22. Obyektdan destructuring bilan `ism` va `yosh` ni ajratib oling. 23. 2 ta massivni `spread (...)` bilan birlashtiring. 24. Obyektga `spread` bilan nusxa olib, ustiga yangi maydon qo'shing (eskisini buzmasdan). 25. `for` tsikli bilan 1 dan 10 gacha sonlarni chiqaring, lekin faqat juftlarini (`if + %` bilan).

**Qo'shimcha challenge:** `[5, -3, 0, 12, -8, 7]` massividan: (a) musbatlarni ajrating, (b) ularning yig'indisini toping, (c) har birini kvadratga ko'taring. Hammasini `.filter`, `.reduce`, `.map` bilan qiling.

---

## 2-BOB. React asoslari

Endi JavaScript poydevori bor. React — bu JavaScript yordamida **foydalanuvchi interfeysi** (tugmalar, formalar, ro'yxatlar) yasash vositasi. Next.js to'liq React ustiga qurilgan, shuning uchun bu bob — markaziy.

### 2.1. React nima va nega kerak?

Sof JavaScript bilan sahifani o'zgartirish ancha murakkab va chalkash bo'ladi. React buni **soddalashtiradi** va 2 ta katta g'oyani olib keladi:

1. **Component (komponent)** — sahifani qayta ishlatiladigan **bo'laklarga** bo'lish.
2. **State (holat)** — ma'lumot o'zgarganda, ekran **avtomatik** yangilanishi.

Tasavvur qiling, Lego. Katta narsani kichik, bir xil bo'laklardan yig'asiz. React'da sahifa — Lego, komponentlar — bo'laklar.

## 2.2. Component (komponent) — qurilish g'ishti

**Komponent** — bu HTML qaytaradigan **funksiya**. Eng oddiy komponent:

```
function Salom() {
 return <h1>Salom, Dunyo!</h1>;
}
```

**Muhim qoidalar:** - Komponent nomi **katta harf** bilan boshlanadi: `Salom`, `UserCard` (oddiy funksiyalardan farqi shu). - U **JSX** qaytaradi (pastda tushuntiramiz — HTMLga o'xshash narsa). - Komponentni boshqa joyda HTML tegi kabi ishlatamiz: `<Salom />`.

```
function Sahifa() {
 return (
 <div>
 <Salom />
 <Salom />
 <Salom />
 </div>
);
}
// Ekranda 3 marta "Salom, Dunyo!" chiqadi
```

## 2.3. JSX — HTML ichida JavaScript

**JSX** — JavaScript ichiga HTML yozish imkonini beradigan maxsus sintaksis. Ko'rinishidan HTML, lekin aslida JavaScript.

```
function Profil() {
 const ism = "Oqil";
 const yosh = 25;

 return (
 <div>
 <h1>{ism}</h1>
 <p>Yosh: {yosh}</p>
 <p>Keyingi yil: {yosh + 1}</p>
 </div>
);
}
```

**JSX qoidalari (yodlang):** 1. `{ }` ichida JavaScript yoziladi: `{ism}, {yosh + 1}, {user.ism}`. 2. **Bitta "ota" element** qaytarish kerak. Bir nechta element bo'lsa, `<div>` yoki bo'sh teg `<>...</>` ichiga o'rang:

```
return (
 <>
 <h1>Sarlavha</h1>
 <p>Matn</p>
 </>
);
```

3. **class o'rniga className**: `<div className="quti">`. 4. Teglar **yopilishi** shart: `<img />`, `<br />` (o'z-o'zidan yopiluvchi). 5. CSS uslublari obyekt sifatida: `style={{ color: "red" }}` (ikki qavat qavs!).

## 2.4. Props — komponentga ma'lumot uzatish

**Props** (properties — xususiyatlar) — komponentga **tashqaridan** ma'lumot berish usuli. Funksiyaga argument bergandek.

Choy retsepti misolida: "choy damla" funksiyasiga "qancha qand?" deb props berasiz. Bittasiga 2 qand, boshqasiga 0 qand.

```
// Komponent props qabul qiladi:
function UserCard(props) {
 return (
 <div>
 <h2>{props.ism}</h2>
 <p>Yosh: {props.yosh}</p>
 </div>
);
}

// Uni ishlatganda props beramiz:
function Sahifa() {
 return (
 <div>
 <UserCard ism="Ali" yosh={20} />
 <UserCard ism="Vali" yosh={25} />
 </div>
);
}
```

**Destructuring bilan chiroyliroq** (ko'pincha shunday yoziladi):

```
function UserCard({ ism, yosh }) {
 return (
 <div>
```

```

 <h2>{ism}</h2>
 <p>Yosh: {yosh}</p>
 </div>
);
}

```

**Muhim:** Props **faqat o'qish uchun**. Komponent o'ziga kelgan propsni o'zgartira olmaydi.

## 2.5. Ro'yxatlarni ko'rsatish (Lists) — `.map()` qaytib keldi!

Ma'lumotlar ro'yxatini ekranga chiqarish — eng ko'p uchraydigan vazifa. Bu yerda `.map()` ishga tushadi:

```

function MevalarRoxxati() {
 const mevalar = ["Olma", "Banan", "Uzum"];

 return (

 {mevalar.map((meva) => (
 <li key={meva}>{meva}
))}

);
}

```

**key nima?** Har bir ro'yxat elementiga **noyob key** berish kerak. React shu orqali qaysi element o'zgarгани tezda topadi. Odatda `id` ishlatiladi:

```

function UserList() {
 const users = [
 { id: 1, ism: "Ali" },
 { id: 2, ism: "Vali" },
 { id: 3, ism: "Guli" }
];

 return (

 {users.map((user) => (
 <li key={user.id}>{user.ism}
))}

);
}

```

**Qoida:** `key` noyob bo'lishi kerak. Massiv indeksini (`map((u, i) => ... key={i})`) iloji boricha ishlatmang — `id` borligi yaxshiroq.

## 2.6. Shartli ko'rsatish (Conditional Rendering)

Ba'zan elementni **shartga qarab** ko'rsatamiz:

```
function Salom({ kirganmi }) {
 return (
 <div>
 {kirganmi ? (
 <h1>Xush kelibsiz!</h1>
) : (
 <h1>Iltimos, tizimga kiring</h1>
)}
 </div>
);
}
```

**&&** bilan "bor bo'lsa ko'rsat":

```
function Xabarlar({ son }) {
 return (
 <div>
 <h1>Bosh sahifa</h1>
 {son > 0 && <p>Sizda {son} ta yangi xabar bor</p>}
 </div>
);
}
// son 0 bo'lsa, hech narsa ko'rsatilmaydi
```

## 2.7. State (holat) va **useState** — Reactning yuragi

Mana React'ning eng muhim qismi. **State** — bu komponentning vaqt o'tishi bilan **o'zgarib turadigan ma'lumoti**. State o'zgarganda — React ekranni **avtomatik qayta chizadi**.

Misol: tugmani bosgan sayin oshib boradigan hisoblagich. "Necha marta bosildi" — bu state.

```
import { useState } from "react";

function Hisoblagich() {
 const [son, setSon] = useState(0);

 return (
 <div>
 <p>Siz {son} marta bosdingiz</p>
 <button onClick={() => setSon(son + 1)}>
 Bosing
 </button>
 </div>
);
}
```

```

 </div>
);
}

```

**Buni qatorma-qator tushunamiz:** - `import { useState } from "react";` — `useState` ni olib kelamiz. - `const [son, setSon] = useState(0);` — bu eng muhim qator: - `son` — joriy qiymat (boshlang'ich `0`). - `setSon` — qiymatni **o'zgartiradigan** funksiya. - `useState(0)` — boshlang'ich qiymat `0`. - `onClick={() => setSon(son + 1)}` — tugma bosilganda `son` ni `1` ga oshir. - `setSon` chaqirilgach, React **avtomatik** ekranni yangilaydi.

**OLTIN QOIDA:** State'ni hech qachon to'g'ridan-to'g'ri o'zgartirmang (`son = son + 1` ❌). Doim `setSon(...)` funksiyasidan foydalaning ✅. Aks holda React o'zgarishni "sezmaydi".

State faqat son emas — har qanday tur bo'lishi mumkin:

```

const [ism, setIsm] = useState(""); // matn
const [tayyormi, setTayyormi] = useState(false); // boolean
const [royxat, setRoyxat] = useState([]); // massiv
const [user, setUser] = useState({ ism: "", yosh: 0 }); // obyekt

```

## 2.8. Hodisalar (Events) — foydalanuvchi bilan muloqot

Tugma bosish, matn yozish kabi **hodisalar**ga javob beramiz:

```

function Tugmalar() {
 function salomBer() {
 alert("Salom!");
 }

 return (
 <div>
 {/* Usul 1: funksiya nomini berish */}
 <button onClick={salomBer}>Salom ayt</button>

 {/* Usul 2: to'g'ridan-to'g'ri arrow funksiya */}
 <button onClick={() => alert("Xayr!")}>Xayr ayt</button>
 </div>
);
}

```

**Diqqat:** `onClick={salomBer}` ✅ (qavssiz!), `onClick={salomBer()}` ❌ (qavs bilan — bu funksiyani darrov chaqirib yuboradi, xato).

Eng ko'p ishlatadigan hodisalar: `onClick` (bosish), `onChange` (o'zgarish), `onSubmit` (forma yuborish).

## 2.9. Formalar (Controlled Inputs)

React'da input maydonlarini state bilan **boshqaramiz**. Bu "controlled input" deyiladi:

```
import { useState } from "react";

function IsmFormasi() {
 const [ism, setIs] = useState("");

 return (
 <div>
 <input
 type="text"
 value={ism}
 onChange={(e) => setIs(e.target.value)}
 placeholder="Ismingizni kiriting"
 />
 <p>Salom, {ism || "mehmon"}!</p>
 </div>
);
}
```

**Tushuntirish:** - `value={ism}` — input'ning qiymati state'ga bog'langan. - `onChange={ (e) => setIs(e.target.value)}` — har bir harf yozilganda state yangilanadi. - `e.target.value` — input'ga yozilgan joriy matn.

Foydalanuvchi har harf yozganda, state yangilanadi va ekran shu zahoti yangilanadi.

## 2.10. `useEffect` — "yon ta'sirlar" (side effects)

Ba'zan komponent ekranga chiqqanda biror ish bajarish kerak bo'ladi — masalan, serverdan ma'lumot olish yoki sarlavhani o'zgartirish. Buni `useEffect` bilan qilamiz.

```
import { useState, useEffect } from "react";

function Soat() {
 const [vaqt, setVaqt] = useState(new Date());

 useEffect(() => {
 // Bu kod komponent ekranga chiqqach ishga tushadi
 const interval = setInterval(() => {
 setVaqt(new Date());
 }, 1000);

 // "Tozalash" — komponent yo'qolganda intervalni to'xtatadi
 return () => clearInterval(interval);
 }, []); // [] — faqat bir marta ishlasin
```

```
return <p>Hozir: {vaqt.toLocaleTimeString()}</p>;
}
```

**useEffect ning oxiridagi [] (dependency array) muhim:** - [] (bo'sh) — effekt faqat **bir marta**, komponent paydo bo'lganda ishlaydi. - [son] — son o'zgarganda **har gal** qayta ishlaydi. - (massiv yo'q) — har renderda ishlaydi (kamdan-kam kerak).

**Eslatma:** Next.js'da ma'lumot olishni ko'pincha `useEffect` siz, to'g'ridan-to'g'ri Server Component'da qilamiz (2-qismda ko'ramiz). Lekin `useEffect` ni bilish baribir shart.

## 2.11. Komponentlarni birlashtirish (Composition)

Katta sahifa kichik komponentlardan yig'iladi. Bu — React falsafasi:

```
function Header() {
 return <header><h1>Mening Saytim</h1></header>;
}

function Footer() {
 return <footer><p>© 2026</p></footer>;
}

function Asosiy() {
 return <main><p>Asosiy kontent shu yerda</p></main>;
}

// Hammasini birlashtiramiz:
function App() {
 return (
 <div>
 <Header />
 <Asosiy />
 <Footer />
 </div>
);
}
```

**children prop** — komponent ichiga boshqa elementlarni "joylash":

```
function Quti({ children }) {
 return <div className="quti">{children}</div>;
}

function Sahifa() {
 return (
 <Quti>
 <h2>Bu matn quti ichida</h2>
 <p>Bu ham</p>
 </Quti>
);
}
```

```
);
}
```

children — `<Quti>` va `</Quti>` orasiga yozilgan hamma narsa.

---

## 2-BOB MASALALARI (22 ta)

Bularni sinash uchun [react.new](https://react.new) yoki onlayn React muharririda yozing. (3-bobdan keyin Next.js loyihasida ham sinaysiz.)

**Komponent va JSX (1–6):** 1. `Salom` komponentini yarating — `<h1>Salom!</h1>` qaytarsin. 2. `Footer` komponentini yarating — ichida sayt nomi va yil bo'lsin. 3. Bitta `App` ichida `Salom` ni 3 marta ishlatib chiqaring. 4. JSX ichida 2 ta sonni qo'shib, natijasini `<p>{...}</p>` da ko'rsating. 5. `<div>` ichiga `style={{ color: "blue" }}` qo'shib, matnni ko'k qiling. 6. Bir komponentda `<>...</>` (fragment) ishlatib, 3 ta elementni qaytaring.

**Props (7–11):** 7. `UserCard({ ism, yosh })` komponentini yarating va 3 xil user bilan chaqiring. 8. `Tugma({ matn })` komponenti — propsdan kelgan matnni tugmada ko'rsatsin. 9. `Salom({ ism })` — agar `ism` berilmasa "Mehmon" ko'rsatadigan qiling (`||` bilan). 10. `Narx({ qiymat })` komponenti — narxni `{qiymat}` so'm ko'rinishida chiqarsin. 11. `Avatar({ rasmUrl, ism })` — `<img>` va `ism` ko'rsatadigan komponent yarating.

**Ro'yxatlar va shartlar (12–16):** 12. 5 ta shahar massivini `.map()` bilan `<ul><li>` ro'yxatda chiqaring (`key` bilan). 13. 4 ta user obyektidan iborat massivni `.map()` bilan kartochkalar qilib chiqaring. 14. Shu ro'yxatdan faqat 25 yoshdan kattalarni ko'rsating (`.filter() + .map()`). 15. `Salom({ kirganmi })` — `kirganmi` `true` bo'lsa "Xush kelibsiz", aks holda "Kiring" (ternar). 16. `Xabar({ son })` — son 0 dan katta bo'lsagina xabar ko'rsatadigan qiling (`&&`).

**State va hodisalar (17–22):** 17. `Tugma` bosilganda 1 ga oshadigan hisoblagich (`useState`) yarating. 18. Unga "Kamaytirish" tugmasini ham qo'shing (1 ga kamaytirsin). 19. "Reset" tugmasini qo'shing — hisobni 0 ga qaytarsin. 20. `Input` yarating: yozgan matningiz pastda jonli ko'rinib tursin (controlled input). 21. "Yashirish/Ko'rsatish" tugmasi — bosilganda matnni yashirsin/ko'rsatsin (boolean state). 22. `useEffect` bilan: komponent yuklanganda konsolga "Komponent yuklandi" yozsin (bir marta).

**Qo'shimcha challenge:** Oddiy **vazifalar ro'yxati (To-Do)** yasang: input + "Qo'shish" tugmasi. Yozib "Qo'shish" bosilganda, vazifa pastdagi ro'yxatga qo'shilsin. (Maslahat: `useState([])` va `setRoyxat([...royxat, yangiVazifa])`.)

---

## 3-BOB. Next.js bilan birinchi qadamlar

Tabriklaymiz! JavaScript va React asoslari sizda bor. Endi asosiy maqsadimiz — **Next.js**ga o'tamiz. Bu bobda birinchi Next.js loyihangizni yaratasiz va uning tuzilishini tushunasiz.

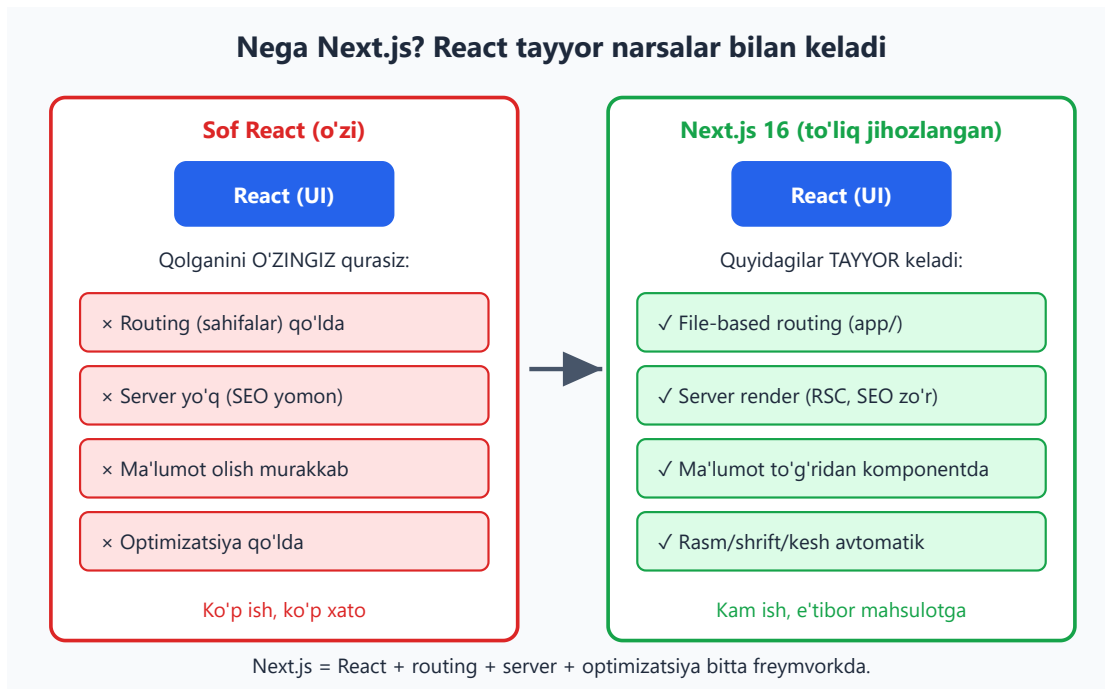
### 3.1. Nega Next.js? (React'dan farqi)

Sof React bilan ishlaganda bir nechta narsani o'zingiz qurishingiz kerak edi. Next.js ularni **tayyor** beradi:

Vazifa	Sof React	Next.js
Sahifalar (routing)	O'zingiz qurasiz	Papka yaratasiz — tayyor
Serverdan ma'lumot	Murakkab ( <code>useEffect</code> + <code>fetch</code> )	To'g'ridan-to'g'ri komponentda
SEO (Google'da chiqish)	Yomon	Zo'r (server tayyorlaydi)
Rasm/shrift optimizatsiyasi	Qo'lda	Avtomatik
Backend (API)	Alohida server kerak	Bitta loyihada

Bitta jumlada: **Next.js — bu "to'liq jihozlangan React"**.

Quyidagi diagramma sof React bilan Next.js o'rtasidagi farqni — nimani o'zingiz qurishingiz va nima tayyor kelishini — yonma-yon ko'rsatadi:



## 3.2. Birinchi loyihani yaratish

Terminalni oching (VS Code ichida `Terminal` → `New Terminal`) va quyidagini yozing:

```
npx create-next-app@latest
```

`npx create-next-app` — bu Next.js loyahasini avtomatik yaratib beradigan vosita. U sizdan bir necha savol so'raydi. Yangi boshlovchi uchun tavsiya:

```
✓ What is your project named? ... birinchi-app
✓ Would you like to use TypeScript? ... Yes ← Ha (xatolardan saqlaydi)
✓ Would you like to use ESLint? ... Yes ← Ha
✓ Would you like to use Tailwind CSS? ... Yes ← Ha (oson styling)
✓ Would you like your code inside a `src/` directory? ... No ← Yo'q (soddaroq)
✓ Would you like to use App Router? ... Yes ← HA! (zamonaviy usul)
✓ Would you like to use Turbopack? ... Yes ← Ha (tezroq)
```

**TypeScript haqida:** Bu — JavaScript'ning "xatolarni oldindan ushlaydigan" versiyasi. Boshida `.tsx` fayllarda biroz qo'shimcha yozuv (`: string` kabi) ko'rasiz, lekin bu sizni katta xatolardan asraydi. Hozir qo'rqmang — sekin o'rganib ketasiz.

Tayyor bo'lgach, loyiha papkasiga kiring va ishga tushiring:

```
cd birinchi-app
npm run dev
```

Terminalda shunday yozuv chiqadi:

```
✓ Ready in 2s
o Local: http://localhost:3000
```

Brauzeringizda `http://localhost:3000` manzilini oching. Next.js'ning xush kelibsiz sahifasini ko'rasiz! 🎉

`localhost:3000` nima? `localhost` — bu sizning **o'z kompyuteringiz**. `3000` — port (eshik) raqami. Ya'ni sayt hozir faqat sizning kompyuteringizda ishlamoqda, internetda emas. Internetga chiqarish (deployment) — 4-qismda.

### 3.3. Loyiha tuzilishi (papkalar va fayllar)

VS Code'da loyihani oching (`File → Open Folder → birinchi-app`). Quyidagi muhim papka/fayllarni ko'rasiz:

birinchi-app/	
├─ app/	← ENG MUHIM papka. Barcha sahifalar shu yerda
│   ├─ layout.tsx	← Umumiy "qoplama" (har sahifada ko'rinadi)
│   ├─ page.tsx	← Bosh sahifa ("/" manzili)
│   └─ globals.css	← Butun saytga tegishli CSS
├─ public/	← Rasmlar, statik fayllar (logo, favicon)
├─ node_modules/	← Kutubxonalar (TEGMANG, avtomatik)
├─ package.json	← Loyiha sozlamalari, kutubxonalar ro'yxati
├─ next.config.ts	← Next.js sozlamalari
└─ tsconfig.json	← TypeScript sozlamalari

**Eng muhim 2 ta fayl:**

`app/page.tsx` — bu **bosh sahifa**. Brauzerda `/` ochilganda shu ko'rinadi.

`app/layout.tsx` — bu **umumiy qoplama**. Har bir sahifaning ustki/ostki qatlami (masalan, menyu, footer) shu yerda — barcha sahifalarda bir xil ko'rinadi.

### 3.4. Birinchi sahifani o'zgartirish

`app/page.tsx` faylini oching. Ichidagi hamma narsani o'chirib, quyidagini yozing:

```
export default function Home() {
 return (
 <div>
 <h1>Salom, men Next.js o'rganyapman!</h1>
 </div>
)
}
```

```
 <p>Bu mening birinchi sahifam.</p>
 </div>
);
}
```

Faylni saqlang ( `Ctrl+S` / `Cmd+S` ). Brauzerga qaytsangiz — sahifa **avtomatik** yangilanganini ko'rasiz! Bu **Hot Reload** (issiq qayta yuklash) deyiladi — saqlaganingiz zahoti o'zgarish ko'rinadi.

**export default** nima? Bu "bu faylning asosiy komponenti shu" degani. Next.js har bir `page.tsx` dan aynan shu `export default` qilingan komponentni oladi va ko'rsatadi.

Komponent — bu xuddi 2-bobdagi React komponenti. Next.js sahifalari — oddiy React komponentlari, xolos!

## 3.5. Layout (umumiy qoplama)

`app/layout.tsx` ni ochsangiz, taxminan shunday ko'rinadi:

```
import './globals.css';

export const metadata = {
 title: "Mening saytim",
 description: "Next.js bilan yasalgan",
};

export default function RootLayout({
 children,
}): {
 children: React.ReactNode;
}) {
 return (
 <html lang="uz">
 <body>{children}</body>
 </html>
);
}
```

**Tushuntirish:** - `{children}` — bu yerga **joriy sahifa** (`page.tsx`) joylashadi.

Esingizdami, 2-bobdagi `children`? Aynan o'sha. - `metadata` — sahifa sarlavhasi va tavsifi (brauzer tabida va Google'da ko'rinadi). SEO uchun muhim. - Layout har bir sahifada **bir xil** bo'ladi. Masalan, menyu yoki footer'ni shu yerga qo'ysangiz, u barcha sahifalarda chiqadi.

**Misol:** Har sahifada chiqadigan oddiy menyu qo'shamiz:

```
export default function RootLayout({ children }: { children:
React.ReactNode }) {
 return (
 <html lang="uz">
 <body>
 <nav style={{ padding: "16px", background: "#eee" }}>
 Mening Saytim
 </nav>
 <main style={{ padding: "16px" }}>{children}</main>
 <footer style={{ padding: "16px", background: "#eee" }}>
 © 2026 Barcha huquqlar himoyalangan
 </footer>
 </body>
 </html>
);
}
```

Endi qaysi sahifaga o'tsangiz ham, yuqorida menyu, pastda footer ko'rinadi.

### 3.6. Styling — sahifani chiroyli qilish

Next.js'da bir necha xil styling usuli bor. Boshlovchilar uchun eng oson — **Tailwind CSS** (loyiha yaratganda "Yes" degan edingiz).

**Tailwind nima?** Tayyor "klasslar" to'plami. CSS faylga o'tib yozish o'rniga, to'g'ridan-to'g'ri `className` da yozasiz:

```
export default function Home() {
 return (
 <div className="p-8">
 <h1 className="text-3xl font-bold text-blue-600">
 Salom, Next.js!
 </h1>
 <p className="mt-4 text-gray-700">
 Bu Tailwind bilan stillangan sahifa.
 </p>
 <button className="mt-6 px-4 py-2 bg-blue-600 text-white rounded-
lg hover:bg-blue-700">
 Tugma
 </button>
 </div>
);
}
```

**Eng ko'p ishlatiladigan Tailwind klasslari:**

Klass	Ma'nosi
<code>p-8</code>	ichki bo'sh joy (padding)
<code>m-4</code>	tashqi bo'sh joy (margin)
<code>mt-4</code>	faqat yuqoridan margin
<code>text-3xl</code>	matn kattaligi
<code>font-bold</code>	qalin matn
<code>text-blue-600</code>	matn rangi
<code>bg-blue-600</code>	fon rangi
<code>rounded-lg</code>	yumaloq burchaklar
<code>flex</code>	elementlarni yonma-yon joylash
<code>gap-4</code>	elementlar orasidagi masofa
<code>hover:bg-blue-700</code>	sichqoncha ustiga kelganda rang

Boshqa styling usullari ham bor (CSS Modules, oddiy CSS), lekin Tailwind boshlovchi uchun eng tez. Klasslarni yodlash shart emas — [tailwindcss.com/docs](https://tailwindcss.com/docs) dan qidirasiz, VS Code ham avtomatik takliflar beradi.

## 3.7. Rasmlar va shriftlar (Image, Font)

Next.js rasmlarni avtomatik optimallashtiradigan maxsus `<Image>` komponenti beradi:

```
import Image from "next/image";

export default function Home() {
 return (
 <Image
 src="/logo.png" // public papkasidan
 alt="Logotip"
 width={200}
 height={100}
 />
);
}
```

```
);
}
```

- `src="/logo.png"` — `public/` papkasidagi `logo.png` fayli (oldiga `/public` yozilmaydi).
- `<Image>` rasmni avtomatik kichraytiradi, tez yuklaydi — sahifa tezroq ishlaydi.

Hozircha rasm bilan ko'p shug'ullanmasangiz ham bo'ladi. Bunday imkoniyat borligini bilib qo'ying.

## 3-BOB MASALALARI (22 ta)

Bularni o'zingiz yaratgan `birinchi-app` loyihasida bajaring. `npm run dev` ishlab tursin, har o'zgarishni brauzerda ko'ring.

**Loyiha va tuzilish (1–6):** 1. `create-next-app` bilan yangi loyiha yarating (yuqoridagi sozlamalar bilan). 2. `npm run dev` bilan ishga tushiring va `localhost:3000` ni brauzerda oching. 3. Loyihada `app`, `public`, `package.json` qayerdaligini toping va ularning vazifasini yozing. 4. `package.json` faylini oching — "`dependencies`" ostida qanday paketlar bor? 5. Dev serverni to'xtating (terminalda `Ctrl+C`), keyin qayta ishga tushiring. 6. `app/page.tsx` qaysi manzilda (URL) ko'rinishini ayting va tekshiring.

**Sahifani o'zgartirish (7–12):** 7. `app/page.tsx` dagi hamma narsani o'chirib, o'zingizning sarlavha va matningizni yozing. 8. Sahifaga 3 ta paragraf qo'shing (o'zingiz haqingizda). 9. JSX ichida `{2026 - 1998}` kabi hisobni ko'rsating. 10. Sahifa ichida `const ism = "..."` o'zgaruvchi yaratib, `{ism}` bilan chiqaring. 11. Bir nechta element qaytarganda `<>...</>` (fragment) ishlatting. 12. `<button>` qo'shing va `onClick={() => alert("Salom!")}` qiling. (Eslatma: bu interaktiv element — keyingi bobda "use client" haqida bilib olasiz; hozir ishlamasligi mumkin, sababini 2-qismda tushuntiramiz.)

**Layout (13–17):** 13. `app/layout.tsx` ni oching va `{children}` qayerdaligini toping. 14. Layoutga yuqori menyu (`<nav>`) qo'shing — sayt nomi chiqsin. 15. Layoutga footer qo'shing — yil va ismingiz chiqsin. 16. `metadata` dagi `title` ni o'zgartiring — brauzer tabida ko'ringanini tekshiring. 17. `<html lang="uz">` ekanini tekshiring va nega muhimligini izohlang.

**Styling (18–22):** 18. Sarlavhani Tailwind bilan katta va qalin qiling (`text-4xl font-bold`). 19. Matnga rang bering (`text-gray-600`) va `p-8` bilan bo'sh joy qo'shing. 20. Tugmaga Tailwind stillarini bering: ko'k fon, oq matn, yumaloq burchak. 21. Tugmaga `hover:` effekti qo'shing — ustiga kelganda rang o'zgarsin. 22. `flex gap-4` bilan 3 ta tugmani yonma-yon joylang.

**Qo'shimcha challenge:** O'zingiz haqingizda oddiy **"Men haqimda"** sahifasi yasang: rasm ( `<Image>` yoki oddiy emoji), ism, qisqa tavsif, va 2–3 ta tugma. Tailwind bilan chiroyli qiling. Layout'da menyu va footer bo'lsin.

---

## 4-BOB. Routing — sahifalar tizimi

Bu — Next.js'ning eng kuchli va sevimli xususiyatlaridan biri. Boshqa freymvorklarda sahifalar tizimini o'zingiz qurishingiz kerak. Next.js'da esa — **papka yaratasiz, sahifa paydo bo'ladi**. Buni **file-based routing** (fayl asosidagi yo'naltirish) deyiladi.

### 4.1. Asosiy qoida: papka = sahifa

Qoida juda oddiy: - `app/` ichida **papka** yaratasiz. - Uning ichiga `page.tsx` fayli qo'yasiz. - Papka nomi — URL manzili bo'ladi.

**Misollar:**

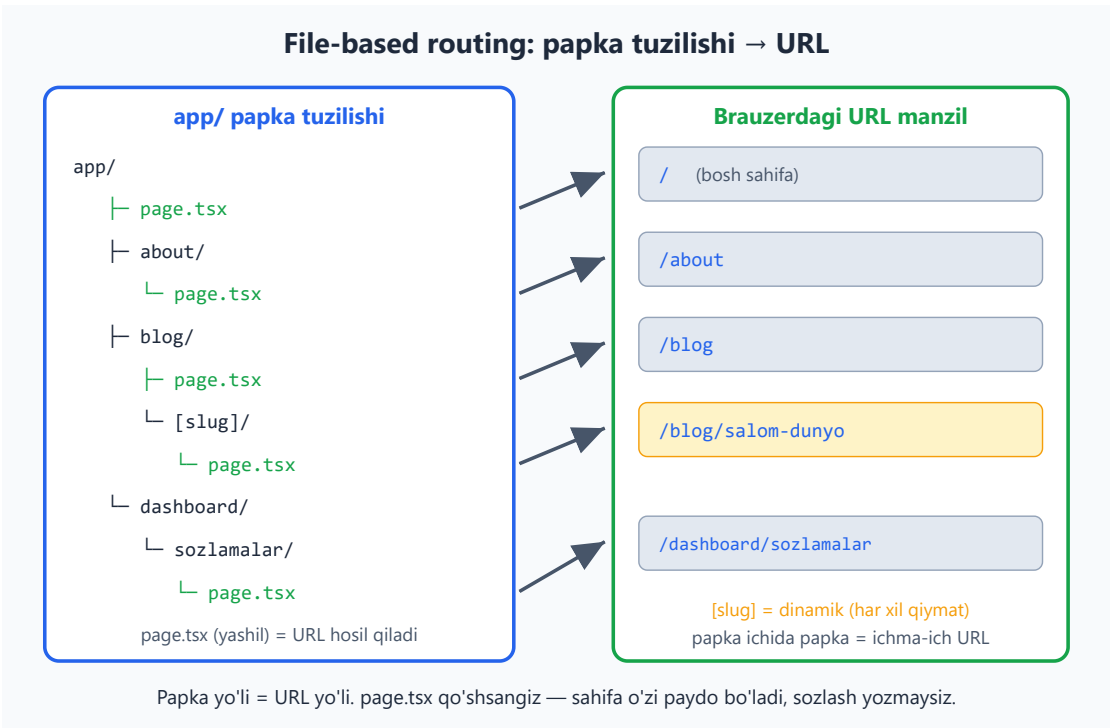
Fayl	URL manzil
<code>app/page.tsx</code>	<code>/</code> (bosh sahifa)
<code>app/about/page.tsx</code>	<code>/about</code>
<code>app/contact/page.tsx</code>	<code>/contact</code>
<code>app/blog/page.tsx</code>	<code>/blog</code>

**Amalda sinaymiz.** `app/` ichida `about` papkasini yarating, ichiga `page.tsx` qo'ying:

```
// app/about/page.tsx
export default function AboutPage() {
 return (
 <div className="p-8">
 <h1 className="text-2xl font-bold">Biz haqimizda</h1>
 <p>Bu — "Biz haqimizda" sahifasi.</p>
 </div>
);
}
```

Brauzerda `localhost:3000/about` ni oching — yangi sahifa tayyor! Hech qanday sozlash yozmadingiz. Faqat papka va fayl yaratdingiz.

Mana shu g'oya — `app/` ichidagi papka tuzilishi qanday qilib URL manzillarga aylanishini umumiy ko'rinishda ko'rsatadigan diagramma (ichma-ich va dinamik holatlar ham bor):



## 4.2. Ichma-ich sahifalar (Nested Routes)

Papka ichida papka — URL ham ichma-ich bo'ladi:

Fayl	URL
<code>app/blog/page.tsx</code>	<code>/blog</code>
<code>app/blog/yangiliklar/page.tsx</code>	<code>/blog/yangiliklar</code>
<code>app/dashboard/sozlamalar/page.tsx</code>	<code>/dashboard/sozlamalar</code>

```
// app/blog/yangiliklar/page.tsx
export default function YangiliklarPage() {
 return <h1>Yangiliklar</h1>;
}
// URL: /blog/yangiliklar
```

## 4.3. Sahifalar orasida o'tish — `<Link>`

Sahifadan sahifaga o'tish uchun oddiy `<a>` teg **ishlatilmaydi** (u sahifani to'liq qayta yuklaydi — sekin). Next.js maxsus `<Link>` beradi — u tez, sahifani qayta yuklamasdan o'tadi:

```
import Link from "next/link";

export default function Home() {
 return (
 <nav className="flex gap-4 p-4">
 <Link href="/">Bosh sahifa</Link>
 <Link href="/about">Biz haqimizda</Link>
 <Link href="/blog">Blog</Link>
 </nav>
);
}
```

- `import Link from "next/link";` — Link'ni olib kelamiz.
- `<Link href="/about">` — href ga manzilni yozamiz.
- Bosilganda sahifa **bir zumda** o'zgaradi (qayta yuklanmaydi).

**Qoida:** Loyiha ichidagi sahifalar uchun doim `<Link>` ishlateng. `<a>` ni faqat **tashqi** saytlarga (masalan `href="https://google.com"`) ishlateng.

Bu menyuni `layout.tsx` ga qo'ysangiz, har sahifada ko'rinadi va saytni "navigatsiya qilsa bo'ladigan" qilib qo'yasiz.

## 4.4. Dinamik sahifalar (Dynamic Routes) — `[id]`

Tasavvur qiling, blogda 1000 ta maqola bor. Har biriga alohida papka yasab bo'lmaydi! Buning yechimi — **dinamik sahifa**. Papka nomini **kvadrat qavs** ichiga olamiz: `[id]`, `[slug]`.

```
app/
├─ blog/
│ └─ [slug]/
│ └─ page.tsx
```

Bu bitta fayl quyidagi **barcha** manzillarni qamrab oladi: - `/blog/birinchi-maqola` - `/blog/nextjs-darslari` - `/blog/istalgan-narsa`

URL'dagi qiymatni (`slug`) sahifa ichida `params` orqali olamiz.

⚠ **Next.js 16 MUHIM o'zgarishi:** `params` endi **asinxron** (Promise). Shuning uchun komponent `async` bo'lishi va `params` ni `await` qilish kerak. (Eski 14-versiyada bunday emas edi — eski darsliklarni ko'rsangiz adashmang.)

```
// app/blog/[slug]/page.tsx
export default async function BlogPost({
 params,
}): {
 params: Promise<{ slug: string }>;
}() {
 const { slug } = await params; // ← 16-versiyada AWAIT shart!

 return (
 <div className="p-8">
 <h1 className="text-2xl font-bold">Maqola: {slug}</h1>
 <p>Siz "{slug}" maqolasini ochdingiz.</p>
 </div>
);
}
```

Endi `/blog/salom-dunyo` ga kirsangiz, ekranda "Maqola: salom-dunyo" chiqadi.  
`/blog/test` ga kirsangiz — "Maqola: test". Bitta fayl, cheksiz sahifa!

**Real misol:** maqolalar ro'yxati va ularga link:

```
// app/blog/page.tsx
import Link from "next/link";

export default function BlogList() {
 const maqolalar = ["nextjs-asoslari", "react-darslari", "javascript-tips"];

 return (
 <ul className="p-8">
 {maqolalar.map((slug) => (
 <li key={slug}>
 <Link href={` /blog/${slug}`} className="text-blue-600">
 {slug}
 </Link>

))}

);
}
```

Har bir link `[slug]` sahifasiga olib boradi.

## 4.5. Catch-all sahifalar — `[...slug]`

Ba'zan bir nechta darajali manzilni bitta fayl bilan ushlash kerak. Buni `[...slug]` (uch nuqta bilan) qiladi:

```
app/docs/[...slug]/page.tsx
```

Bu quyidagilarning **hammasini** ushlaydi: - `/docs/a` - `/docs/a/b` - `/docs/a/b/c`

```
export default async function DocsPage({
 params,
}: {
 params: Promise<{ slug: string[] }>;
}) {
 const { slug } = await params; // slug — endi MASSIV: ["a", "b", "c"]
 return <p>Yo'l: {slug.join(" / ")}</p>;
}
```

Bu ko'pincha hujjatlar (documentation) saytlarida ishlatiladi. Hozir shaklini tanib qo'ying, kerak bo'lganda qaytasiz.

## 4.6. Maxsus fayllar: layout, loading, error, not-found

Next.js'da maxsus nomli fayllar bor — ular alohida vazifani bajaradi. Har bir papkaga qo'yish mumkin.

### `layout.tsx` — ichma-ich qoplama

Har bir papka o'z `layout.tsx` siga ega bo'lishi mumkin. Masalan, blog uchun alohida menyu:

```
// app/blog/layout.tsx
export default function BlogLayout({ children }: { children:
React.ReactNode }) {
 return (
 <div>
 <aside className="p-4 bg-gray-100">Blog menyu</aside>
 <div>{children}</div>
 </div>
);
}
```

Bu faqat `/blog/...` sahifalarida ko'rinadi (asosiy layout ustiga qo'shiladi).

### `loading.tsx` — yuklanish ko'rsatkichi

Sahifa ma'lumot yuklayotganda avtomatik ko'rsatiladigan "Yuklanmoqda..." ekrani:

```
// app/blog/loading.tsx
export default function Loading() {
 return <p className="p-8">⌚ Yuklanmoqda...</p>;
}
```

Next.js buni avtomatik ishlatadi (React'ning `Suspense` mexanizmi orqasida).

## `error.tsx` — xato ekrani

Sahifada xato yuz bersa, butun sayt "qulamaydi" — shu fayl ko'rsatiladi:

```
"use client"; // error.tsx doim Client Component bo'ladi

export default function Error({ reset }: { reset: () => void }) {
 return (
 <div className="p-8">
 <h2>Nimadir xato ketdi 😞</h2>
 <button onClick={() => reset()} className="mt-4 px-4 py-2 bg-blue-600 text-white rounded">
 Qayta urinish
 </button>
 </div>
);
}
```

## `not-found.tsx` — "sahifa topilmadi" (404)

Mavjud bo'lmagan manzilga kirilganda ko'rsatiladigan ekran:

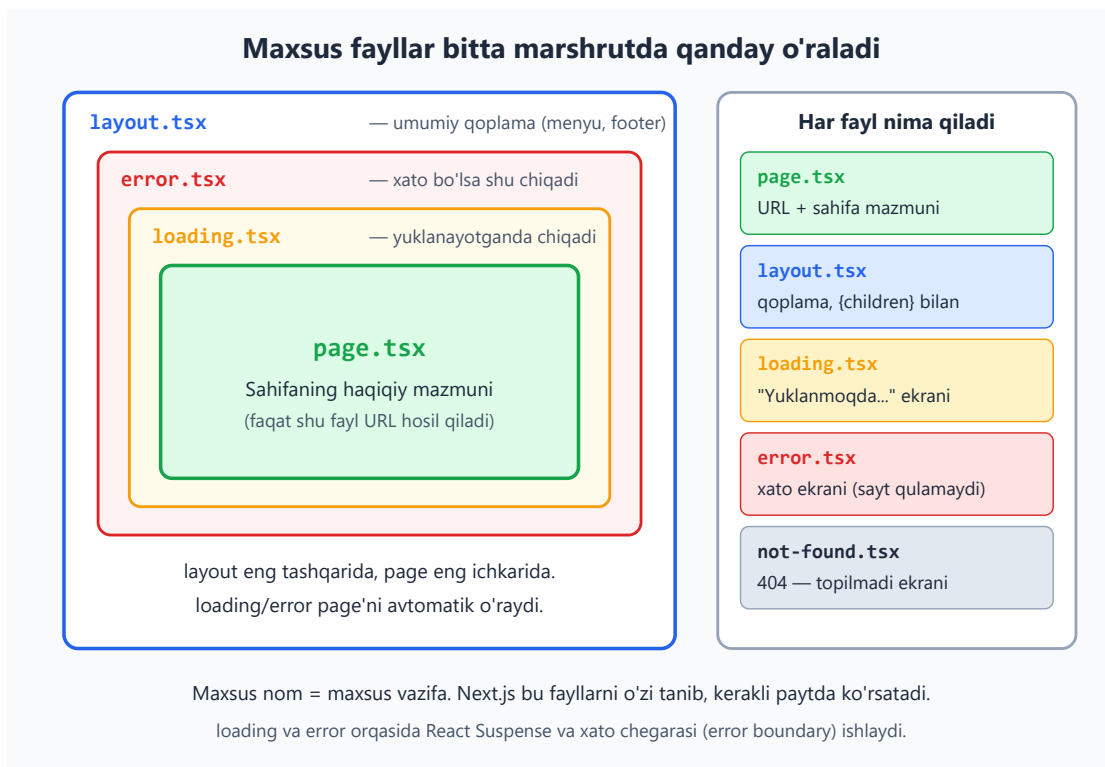
```
// app/not-found.tsx
import Link from "next/link";

export default function NotFound() {
 return (
 <div className="p-8 text-center">
 <h1 className="text-3xl font-bold">404 - Sahifa topilmadi</h1>
 <Link href="/" className="text-blue-600">Bosh sahifaga
 qaytish</Link>
 </div>
);
}
```

**Maxsus fayllar xulosasi:**

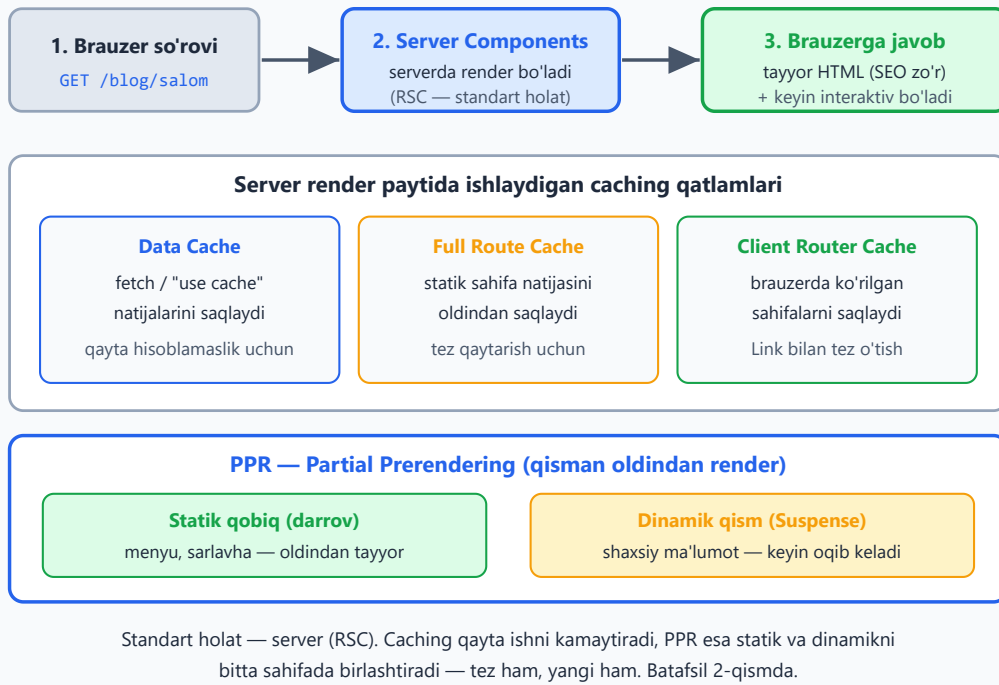
Fayl	Vazifasi
<code>page.tsx</code>	Sahifaning o'zi (URL hosil qiladi)
<code>layout.tsx</code>	Umumiy qoplama (menyu, footer)
<code>loading.tsx</code>	Yuklanish ekrani
<code>error.tsx</code>	Xato ekrani
<code>not-found.tsx</code>	404 ekrani

Quyidagi diagramma bu maxsus fayllar bitta marshrutda qanday joylashishini — `layout` eng tashqarida, `page` eng ichkarida, `loading` va `error` esa `page` ni avtomatik o'rashini — ko'rsatadi:



**Next.js 16 eslatma:** bu sahifalar standart holatda **serverda** render bo'ladigan Server Component'lardir. `page.tsx` serverda tayyorlanadi, `loading.tsx` orqasida React **Suspense**, `error.tsx` orqasida esa **xato chegarasi (error boundary)** ishlaydi. Server va Client Components farqini 2-qismda batafsil ko'ramiz; quyidagi diagramma esa so'rov serverda umuman qanday tayyorlanishiga umumiy nazar tashlaydi:

## Next.js 16: so'rov serverda qanday tayyorlanadi



## 4.7. Dasturiy navigatsiya — useRouter

Ba'zan tugma bosilganda kod ichida sahifaga o'tish kerak (masalan, login muvaffaqiyatli bo'lgach):

```
"use client"; // interaktiv bo'lgani uchun

import { useRouter } from "next/navigation";

export default function LoginTugma() {
 const router = useRouter();

 function kirish() {
 // ... login tekshiruvi ...
 router.push("/dashboard"); // dashboard'ga o'tkazadi
 }

 return <button onClick={kirish}>Kirish</button>;
}
```

- `useRouter()` — navigatsiya vositasi.
- `router.push("/dashboard")` — boshqa sahifaga o'tkazadi.

**"use client" nima?** Yuqorida bir necha bor ko'rdingiz. `useRouter`, `useState`, `onClick` kabi **interaktiv** narsalar ishlatadigan fayl yuqorisiga `"use client"` yozish

kerak. Nega? Buni 2-qismda batafsil tushuntiramiz (Server vs Client Components).  
Hozir qoida sifatida eslab qoling: **interaktiv komponent** → **yuqorida "use client"**.

## 4.8. Route Groups — papka, lekin URL'ga ta'sir qilmaydi

Papkani **oddiy qavs** (nom) ichiga olsangiz, u URLda **ko'rinmaydi**. Bu fayllarni guruhlash uchun ishlatiladi:

```
app/
├── (marketing)/
│ ├── about/page.tsx → /about (marketing ko'rinmaydi!)
│ └── contact/page.tsx → /contact
└── (shop)/
 └── products/page.tsx → /products
```

(marketing) va (shop) — bu shunchaki fayllarni tartibga solish uchun. URL'da chiqmaydi. Masalan, har bir guruhga alohida `layout.tsx` berish uchun foydali.

## 4-BOB MASALALARI (24 ta)

Bularni **birinchi-app** loyihangizda bajaring. Har bir yangi sahifani brauzerda oching va tekshiring.

**Asosiy sahifalar (1–7):** 1. `app/about/page.tsx` yaratib, `/about` sahifasini hosil qiling. 2. `app/contact/page.tsx` yarating — aloqa ma'lumotlari bo'lsin. 3. `app/services/page.tsx` yarating — xizmatlar ro'yxati bo'lsin. 4. `app/blog/page.tsx` yarating — "Blog" sarlavhasi bo'lsin. 5. Ichma-ich: `app/blog/yangiliklar/page.tsx` yarating (`/blog/yangiliklar`). 6. `app/dashboard/page.tsx` va `app/dashboard/profil/page.tsx` yarating. 7. Har bir sahifaga Tailwind bilan oddiy stil bering.

**Navigatsiya — Link (8–13):** 8. `layout.tsx` ga `<Link>` lar bilan menyu qo'shing (Bosh sahifa, About, Blog, Contact). 9. Menyuni Tailwind bilan chiroyli qiling (`flex gap-4`, ranglar). 10. `<a>` va `<Link>` farqini sinab ko'ring: bittasi bilan o'tib, sahifa qayta yuklanishini kuzating. 11. Bosh sahifada barcha sahifalarga olib boruvchi linklar bo'lsin. 12. Tashqi sayt (masalan `https://nextjs.org`) uchun `<a target="_blank">` ishlatib. 13. `/blog/yangiliklar` ga olib boradigan linkni `/blog` sahifasiga qo'ying.

**Dinamik sahifalar (14–19):** 14. `app/blog/[slug]/page.tsx` yarating va `params` dan `slug` ni chiqaring (Next.js 16 — `async + await`!). 15. `/blog/test`, `/blog/salom` manzillariga kirib, slug to'g'ri chiqishini tekshiring. 16. `app/blog/page.tsx` da 4 ta maqola slug'idan ro'yxat yasang va ularga `<Link>` bering. 17. Linklarni `href=`

`{ /blog/${slug} }` ko'rinishida yozing. 18. `app/users/[id]/page.tsx` yarating — `id` ni ekranda ko'rsating (`User ID: 5` kabi). 19. `app/products/[category]/[id]/page.tsx` yarating — ham `category`, ham `id` ni chiqaring (ikkalasi ham `params` da).

**Maxsus fayllar (20–24):** 20. `app/not-found.tsx` yaratib, 404 sahifasini chiroyli qiling (bosh sahifaga link bilan). 21. Mavjud bo'lmagan manzilga (`/mavjudemas`) kirib, 404 ko'rinishini tekshiring. 22. `app/blog/loading.tsx` yarating — "Yuklanmoqda..." ko'rsatsin. 23. `app/blog/layout.tsx` yarating — blog sahifalariga alohida menyu/sarlavha qo'shing. 24. `"use client"` qo'shilgan komponent yarating: tugma bosilganda `useRouter` bilan `/about` ga o'tsin.

**Qo'shimcha challenge:** Kichik **portfolio sayti** strukturasini quring: - Bosh sahifa (`/`) — qisqa tanishuv + menyu. - `/about` — to'liq ma'lumot. - `/projects` — loyihalar ro'yxati (massivdan `.map` bilan). - `/projects/[id]` — har bir loyihaning alohida sahifasi (dinamik). - `/contact` — aloqa. - Umumiy `layout.tsx` da menyu va footer. - Chiroyli `not-found.tsx`. Bu — haqiqiy sayt strukturasini! Tugatsangiz, 1-qismni to'liq o'zlashtirgan bo'lasiz.

---



## 1-QISM YAKUNI

Tabriklayman! Agar shu yergacha kelib, masalalarni yechgan bo'lsangiz, sizda allaqachon mustahkam poydevor bor:

✅ Web va dasturlash qanday ishlashini tushunasiz    ✅ JavaScript'ning asosiy qismlarini bilasiz (o'zgaruvchi, funksiya, massiv, obyekt, `.map/.filter`, `async`)    ✅ React komponentlari, props, state, hodisalarni yoza olasiz    ✅ Next.js loyihasini yaratib, ishga tushira olasiz    ✅ Sahifalar tizimini (routing), dinamik sahifalarni quryapsiz

Bu — Next.js dasturchisi bo'lishning **poydevori**. Hali "ma'lumot bilan ishlash" (eng qiziq qismi) oldinda.

Keyingi qadam: 2-QISM nimani o'z ichiga oladi?

- **Server Components vs Client Components** — `"use client"` aslida nima va qachon ishlatiladi (1-qismda ko'p marta ko'rgan sirning javobi)
- **Ma'lumot olish (data fetching)** — serverdan real ma'lumotni to'g'ridan-to'g'ri sahifaga olib kelish
- **Server Actions** — forma yuborish, ma'lumotni serverga saqlash (login, ro'yxatdan o'tish)

- **Caching** va `"use cache"` — saytni tez qilish, Next.js 16'ning yangi keshlash modeli

**Maslahat:** 2-qismga o'tishdan oldin, 1-qism masalalarining kamida 70%ini yeching va "portfolio" challenge'ini bajaring. Poydevor mustahkam bo'lsa, qolgani oson ketadi.

---

*Bu — "Next.js 0 dan Expert darajagacha" qo'llanmasining 1-qismi.*

---

 [README / Mundarija](#) · [Keyingi: 2-qism — Server & Actions](#) →

# Next.js — 0 dan Expert darajagacha (2-QISM)

 [README](#) · [← 1-qism — Asoslar](#) · [Keyingi: 3-qism — Full-stack →](#)

**Eslatma:** Bu — qo'llanmaning 2-qismi. 1-qismda JavaScript, React asoslari, Next.js loyihasi va routing'ni o'rgangan edingiz. Agar ularni hali ko'rmagan bo'lsangiz — avval 1-qismni tugating. Bu qism uning ustiga quriladi.

**Versiya:** Next.js **16**, React 19. Node.js **20.9+**.

## Bu qismda nimalarni o'rganamiz

1-qism — "sahifa yasash" haqida edi. 2-qism — "**haqiqiy ilova yasash**" haqida. Ya'ni serverdan ma'lumot olish, formalar, ma'lumotni saqlash. Mana rejasi:

- **5-bob. Server Components va Client Components** — 1-qismdagi sirli `"use client"` ning javobi. Eng muhim mavzu.
- **6-bob. Ma'lumot olish (Data Fetching)** — serverdan real ma'lumotni sahifaga olib kelish.
- **7-bob. Server Actions** — formalar, ma'lumotni serverga saqlash (login, ro'yxatdan o'tish, qo'shish/o'chirish).
- **8-bob. Keshlash (Caching)** — saytni tez qilish, Next.js 16'ning yangi `"use cache"` modeli.

**Maslahat:** Bu qism 1-qismdan biroz qiyinroq. Shoshilmang, kod misollarini o'zingiz yozib ko'ring va masalalarni yeching.

---

## 5-BOB. Server Components va Client Components

Bu — zamonaviy Next.js'ning eng muhim va eng ko'p chalkashtiradigan tushunchasi. 1-qismda bir necha bor `"use client"` yozgan edingiz, lekin "bu nima?" degan savol javobsiz qoldi. Mana endi to'liq ochamiz. Bu bobni puxta tushunsangiz, qolgani osonlashadi.

## 5.1. Muammo: kod qayerda ishlaydi?

Esingizdami, 1-qismda **frontend** (brauzerda ishlaydigan qism) va **backend** (serverda ishlaydigan qism) bor edi?

Klassik React'da **hamma kod brauzerda** ishlardi. Bu 2 ta muammo tug'dirardi: 1. **Sekin**: Brauzer avval bo'sh sahifa oladi, keyin JavaScript yuklaydi, keyin ishga tushiradi, keyin ma'lumot oladi. Foydalanuvchi kutib o'tiradi. 2. **Xavfsizlik**: Ma'lumotlar bazasi paroli, maxfiy kalitlar — bularni brauzerda ishlatib bo'lmaydi (har kim ko'rib oladi).

**Next.js yechimi**: kodning bir qismini **serverda**, bir qismini **brauzerda** ishlatish. Shu maqsadda 2 xil komponent bor: - **Server Component** — serverda ishlaydi. - **Client Component** — brauzerda ishlaydi.

## 5.2. Restoran analogiyasi (eng oson tushunish yo'li)

Restoranni eslang:

- **Oshxona (Server Component)** — taom shu yerda tayyorlanadi. Oshxonada xolodilnik (ma'lumotlar bazasi), maxfiy retseptlar (maxfiy kalitlar) bor. Mijoz oshxonani **ko'rmaydi**. Oshxona faqat **tayyor taomni** (HTML'ni) zalga jo'natadi.
- **Stol (Client Component)** — mijoz shu yerda **o'tiradi va ta'sir o'tkazadi**: tugma bosadi, qo'shimcha tuz so'raydi, likopchalarni suradi. Bu yerda interaktivlik bor.

**Asosiy g'oya**: - Taom tayyorlash, xolodilnikka kirish — **oshxonada** (Server Component). - Tugma bosish, animatsiya, foydalanuvchi bilan jonli muloqot — **stolda** (Client Component).

Oshxonada tugma bosib bo'lmaydi (mijoz u yerda yo'q). Stolda esa taom pishirib bo'lmaydi (u yerda pechka yo'q). Har birining o'z vazifasi bor.

## 5.3. Server Component — sukut bo'yicha (default)

**Muhim**: Next.js'da har bir komponent **avtomatik ravishda Server Component** hisoblanadi. Hech narsa yozmasangiz — u serverda ishlaydi. 1-qismda yozgan barcha sahifalaringiz aslida Server Component edi!

**Server Component nima qila oladi**: - ☒ `async` bo'lishi va `await` bilan to'g'ridan-to'g'ri ma'lumot olishi (6-bobda ko'ramiz) - ☒ Ma'lumotlar bazasiga to'g'ridan-to'g'ri murojaat qilishi - ☒ Maxfiy kalitlardan foydalanishi (brauzerga sizib chiqmaydi) - ☒ Kichikroq sayt (uning kodi brauzerga **yuborilmaydi** — shuning uchun tez)

**Server Component nima qila OLMAYDI:** - ❌ `useState`, `useEffect` kabi hook'lar ishlatish - ❌ `onClick`, `onChange` kabi hodisalar - ❌ Brauzer narsalari: `window`, `localStorage` - ❌ Foydalanuvchi bilan jonli muloqot (interaktivlik)

```
// app/page.tsx – bu SERVER COMPONENT (hech narsa yozmadik)
export default function Home() {
 return <h1>Bu serverda tayyorlangan sahifa</h1>;
}
```

## 5.4. Client Component — `"use client"` bilan

Agar komponentga **interaktivlik** kerak bo'lsa (tugma, forma, state), uni **Client Component** qilamiz. Buning uchun faylning **eng yuqorisiga** `"use client"` yozamiz.

```
"use client"; // ← Bu qator faylning ENG BOSHIDA bo'lishi shart!

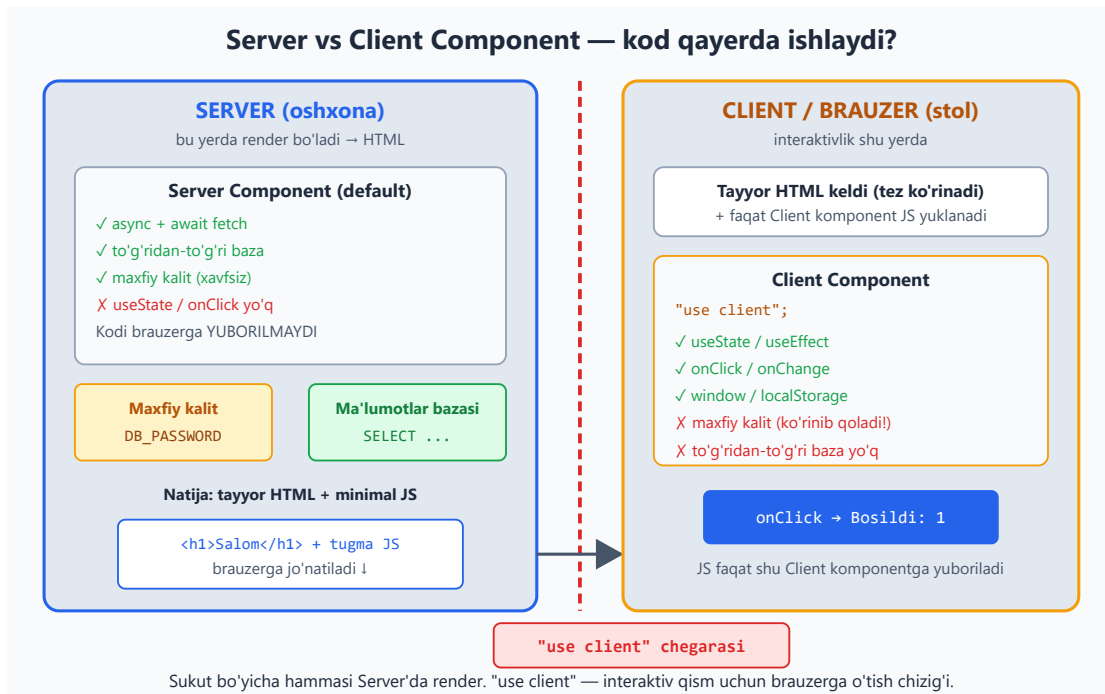
import { useState } from "react";

export default function Hisoblagich() {
 const [son, setSon] = useState(0);

 return (
 <button onClick={() => setSon(son + 1)}>
 Bosildi: {son}
 </button>
);
}
```

**Mana endi 1-qismdagi sirning javobi:** `useState`, `onClick`, `useRouter` — bularning hammasi **interaktiv** narsalar. Ular faqat brauzerda (Client Component'da) ishlaydi. Shuning uchun ularni ishlatadigan har bir fayl yuqorisiga `"use client"` yozish kerak edi.

Quyidagi diagramma bu bobning eng muhim g'oyasini bir rasmda jamlaydi — kod qayerda ishlaydi va `"use client"` chegarasi nimani anglatadi:



**Client Component nima qila oladi:** - ✓ `useState`, `useEffect` va boshqa hook'lar - ✓  
`onClick`, `onChange` hodisalar - ✓ Brauzer narsalari: `window`, `localStorage` - ✓  
Foydalanuvchi bilan jonli muloqot

**Client Component nima qila OLMAYDI:** - ✗ `async` komponent bo'lishi (to'g'ridan-to'g'ri `await` qilolmaydi) - ✗ Ma'lumotlar bazasiga to'g'ridan-to'g'ri murojaat - ✗ Maxfiy kalitlardan foydalanish (brauzerda ko'rinib qoladi!)

## 5.5. Qaysi birini qachon ishlatish kerak?

Oddiy qoida:

**Sukut bo'yicha Server Component ishlatish. Faqat interaktivlik kerak bo'lganda Client Component'ga o'ting.**

**Server Component (default) ishlatish, agar:** - Faqat ma'lumot ko'rsatayapsiz (matn, ro'yxat, kartochka) - Serverdan/bazadan ma'lumot olyapsiz - Maxfiy kalit ishlatayapsiz

**Client Component ( "use client" ) ishlatish, agar:** - Tugma, forma kabi interaktiv narsa bor - `useState`, `useEffect` kerak - Sichqoncha/klaviatura hodisalari kerak - Brauzer xususiyatlari ( `localStorage`, `window` ) kerak

**Belgilar — qaysi xato chiqsa:** - " `useState` only works in a Client Component" → faylga "use client" qo'shing. - "async/await is not supported in Client Components" → "use client" ni olib tashlang yoki ma'lumot olishni boshqa joyga ko'chiring.

## 5.6. Eng muhim pattern: ikkalasini birga ishlatish

Real loyihada **ko'pchilik komponent Server, ozchilik Client** bo'ladi. Maslahat: **interaktiv qismni kichik alohida Client Component qiling**, qolgan hammasini Server qoldiring.

Restoranda: butun restoran "oshxona" emas. Faqat kichik stollar — interaktiv.  
Qolgani — oshxona.

**Misol.** Mahsulot sahifasi: ma'lumot (Server) + "Savatga qo'shish" tugmasi (Client).

```
// app/products/[id]/AddToCartButton.tsx — KICHIK Client Component
"use client";

import { useState } from "react";

export default function AddToCartButton() {
 const [qoshildi, setQoshildi] = useState(false);

 return (
 <button onClick={() => setQoshildi(true)}>
 {qoshildi ? "✓ Savatga qo'shildi" : "Savatga qo'shish"}
 </button>
);
}
```

```
// app/products/[id]/page.tsx — Server Component (default)
import AddToCartButton from "../AddToCartButton";

export default async function ProductPage({
 params,
}: {
 params: Promise<{ id: string }>;
}) {
 const { id } = await params; // Next.js 16 — async params

 return (
 <div className="p-8">
 <h1>Mahsulot #{id}</h1>
 <p>Bu mahsulot tavsifi (serverda tayyorlandi)</p>

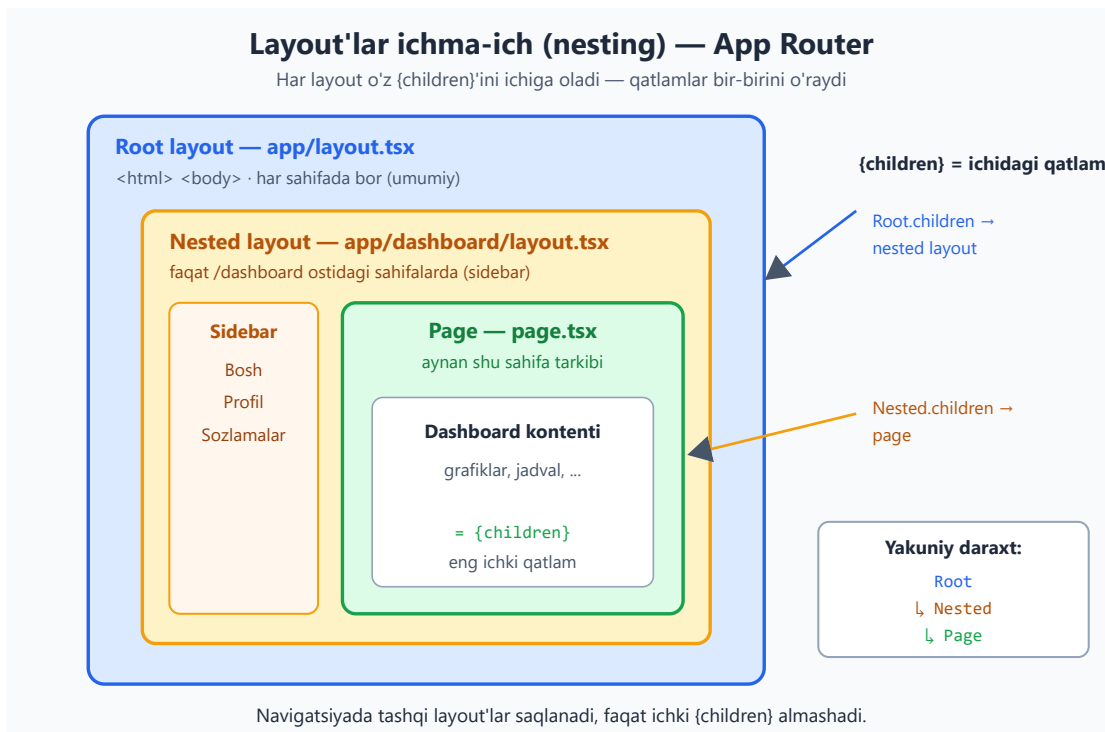
 {/* Kichik interaktiv qismni shu yerga joylaymiz */}
 <AddToCartButton />
 </div>
);
}
```

Bu yerda: - Sahifaning o'zi — Server Component (ma'lumotni serverda tayyorlaydi, tez). -  
Faqat tugma — Client Component (interaktiv).

**Server Component, Client Component'ni ichiga olishi mumkin** (yuqoridagi kabi).

Lekin teskarisi — Client Component ichida Server Component'ni `import` qilib bo'lmaydi. (Lekin Client Component, Server Component'ni `children` orqali qabul qila oladi — bu ilg'or mavzu, hozir kerak emas.)

Aynan shu `children` orqali ichma-ich joylashish App Router'da layout'larda ham ishlaydi: 1-qismda ko'rgan `layout.tsx` 'lar bir-birini o'rab, eng ichkarisida `page.tsx` turadi. Quyidagi diagramma bu qatlamlarni ko'rsatadi:



## 5.7. Ma'lumotni Server'dan Client'ga uzatish — props orqali

Server Component'da ma'lumot olib, uni Client Component'ga **props** orqali berishingiz mumkin:

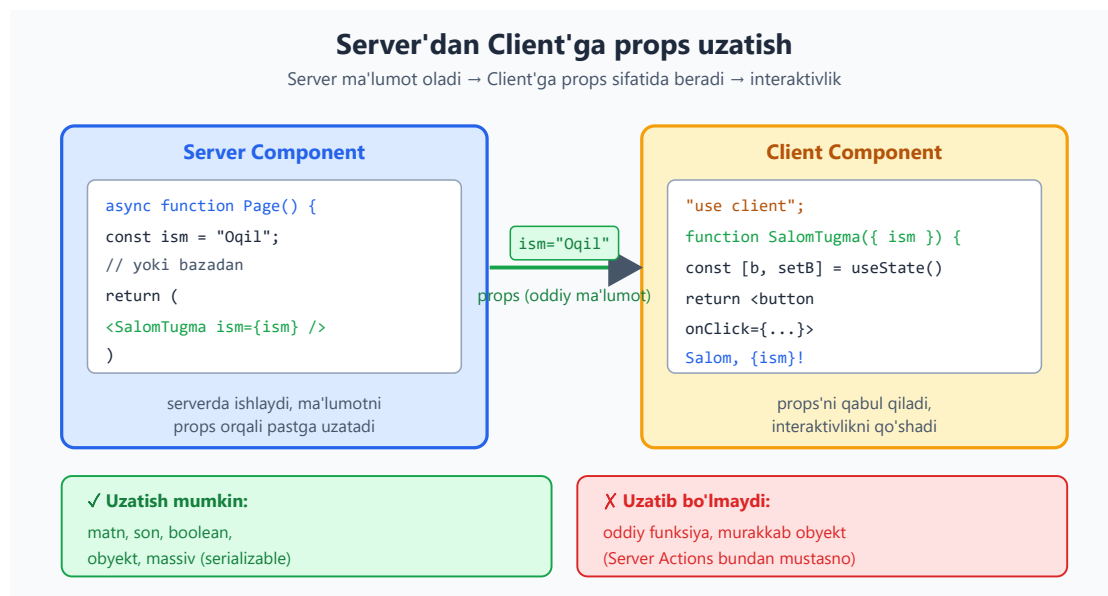
```
// Server Component
export default async function Page() {
 const ism = "Oqil"; // (yoki bazadan olingan ma'lumot)
 return <SalomTugma ism={ism} />;
}
```

```
// Client Component
"use client";
import { useState } from "react";
```

```
export default function SalomTugma({ ism }: { ism: string }) {
 const [bosildi, setBosildi] = useState(false);
 return (
 <button onClick={() => setBosildi(true)}>
 {bosildi ? `Salom, ${ism}!` : "Bosing"}
 </button>
);
}
```

**Qoida:** Server'dan Client'ga faqat **oddiy ma'lumot** (matn, son, obyekt, massiv) uzatish mumkin. Funksiya yoki murakkab narsalarni emas (Server Actions bundan mustasno — 7-bobda).

Mana shu props oqimi rasmda:



## 5-BOB MASALALARI (20 ta)

Bularni 1-qismda yaratgan loyihada yoki yangi loyihada bajaring.

**Tushunish savollari (1–7):** 1. O'z so'zlaringiz bilan: Server Component va Client Component farqi nima? 2. Restoran misolida oshxona — qaysi komponent? Stol-chi? 3. Next.js'da hech narsa yozmasangiz, komponent qaysi turda bo'ladi? 4. "use client" qayerga yoziladi va u nimani anglatadi? 5. Nega maxfiy kalitni Client Component'da ishlatib bo'lmaydi? 6. Server Component nega tezroq (brauzerga nima yuborilmaydi)? 7. Quyidagilarning qaysi biri Client Component talab qiladi: (a) matn ko'rsatish, (b) tugma bosish, (c) bazadan o'qish, (d) useState ?

**Amaliy (8–16):** 8. Oddiy Server Component sahifa yarating — faqat matn ko'rsatsin ("use client" siz). 9. Hisoblagich Client Component yarating ("use client" +

`useState` + tugma). 10. Client Component'ga "Kamaytirish" tugmasini qo'shing. 11. "Yashirish/Ko'rsatish" Client Component yarating (boolean state bilan). 12. Server Component sahifaga `const` o'zgaruvchi qo'shing va uni Client Component'ga props orqali bering. 13. Bir Server sahifaning ichida 1 ta Client tugma joylang (yuqoridagi pattern bo'yicha). 14. Client Component'da `useState` siz tugmaga `onClick={() => alert("Salom")}` qo'shing. 15. Server Component'ni `async` qiling va ichida `params` ni `await` qiling. 16. Input maydonli Client Component yarating — yozgan matn pastda ko'rinib tursin.

**Xatolarni tushunish (17–20):** 17. Server Component'ga `useState` qo'shib ko'ring. Qanday xato chiqdi? Uni tuzating. 18. Client Component'ni `async` qilib, ichida `await` ishlatib ko'ring. Xato chiqdimi? 19. `"use client"` ni faylning o'rtasiga yozing. Ishladimi? Nega? 20. Bir sahifada qaysi qism Server, qaysi qism Client bo'lishi kerakligini rejalashtiring: blog maqolasi (matn) + "Yoqdi" tugmasi (like).

**Qo'shimcha challenge:** Mahsulot kartochkasi yasang: mahsulot nomi, narxi, tavsifi — Server Component'da. "Sevimlilarga qo'shish" (yurakcha) tugmasi — alohida Client Component. Tugma bosilganda yurakcha to'lsin (`useState`).

---

## 6-BOB. Ma'lumot olish (Data Fetching)

Endi eng qiziq qism boshlanadi: real ma'lumotni serverdan olib, sahifada ko'rsatish. Masalan, foydalanuvchilar ro'yxati, maqolalar, mahsulotlar.

### 6.1. Eski usul va yangi usul

Esingizdami, 1-qismda (2-bob) `useEffect` bilan ma'lumot olishni eslatib o'tgan edik? Klassik React'da shunday qilinardi:

```
// ESKI USUL (klassik React) — chalkash va sekin
"use client";
import { useState, useEffect } from "react";

function UsersList() {
 const [users, setUsers] = useState([]);
 const [yuklanmoqda, setYuklanmoqda] = useState(true);

 useEffect(() => {
 fetch("https://jsonplaceholder.typicode.com/users")
 .then((res) => res.json())
 .then((data) => {
```

```

 setUsers(data);
 setYuklanmoqda(false);
 });
}, []);

if (yuklanmoqda) return <p>Yuklanmoqda...</p>;
return {users.map((u) => <li key={u.id}>{u.name})};
}

```

Ko'p kod, ko'p state, va sahifa avval bo'sh keladi, keyin ma'lumot to'ldiriladi (sekin, SEO yomon).

**Next.js'ning YANGI usuli** — Server Component'da to'g'ridan-to'g'ri `async` / `await` :

```

// YANGI USUL (Next.js Server Component) — oddiy va tez
export default async function UsersList() {
 const res = await
fetch("https://jsonplaceholder.typicode.com/users");
 const users = await res.json();

 return (

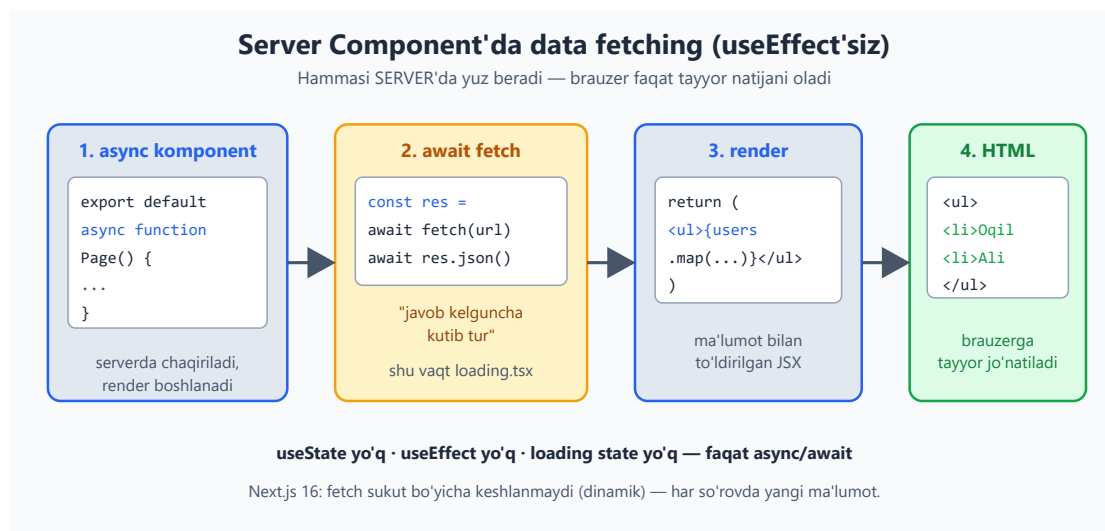
 {users.map((u: { id: number; name: string }) => (
 <li key={u.id}>{u.name}
))}

);
}

```

Farqni ko'ryapsizmi? `useState` **yo'q**, `useEffect` **yo'q**, `loading state` **yo'q**. Komponent oddiygina `async` bo'ladi va ma'lumotni `await` qiladi. Ma'lumot **serverda** olinadi, sahifa tayyor holatda brauzerga keladi — tez va SEO uchun zo'r.

Bu jarayonni bosqichma-bosqich ko'rsatadigan diagramma:



**Bu mumkin, chunki Server Component serverda ishlaydi** (5-bobni eslang).

Brauzerda `async` komponent bo'lmaydi, lekin serverda — bemaolol.

## 6.2. `fetch` nima va `async/await` qanday ishlaydi

`fetch(url)` — internet/serverdan ma'lumot olib keladi. U **vaqt oladi**, shuning uchun `await` bilan kutamiz:

```
const res = await fetch("https://api.example.com/data");
// ↑ "javob kelguncha kutib tur"
const data = await res.json();
// ↑ javobni o'qiladigan ma'lumotga aylantir
```

- `fetch(...)` — so'rov yuboradi, **javob (response)** qaytaradi.
- `res.json()` — javobni JavaScript obyekti/massiviga aylantiradi (bu ham vaqt oladi → `await`).

**JSON nima?** Ma'lumot almashishning standart formati. Obyekt/massiv ko'rinishida:

`{"name": "Oqil", "age": 25}`. Serverlar ma'lumotni shu ko'rinishda yuboradi.

**Mashq qilish uchun bepul API:** [jsonplaceholder.typicode.com](https://jsonplaceholder.typicode.com) — soxta foydalanuvchilar, maqolalar, izohlar beradi. Mashqlar uchun ideal.

## 6.3. Bitta elementni olish (dinamik sahifa bilan)

1-qismdagi dinamik sahifalarni ( `[id]` ) eslang. Ularni real ma'lumot bilan to'ldiramiz:

```
// app/users/[id]/page.tsx
export default async function UserPage({
 params,
}: {
 params: Promise<{ id: string }>;
}) {
 const { id } = await params; // Next.js 16 - async params

 const res = await
 fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
 const user = await res.json();

 return (
 <div className="p-8">
 <h1 className="text-2xl font-bold">{user.name}</h1>
 <p>Email: {user.email}</p>
 <p>Telefon: {user.phone}</p>
 <p>Shahar: {user.address?.city}</p>
 </div>
);
}
```

```
);
}
```

`/users/1` ga kirsangiz — 1-foydalanuvchi, `/users/2` — 2-foydalanuvchi. URL'dagi `id` to'g'ridan-to'g'ri so'rovga ishlatildi.

## 6.4. Yuklanish holati — `loading.tsx`

Ma'lumot olish vaqt oladi. Foydalanuvchiga "Yuklanmoqda..." ko'rsatamiz. 1-qismda ko'rgan `loading.tsx` aynan shu uchun:

```
// app/users/loading.tsx
export default function Loading() {
 return <p className="p-8">⌚ Foydalanuvchilar yuklanmoqda...</p>;
}
```

Next.js buni **avtomatik** ishlatadi: sahifa ma'lumot kutayotganda `loading.tsx` ko'rinadi, ma'lumot kelgach — asl sahifa. Hech qanday `useState` kerak emas! (Bu React'ning `Suspense` mexanizmi orqasida ishlaydi.)

## 6.5. Xatoni boshqarish — `error.tsx`

Server javob bermasa yoki xato bo'lsa, butun sayt "qulamasligi" kerak. 1-qismdagi `error.tsx` shu uchun:

```
// app/users/error.tsx
"use client"; // error.tsx doim Client Component

export default function Error({
 error,
 reset,
}: {
 error: Error;
 reset: () => void;
}) {
 return (
 <div className="p-8">
 <h2>Ma'lumot yuklanmadi 🤖</h2>
 <button onClick={() => reset()} className="mt-2 px-4 py-2 bg-blue-600 text-white rounded">
 Qayta urinish
 </button>
 </div>
);
}
```

Sahifada xato chiqsa, Next.js avtomatik shu ekranni ko'rsatadi.

## 6.6. Bir nechta so'rovni parallel olish

Agar 2 xil ma'lumot kerak bo'lsa, ularni **birin-ketin emas, baravar** (parallel) olish tezroq:

```
export default async function Dashboard() {
 // ❌ Sekin - birin-ketin (biri tugagach ikkinchisi):
 // const users = await fetch("../users").then(r => r.json());
 // const posts = await fetch("../posts").then(r => r.json());

 // ✅ Tez - baravar (Promise.all):
 const [usersRes, postsRes] = await Promise.all([
 fetch("https://jsonplaceholder.typicode.com/users"),
 fetch("https://jsonplaceholder.typicode.com/posts"),
]);
 const users = await usersRes.json();
 const posts = await postsRes.json();

 return (
 <div className="p-8">
 <p>Foydalanuvchilar soni: {users.length}</p>
 <p>Maqolalar soni: {posts.length}</p>
 </div>
);
}
```

`Promise.all([...])` — bir nechta so'rovni baravar yuboradi, hammasi tugaguncha kutadi. Ikki marta tezroq.

## 6.7. Muhim: Next.js 16'da `fetch` keshlanmaydi

⚠️ **Next.js 16 muhim o'zgarishi:** Avvalgi versiyalarda `fetch` natijasi **avtomatik keshlanardi** (saqlanardi). Next.js 16'da bu **o'zgardi** — `fetch` endi sukut bo'yicha **keshlanmaydi**, har safar yangi ma'lumot olinadi (dinamik).

Bu nima degani? Hozircha — har sahifa ochilganda yangi so'rov yuboriladi. Bu ko'p hollarda yaxshi (yangi ma'lumot). Lekin tez-tez o'zgarmaydigan ma'lumotni (masalan, blog maqolasi) **keshlash** (saqlab qo'yish) tezlikni oshiradi. Buni **8-bobda** ( `"use cache"` ) o'rganamiz.

Hozircha shuni biling: Next.js 16'da `fetch` → har safar yangi ma'lumot.

## 6.8. To'liq misol: foydalanuvchilar ro'yxati va detallari

Hamma narsani birlashtiramiz:

```
// app/users/page.tsx - ro'yxat sahifasi
import Link from "next/link";

export default async function UsersPage() {
 const res = await
 fetch("https://jsonplaceholder.typicode.com/users");
 const users = await res.json();

 return (
 <div className="p-8">
 <h1 className="text-2xl font-bold mb-4">Foydalanuvchilar</h1>
 <ul className="space-y-2">
 {users.map((user: { id: number; name: string }) => (
 <li key={user.id}>
 <Link href={` /users/${user.id}`} className="text-blue-600
 hover:underline">
 {user.name}
 </Link>

))}

 </div>
);
}
```

Ro'yxatdagi har bir ismga bossangiz — uning detallari sahifasiga ( /users/[id] ) o'tasiz. To'liq, real ishlaydigan ilova!

## 6-BOB MASALALARI (21 ta)

Mashqlarda [jsonplaceholder.typicode.com](https://jsonplaceholder.typicode.com) API'sidan foydalaning. Mavjud yo'llar: /users, /posts, /comments, /todos, /albums, /photos.

**Asosiy fetch (1–8):** 1. Server Component'da /users ni olib, ismlarni ro'yxatda chiqaring. 2. Shu ro'yxatda email'larni ham ko'rsating. 3. /posts ni olib, maqola sarlavhalarini ( title ) chiqaring. 4. /todos ni olib, faqat title larni ro'yxatda chiqaring. 5. /users ni olib, foydalanuvchilar sonini ko'rsating ( .length ). 6. Eski usul ( useEffect ) va yangi usul ( async Server Component ) farqini izohlang. 7. fetch va res.json() nima qilishini o'z so'zingiz bilan tushuntiring. 8. JSON nima? Misol keltiring.

**Dinamik sahifa bilan (9–14):** 9. app/users/[id]/page.tsx yaratib, bitta foydalanuvchi detallarini ko'rsating ( async params !). 10. Detal sahifasiga telefon, shahar ( address.city ), kompaniya ( company.name ) qo'shing. 11. /posts/[id] sahifasi

yarating — bitta maqola detallari ( `title` , `body` ). 12. `/users` ro'yxatidagi har bir ismga `<Link>` qo'yib, detal sahifasiga bog'lang. 13. `/posts` ro'yxatini detal sahifasiga bog'lang. 14. Detal sahifaga "Orqaga" linkini qo'shing (ro'yxatga qaytsin).

**Loading va Error (15–18):** 15. `/users` uchun `loading.tsx` yarating — "Yuklanmoqda..." ko'rsatsin. 16. `loading.tsx` ni chiroyli qiling (Tailwind bilan oddiy "skeleton" yoki spinner matni). 17. `/users` uchun `error.tsx` yarating ("Qayta urinish" tugmasi bilan). 18. Noto'g'ri URL'ga ( `.../usersXXX` ) so'rov yuborib, error ekranini sinab ko'ring.

**Ilg'or (19–21):** 19. `Promise.all` bilan `/users` va `/posts` ni baravar oling, ikkalasining sonini ko'rsating. 20. `/users` va `/todos` ni baravar olib, "X ta foydalanuvchi, Y ta vazifa" ko'rsating. 21. Bitta sahifada `/posts/1` maqolasini va uning izohlarini ( `/posts/1/comments` ) baravar oling va ikkalasini ko'rsating.

**Qo'shimcha challenge:** To'liq **blog** yasang: `/blog` — maqolalar ro'yxati ( `/posts` dan), har biri `/blog/[id]` ga link. `/blog/[id]` — maqola to'liq matni + o'sha maqola izohlari ( `/posts/[id]/comments` ). `loading.tsx` va `error.tsx` ham bo'lsin. Bu — haqiqiy, ishlaydigan blog!

---

## 7-BOB. Server Actions — ma'lumot yuborish va saqlash

Hozirgacha biz ma'lumotni faqat **o'qidik** (serverdan oldik). Endi teskarisini o'rganamiz: foydalanuvchidan ma'lumot olib, **serverga yuborish va saqlash**. Masalan: ro'yxatdan o'tish, login, izoh qoldirish, mahsulot qo'shish.

Bu uchun Next.js **Server Actions** beradi — bu juda kuchli xususiyat.

### 7.1. Muammo: forma ma'lumotini qayerga yuboramiz?

Klassik webda formani serverga yuborish murakkab edi: alohida API yozasiz, `fetch` bilan POST so'rov jo'natasiz, javobini kutasiz... Ko'p kod.

**Server Action** — bu **serverda ishlaydigan funksiya**, uni to'g'ridan-to'g'ri formaga ulaysiz. Alohida API yozish shart emas!

Restoran misolida: avval mijoz oshxonaga borib, taom uchun buyurtmani o'zi yozib qoldirishi kerak edi. Endi esa stoldagi tugmani bossangiz, buyurtma **avtomatik**

oshxonaga boradi.

## 7.2. Birinchi Server Action

"use server" direktivasi funksiyani Server Action qiladi. Eng yaxshi amaliyot — uni alohida faylga yozish:

```
// app/actions.ts
"use server"; // ← Bu fayldagi barcha funksiyalar Server Action

export async function salomYuborish(formData: FormData) {
 const ism = formData.get("ism");
 console.log("Serverga keldi:", ism);
 // Bu yerda ma'lumotni bazaga saqlash mumkin
}
```

Endi uni formaga ulaymiz:

```
// app/page.tsx — Server Component
import { salomYuborish } from "../actions";

export default function Home() {
 return (
 <form action={salomYuborish} className="p-8 space-y-2">
 <input
 type="text"
 name="ism"
 placeholder="Ismingiz"
 className="border p-2 rounded"
 />
 <button type="submit" className="px-4 py-2 bg-blue-600 text-white rounded">
 Yuborish
 </button>
 </form>
);
}
```

**Tushuntirish:** - `<form action={salomYuborish}>` — forma yuborilganda `salomYuborish` funksiyasi **serverda** ishlaydi. - `formData.get("ism")` — `name="ism"` bo'lgan input qiymatini oladi. - "use server" tufayli bu funksiya brauzerda emas, **serverda** ishlaydi — xavfsiz.

**Diqqat:** `formData.get("ism")` ishlashi uchun input'da `name="ism"` bo'lishi **shart**. `name` yo'q bo'lsa — ma'lumot kelmaydi.

## 7.3. Server Action — nega kuchli?

- ☒ **Xavfsiz** — serverda ishlaydi, maxfiy kalit/parol ko'rinmaydi.
- ☒ **Oddiy** — alohida API yozish shart emas.
- ☒ **To'g'ridan-to'g'ri bazaga** — ma'lumotni bevosita saqlash mumkin.
- ☒ Next.js 16 ma'lumotni shifrlab yuboradi (qo'shimcha himoya).

**Muhim xavfsizlik qoidasi:** Har bir Server Action ichida foydalanuvchi huquqini tekshiring (kim ekanini, ruxsati bor-yo'qligini). Forma faqat "kirgan foydalanuvchi" sahifasida ko'rinsa ham — baribir tekshiring. Hech qachon faqat "ko'rinmaydi" deb ishonmang.

## 7.4. Forma holatini boshqarish — `useActionState`

Yuqoridagi misolda forma yuborildi, lekin foydalanuvchi natijani ko'rmaydi. React 19'ning `useActionState` hook'i buni hal qiladi: u xato/muvaffaqiyat xabarini va "yuborilmoqda" holatini boshqaradi.

**Eslatma:** `useActionState` — hook, shuning uchun u **Client Component** talab qiladi (`"use client"`). 5-bobni eslang: hook'lar faqat Client'da ishlaydi.

Avval Server Action'ni natija qaytaradigan qilamiz:

```
// app/actions.ts
"use server";

export async function royxatdanO'tish(oldingiHolat: any, formData:
FormData) {
 const email = formData.get("email") as string;

 // Oddiy validatsiya (tekshiruv):
 if (!email || !email.includes("@")) {
 return { xato: "To'g'ri email kiriting" };
 }

 // ... bu yerda bazaga saqlash ...

 return { muvaffaqiyat: "Ro'yxatdan o'tdingiz!" };
}
```

**Diqqat:** `useActionState` bilan ishlaganda, Server Action 2 ta argument oladi: `(oldingiHolat, formData)`. Birinchisi — oldingi qaytarilgan holat.

Endi Client Component'da formani yasaymiz:

```
// app/RoyxatFormasi.tsx
"use client";

import { useActionState } from "react";
import { royxatdanOtish } from "../actions";

export default function RoyxatFormasi() {
 const [holat, formAction, yuborilmoqda] =
 useActionState(royxatdanOtish, null);

 return (
 <form action={formAction} className="p-8 space-y-2">
 <input
 type="email"
 name="email"
 placeholder="Email"
 className="border p-2 rounded"
 />
 <button
 type="submit"
 disabled={yuborilmoqda}
 className="px-4 py-2 bg-blue-600 text-white rounded
disabled:opacity-50"
 >
 {yuborilmoqda ? "Yuborilmoqda..." : "Ro'yxatdan o'tish"}
 </button>

 {holat?.xato && <p className="text-red-600">{holat.xato}</p>}
 {holat?.muvaffaqiyat && <p className="text-green-600">
 {holat.muvaqqiyat}</p>}
 </form>
);
}
```

**useActionState** ni qatorma-qator tushunamiz:

```
const [holat, formAction, yuborilmoqda] =
 useActionState(royxatdanOtish, null);
```

- **holat** — Server Action **qaytargan natija** ( { xato: ... } yoki { muvaffaqiyat: ... } ). - **formAction** — formaga ulanadigan "o'ralgan" action ( <form action={formAction}> ). - **yuborilmoqda** — true / false — hozir yuborilyaptimi? (tugmani vaqtincha o'chirish uchun). - **royxatdanOtish** — bizning Server Action. - **null** — boshlang'ich holat.

Endi: email noto'g'ri bo'lsa qizil xato, to'g'ri bo'lsa yashil xabar chiqadi, va yuborilayotganda tugma "Yuborilmoqda..." bo'lib o'chadi. Professional forma!

## 7.5. Ma'lumotni yangilash — `revalidatePath` va `revalidateTag`

Yangi izoh qo'shsangiz, u darrov ro'yxatda ko'rinishi kerak. Lekin Next.js sahifani keshlab qo'ygan bo'lishi mumkin (8-bobda tushunamiz). Saqlangandan keyin sahifani **yangilash** uchun:

```
// app/actions.ts
"use server";

import { revalidatePath } from "next/cache";

export async function izohQoshish(formData: FormData) {
 const matn = formData.get("matn") as string;

 // ... izohni bazaga saqlash ...

 revalidatePath("/izohlar"); // ← /izohlar sahifasini yangila
}
```

`revalidatePath("/izohlar")` — Next.js'ga "bu sahifaning keshini tashlab, yangi ma'lumotni ko'rsat" deydi. Endi yangi izoh darrov ko'rinadi.

`revalidateTag(...)` ham bor — u aniqroq, faqat ma'lum belgilangan ma'lumotni yangilaydi. Buni 8-bobda `cacheTag` bilan birga ko'ramiz.

## 7.6. Boshqa sahifaga o'tkazish — `redirect`

Login muvaffaqiyatli bo'lsa, foydalanuvchini boshqa sahifaga o'tkazish kerak. Server Action ichida `redirect` ishlatamiz:

```
// app/actions.ts
"use server";

import { redirect } from "next/navigation";

export async function login(formData: FormData) {
 const email = formData.get("email") as string;
 const parol = formData.get("parol") as string;

 // ... parolni tekshirish ...

 redirect("/dashboard"); // ← muvaffaqiyatdan keyin dashboard'ga o'tkaz
}
```

**Esda tuting:** Server Action ichida `redirect` (`next/navigation` dan), Client Component ichida esa `useRouter().push(...)` (1-qism, 4-bobni eslang). Ikkalasi turli joyda ishlaydi.

## 7.7. To'liq misol: oddiy "fikir-mulohaza" formasi

Hamma narsani birlashtiramiz:

```
// app/actions.ts
"use server";

export async function fikrYuborish(oldingi: any, formData: FormData) {
 const ism = formData.get("ism") as string;
 const fikr = formData.get("fikr") as string;

 if (!ism || ism.length < 2) {
 return { xato: "Ismingizni to'liq kiriting" };
 }
 if (!fikr || fikr.length < 5) {
 return { xato: "Fikringiz juda qisqa" };
 }

 // ... bazaga saqlash (3-qismda ko'ramiz) ...
 console.log("Yangi fikr:", { ism, fikr });

 return { muvaffaqiyat: `Rahmat, ${ism}! Fikringiz qabul qilindi.` };
}
```

```
// app/FikrFormasi.tsx
"use client";

import { useActionState } from "react";
import { fikrYuborish } from "../actions";

export default function FikrFormasi() {
 const [holat, action, yuborilmoqda] = useActionState(fikrYuborish, null);

 return (
 <form action={action} className="max-w-md p-8 space-y-3">
 <input name="ism" placeholder="Ismingiz" className="w-full border p-2 rounded" />
 <textarea name="fikr" placeholder="Fikringiz" className="w-full border p-2 rounded" />
 <button
 type="submit"
 disabled={yuborilmoqda}
 className="px-4 py-2 bg-blue-600 text-white rounded disabled:opacity-50"
 >

```

```

 {yuborilmoqda ? "Yuborilmoqda..." : "Yuborish"}
 </button>

 {holat?.xato && <p className="text-red-600">{holat.xato}</p>}
 {holat?.muvaqqat && <p className="text-green-600">
 {holat.muvaqqat}</p>}
 </form>
);
 }
}

```

Bu — to'liq ishlaydigan, validatsiyali, holat ko'rsatadigan professional forma.  
Tabriklaymiz!

## 7-BOB MASALALARI (22 ta)

**Asosiy Server Actions (1–8):** 1. "use server" bilan actions.ts fayli yarating va ichida bir funksiya yozing. 2. Bitta input'li forma yarating ( name="ism" ), Server Action console.log qilsin. 3. Server Action'da formData.get(...) bilan 2 ta maydon ( ism, email ) oling. 4. name atributini olib tashlab ko'ring — ma'lumot keldimi? Nega? 5. Server Action'ni redirect("/rahmat") bilan tugatib, o'sha sahifani yarating. 6. "use server" qaysi joyga yoziladi va nimani anglatadi? Izohlang. 7. Nega Server Action xavfsiz (parol/kalit ko'rinmaydi)? Tushuntiring. 8. Server Action va Server Component farqi nima? (Diqqat: bir-biriga o'xshash atamalar!)

**useActionState (9–16):** 9. Server Action'ni (oldingi, formData) ko'rinishida yozing, natija qaytarsin. 10. useActionState bilan Client forma yarating, natijani ekranda ko'rsating. 11. Validatsiya qo'shing: email'da @ bo'lmasa, qizil xato chiqsin. 12. Muvaqqatda yashil xabar chiqaring ( { muvaqqat: ... } ). 13. yuborilmoqda (pending) bilan tugmani "Yuborilmoqda..." qiling va disabled qiling. 14. Bir nechta maydonni tekshiring (ism bo'sh emas, email to'g'ri, parol 6+ belgi). 15. Har bir maydon uchun alohida xato xabari qaytaring ( { xatolar: { email: ..., parol: ... } }). 16. Formaga textarea qo'shib, uzunligini tekshiring (kamida 10 belgi).

**Yangilash va to'liq loyiha (17–22):** 17. revalidatePath nima qiladi? Qachon kerak bo'ladi? Izohlang. 18. Server Action ichida ma'lumotni console.log qilib, terminalda ko'ringanini tasdiqlang (server logi). 19. redirect (Server Action) va useRouter().push (Client) farqini ayting. 20. To'liq "Aloqa formasi" yasang: ism, email, xabar + validatsiya + holat xabari + pending. 21. "Login formasi" yasang: email + parol. Bo'sh bo'lsa xato, to'g'ri bo'lsa redirect("/dashboard"). 22. Forma yuborilgach, holat orqali "✓ Yuborildi" ko'rsatib, inputlarni tozalashga harakat qiling.

**Qo'shimcha challenge:** Sodda "mehmonlar kitobi" (guestbook) yasang: forma orqali ism + xabar yuboriladi. Yuborilgan xabarlar pastda ro'yxatda ko'rinadi (hozircha

oddiy massivda yoki `console.log`; haqiqiy saqlash 3-qismda bazaga o'tganda bo'ladi). Validatsiya va holat xabari bo'lsin.

---

## 8-BOB. Keshlash (Caching) va "use cache"

Mana ilg'or, lekin juda muhim mavzu. **Keshlash (caching)** — saytni **tez** qilishning asosiy usuli. Bu bob biroz qiyinroq, lekin tushunsangiz — saytlaringiz "uchadigan" bo'ladi.

### 8.1. Keshlash nima va nega kerak?

**Kesh (cache)** — bir marta tayyorlangan natijani **saqlab qo'yish**, keyin uni qayta-qayta ishlatish.

**Misol:** Onangiz har kuni bir xil osh damlaydi. Har gal noldan retsept o'ylab topmaydi — bir marta yodlagan, keyin shuni takrorlaydi. **Yodlangan retsept — bu kesh.**

Saytda: ma'lumotni har safar serverdan/bazadan olish **vaqt oladi**. Agar ma'lumot tez-tez o'zgarmasa (masalan, blog maqolasi), uni **bir marta olib, saqlab qo'yamiz**. Keyingi foydalanuvchilarga saqlangandan beramiz — **bir necha barobar tez**.

**Foydalari:** - ⚡ Sayt tezroq ochiladi. - 💰 Server/baza kamroq ishlaydi (xarajat kamayadi). - 😊 Foydalanuvchi mamnun.

### 8.2. Next.js 16'ning yangi modeli: "sukut bo'yicha dinamik"

⚠ **Bu Next.js 16'ning eng katta o'zgarishi.** Eski versiyalarda (14 va undan oldin) sahifalar **avtomatik keshlanardi**. Bu chalkash edi — ba'zan eski ma'lumot ko'rsatilib qolardi.

**Next.js 16'da:** sukut bo'yicha hech narsa keshlanmaydi (**dinamik** — har safar yangi). Keshlash kerak bo'lsa, **o'zingiz aniq belgilaysiz**. Bu "opt-in" (ixtiyoriy yoqish) model deyiladi.

Ya'ni: - **Default:** har sahifa har ochilganda — yangi ma'lumot (6-bobda shunday edi). - **Keshlash kerak bo'lsa:** `"use cache"` direktivasini qo'shasiz.

### 8.3. Keshlashni yoqish — `cacheComponents`

"use cache" dan foydalanish uchun avval uni sozlamada yoqamiz:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
 cacheComponents: true, // ← keshlashni yoqdik
};

export default nextConfig;
```

(Sozlamani o'zgartirgach, dev serverni qayta ishga tushiring: `Ctrl+C`, keyin `npm run dev`.)

## 8.4. "use cache" — natijani saqlash

Endi keshlamoqchi bo'lgan funksiya/komponent yuqorisiga "use cache" yozamiz:

```
// app/blog/page.tsx
export default async function BlogPage() {
 const maqolalar = await maqolalarniOl();
 return (

 {maqolalar.map((m: any) => <li key={m.id}>{m.title})}

);
}

async function maqolalarniOl() {
 "use cache"; // ← bu funksiya natijasi keshlanadi (saqlanadi)
 const res = await
 fetch("https://jsonplaceholder.typicode.com/posts");
 return res.json();
}
```

Endi `maqolalarniOl()` natijasi **saqlanadi**. Birinchi foydalanuvchi ma'lumotni serverdan oladi, keyingilar — saqlangandan (tez).

"use cache" ni butun sahifa (`page.tsx`) yuqorisiga ham, alohida funksiyaga ham, alohida komponentga ham qo'yish mumkin.

## 8.5. Kesh muddati — `cacheLife`

Kesh **abadiy** saqlanmasligi kerak — aks holda ma'lumot eskirib qoladi. `cacheLife` bilan "qancha muddat saqlansin" deymiz:

```
import { cacheLife } from "next/cache";

async function maqolalarniOl() {
 "use cache";
 cacheLife("hours"); // ← soatlab saqla, keyin yangila
 const res = await
 fetch("https://jsonplaceholder.typicode.com/posts");
 return res.json();
}
```

### Tayyor muddat profillari:

Profil	Ma'nosi
"seconds"	bir necha soniya (tez o'zgaradigan ma'lumot)
"minutes"	daqiqalar
"hours"	soatlar (masalan, yangiliklar)
"days"	kunlar (masalan, blog)
"weeks"	haftalar
"max"	juda uzoq (deyarli o'zgarmaydigan ma'lumot)

O'z muddatingizni ham berishingiz mumkin:

```
cacheLife({ revalidate: 3600 }); // har 3600 soniya (1 soat) da yangila
```

**Tanlash:** Ma'lumot qanchalik tez o'zgarsa — shunchalik qisqa muddat. Narxlar — `minutes`. Blog — `days`. "Biz haqimizda" sahifasi — `max`.

## 8.6. Aniq nishonli yangilash — `cacheTag` va `revalidateTag`

Faraz qiling, bitta maqolani tahrirladingiz. Butun keshni tashlash shart emas — faqat **o'sha maqolaning** keshini yangilash kerak. Buning uchun keshga **belgi (tag)** qo'yamiz:

```
import { cacheLife, cacheTag } from "next/cache";

async function maqolaOl(id: string) {
 "use cache";
```

```

 cacheLife("days");
 cacheTag(`maqola-${id}`); // ← bu keshga belgi qo'ydik
 const res = await
 fetch(`https://jsonplaceholder.typicode.com/posts/${id}`);
 return res.json();
 }

```

Endi o'sha maqola o'zgarganda (masalan, Server Action'da tahrirlanganda), faqat shu belgili keshni yangilaymiz:

```

// app/actions.ts
"use server";

import { revalidateTag } from "next/cache";

export async function maqolaTahrirlash(id: string, yangiMatn: string) {
 // ... bazada maqolani yangilash ...

 revalidateTag(`maqola-${id}`); // ← faqat shu maqola keshini yangila
}

```

**Farqi:** - `revalidatePath("/blog")` — butun `/blog` sahifa keshini tashlaydi (qo'pol). - `revalidateTag("maqola-5")` — faqat 5-maqola keshini yangilaydi (aniq, samaraliroq).

**Eslatma:** Next.js 16'da `revalidateTag` ning yangi ko'rinishi bor:

`revalidateTag("maqola-5", "max")` (ikkinchi argument — muddat profili). Faqat bitta argument bilan ishlatish (`revalidateTag("maqola-5")`) eskirgan deb ogohlantirish berishi mumkin.

## 8.7. `updateTag` — "men hozirgina o'zgartirdim, darrov ko'rsat"

Yana bir foydali narsa: `updateTag`. Foydalanuvchi biror narsani tahrirlasa va **o'zgarishni darrov** ko'rishi kerak bo'lsa (kutmasdan):

```

// app/actions.ts
"use server";

import { updateTag } from "next/cache";

export async function profilYangilash(userId: string, malumot: any) {
 // ... bazada profilni yangilash ...

 updateTag(`user-${userId}`); // ← keshni darrov tozala va yangi
 ma'lumotni ko'rsat
}

```

**revalidateTag va updateTag farqi:** - `revalidateTag` — keshni "eskirgan" deb belgilaydi, fonda yangilanadi (boshqalar uchun). - `updateTag` — keshni **darrov** tozalaydi va yangi ma'lumotni ko'rsatadi (o'zgartirgan foydalanuvchi uchun ideal).

## 8.8. MUHIM: dev va production'da kesh boshqacha ishlaydi

⚠ **Eng ko'p uchraydigan chalkashlik:** Keshlash `npm run dev` (ishlab chiqish rejimi) va haqiqiy ishlab turgan sayt (production) da **boshqacha** ishlaydi.

Keshni to'g'ri sinash uchun **production rejimida** tekshiring:

```
npm run build # loyihani "ishlab chiqarish" uchun tayyorlaydi
npm run start # production rejimida ishga tushiradi
```

`npm run dev` da kesh ko'pincha o'chirib turiladi (har o'zgarishni ko'rishingiz uchun). Shuning uchun "kesh ishlamayapti" deb o'ylamang — production'da sinang.

## 8.9. Amaliy strategiya (qachon nima qilish)

Boshlovchi uchun oddiy qoidalar:

1. **Boshda hech narsani keshlamang.** Default (dinamik) bilan boshlang — to'g'ri ishlasin.
2. **Sekin yoki tez-tez ochiladigan sahifani keshlang:** bosh sahifa, blog, "biz haqimizda" — `"use cache" + cacheLife`.
3. **Tez o'zgaradigan ma'lumotni keshlamang yoki qisqa keshlang:** savat, bildirishnomalar, real-time hisoblar.
4. **Ma'lumot o'zgarganda yangilang:** Server Action'da `revalidateTag / revalidatePath`.
5. **Doim production'da sinang** (`build + start`).

**Eslatma:** Keshlash — chuqur mavzu va tajriba bilan o'rganiladi. Hozir asoslarni tushunsangiz yetadi. Real loyihalarda ishlab, sekin ustasi bo'lasiz.

## 8-BOB MASALALARI (20 ta)

Bu bob ilg'or — masalalarning bir qismi amaliy, bir qismi tushunishga qaratilgan. Amaliy mashqlar uchun keshlashni yoqing (`cacheComponents: true`).

**Tushunish (1–8):** 1. Kesh nima? Onaning osh retsepti misolida tushuntiring. 2. Keshlashning 3 ta foydasini ayting. 3. Next.js 16 va eski versiyalarda keshlash farqi nima? 4. "Sukut bo'yicha dinamik" nima degani? 5. `"use cache"` qayerga yoziladi va nima qiladi? 6. `cacheLife` nima uchun kerak? Agar muddat bermasak nima bo'ladi (ma'lumot eskirsa)? 7. `revalidatePath` va `revalidateTag` farqi nima? 8. `revalidateTag` va `updateTag` farqi nima?

**Amaliy (9–16):** 9. `next.config.ts` da `cacheComponents: true` qo'shing va dev serverni qayta ishga tushiring. 10. `/posts` ni oladigan funksiyani `"use cache"` bilan keshlang. 11. Unga `cacheLife("hours")` qo'shing. 12. `cacheLife` da `minutes`, `days`, `max` profillarini sinab ko'ring. 13. `cacheLife({ revalidate: 60 })` bilan o'z muddatingizni bering. 14. Bitta maqola funksiyasiga `cacheTag(\ maqola-${id})` qo'shing. 15. Server Action yarating – `revalidateTag(...)` bilan o'sha maqola keshini yangilasin. 16. `npm run build` va `npm run start` bilan production rejimida ishga tushiring.

**Strategiya (17–20):** 17. Quyidagilardan qaysi birini keshlash kerak, qaysi birini yo'q: (a) "biz haqimizda", (b) foydalanuvchi savati, (c) blog maqolasi, (d) real-vaqt narx kursi? Izohlang. 18. Blog maqolasi uchun qaysi `cacheLife` profili to'g'ri? Narxlar sahifasi uchun-chi? 19. Nega keshni production'da sinash kerak, dev'da emas? 20. O'z so'zingiz bilan: "opt-in keshlash" qoidasini amaliy strategiya sifatida yozing (3-4 qadam).

**Qo'shimcha challenge:** 6-bobda yasagan blog loyihangizni oling. Maqolalar ro'yxati va har bir maqolani `"use cache"` + `cacheLife("days")` + `cacheTag(...)` bilan keshlang. Keyin Server Action yarating, u `revalidateTag` bilan kerakli maqolani yangilasin. `build` + `start` bilan production'da sinang va tezlik farqini sezing.

---

## 2-QISM YAKUNI

Zo'r ish! Endi sizda Next.js'ning eng muhim "haqiqiy ilova" ko'nikmalari bor:

✅ **Server va Client Components** farqini tushunasiz va to'g'ri ishlatasiz ( `"use client"` siri ochildi!) ✅ **Serverdan ma'lumot olasiz** – `async` Server Component'da, `useEffect` siz, oddiy va tez ✅ **loading.tsx** va **error.tsx** bilan yuklanish va xatoni boshqarasiz ✅ **Server Actions** bilan forma yuborib, ma'lumotni serverga saqlaysiz (validatsiya + `useActionState`) ✅ **Keshlash** asoslarini bilasiz – `"use cache"`, `cacheLife`, `cacheTag`, `revalidateTag`

Bu — haqiqiy, dinamik veb-ilova yasash uchun yetarli poydevor. Hali bitta katta narsa yetishmaydi: **doimiy ma'lumotni saqlash** (ma'lumotlar bazasi). Hozir `console.log` qilgan ma'lumotlar yo'qolib ketadi — 3-qismda ularni haqiqiy bazaga saqlashni o'rganamiz.

## Keyingi qadam: 3-QISM nimani o'z ichiga oladi?

- **Route Handlers (API)** — tashqi dasturlar (mobil ilova, boshqa saytlar) uchun API yaratish
- **Ma'lumotlar bazasi** — ma'lumotni haqiqatda saqlash (PostgreSQL/MySQL, Prisma yoki Drizzle ORM bilan)
- **Autentifikatsiya** — to'liq login/register tizimi, sessiyalar, himoyalangan sahifalar
- **Validatsiya (Zod)**, fayl yuklash, xavfsizlik amaliyotlari

**Maslahat:** 3-qismga o'tishdan oldin, bu qismdagi "blog" va "guestbook" challenge'larini bajaring. Server Components, data fetching va Server Actions'ni amaliyotda mustahkamlang — keyin baza qo'shganda hammasi joyiga tushadi.

---

*Bu — "Next.js 0 dan Expert darajagacha" qo'llanmasining 2-qismi.*

---

 [README](#) · [← 1-qism — Asoslar](#) · [Keyingi: 3-qism — Full-stack](#) →

# Next.js — 0 dan Expert darajagacha (3-QISM)

 [README](#) · [← 2-qism — Server & Actions](#) · [Keyingi: 4-qism — Production](#) →

**Eslatma:** Bu — qo'llanmaning 3-qismi. 1-qism (JS, React, routing) va 2-qism (Server/Client Components, data fetching, Server Actions, caching)ni o'rgangan bo'lishingiz kerak. Bu qism ularning ustiga quriladi.

**Versiya:** Next.js **16**, React 19. Node.js **20.9+**.

## Bu qismda nimalarni o'rganamiz

2-qism oxirida bitta katta muammo qolgandi: ma'lumotlar `console.log` da yo'qolib ketardi. Bu qism aynan shu — ma'lumotni **haqiqatan saqlash** va **foydalanuvchilarni himoyalash** haqida. Bu — "to'liq full-stack dasturchi" bo'lishingiz boshlanadigan joy.

- **9-bob. Route Handlers** — tashqi dasturlar (mobil ilova, boshqa saytlar) uchun API yaratish
- **10-bob. Ma'lumotlar bazasi (Prisma)** — ma'lumotni haqiqatan saqlash
- **11-bob. Validatsiya (Zod)** — ma'lumotni tekshirib, xavfsiz qabul qilish
- **12-bob. Autentifikatsiya (Better Auth)** — login/register, sessiyalar, himoyalangan sahifalar + xavfsizlik

**Maslahat:** Bu qism eng "texnik" qism. Ba'zi narsalar (bazani o'rnatish, auth sozlash) ko'p qadamli. Shoshilmang, har qadamni o'zingiz bajaring. Xato chiqsa — bu normal, hammamiz shunday o'rganganmiz.

---

## 9-BOB. Route Handlers — o'z API'ngizni yaratish

2-qismda **Server Actions** bilan forma ma'lumotini serverga yubordik. Bu — **ilova ichidagi** ishlar uchun ideal. Lekin ba'zan bizga **tashqi dunyoga ochiq** API kerak bo'ladi. Mana shu yerda **Route Handlers** ishga tushadi.

# 9.1. API nima va nega kerak?

**API** (Application Programming Interface) — bu dasturlar bir-biri bilan **gaplashish usuli**. Bir dastur ma'lumot so'raydi, ikkinchisi javob beradi.

**Misol:** Restoran menyusi — bu API'ga o'xshaydi. Siz "1-taom" deysiz (so'rov), oshxona o'sha taomni beradi (javob). Menyuda aniq qoidalar bor: nima so'rasa bo'ladi, nima qaytadi.

**Qachon API kerak?** - 📱 Mobil ilova (masalan, Flutter/React Native) saytingiz ma'lumotini olishi kerak bo'lganda - 🔗 Boshqa sayt sizning ma'lumotingizni ishlatganda - 📡 Webhook — boshqa xizmat (masalan, to'lov tizimi) sizga xabar yuborganda - 🌐 Frontend'dan ma'lumotni alohida olish kerak bo'lganda

# 9.2. Server Actions vs Route Handlers — qaysi birini qachon?

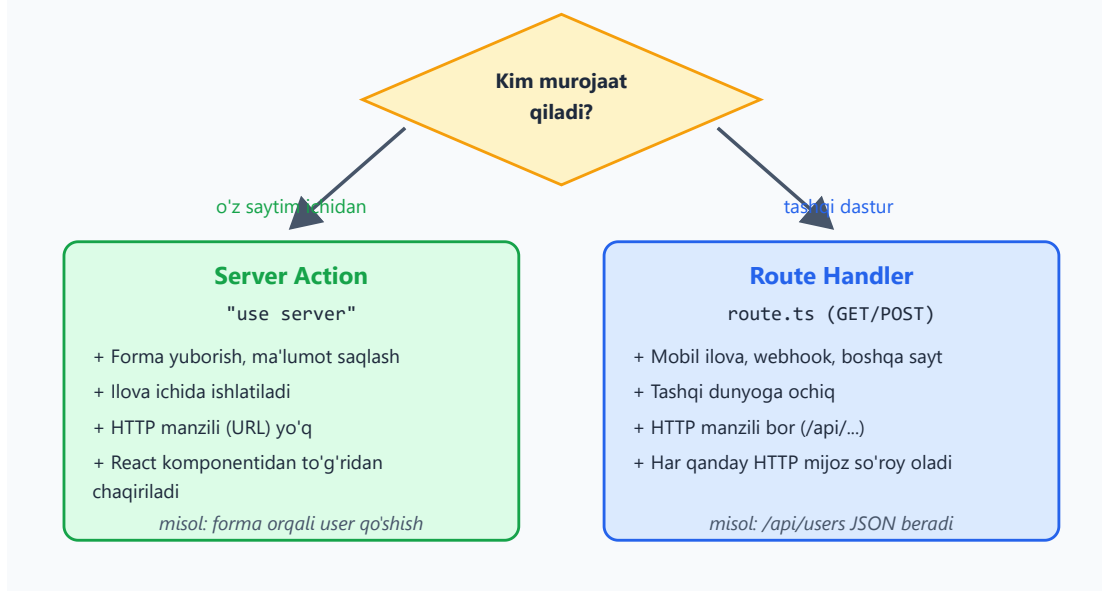
Bu — boshlovchilarni ko'p chalkashtiradigan savol. Oddiy qoida:

Server Actions (2-qism)	Route Handlers (9-bob)
Ilova ichida ishlatiladi	Tashqi dunyoga ochiq
Forma yuborish, ma'lumot saqlash	Mobil ilova, webhook, boshqa saytlar
"use server" funksiya	route.ts fayli (GET/POST)
HTTP manzili yo'q	HTTP manzili bor ( /api/... )

**Sodda qoida:** Saytingiz **o'z formasi** ma'lumotini saqlayaptimi? → **Server Action**. Tashqi dastur ma'lumot so'rayaptimi? → **Route Handler**.

Quyidagi sxema bu qarorni ko'rsatadi — kim murojaat qilishiga qarab qaysi birini tanlash kerak:

## Server Action yoki Route Handler — qaysi qachon?



### 9.3. Birinchi Route Handler

Routing'ni eslang (1-qism, 4-bob): papka = sahifa. Route Handler ham xuddi shunday, faqat `page.tsx` o'rniga `route.ts` fayli yozamiz.

```
app/
├── api/
│ └── salom/
│ └── route.ts
```

```
// app/api/salom/route.ts
import { NextResponse } from "next/server";

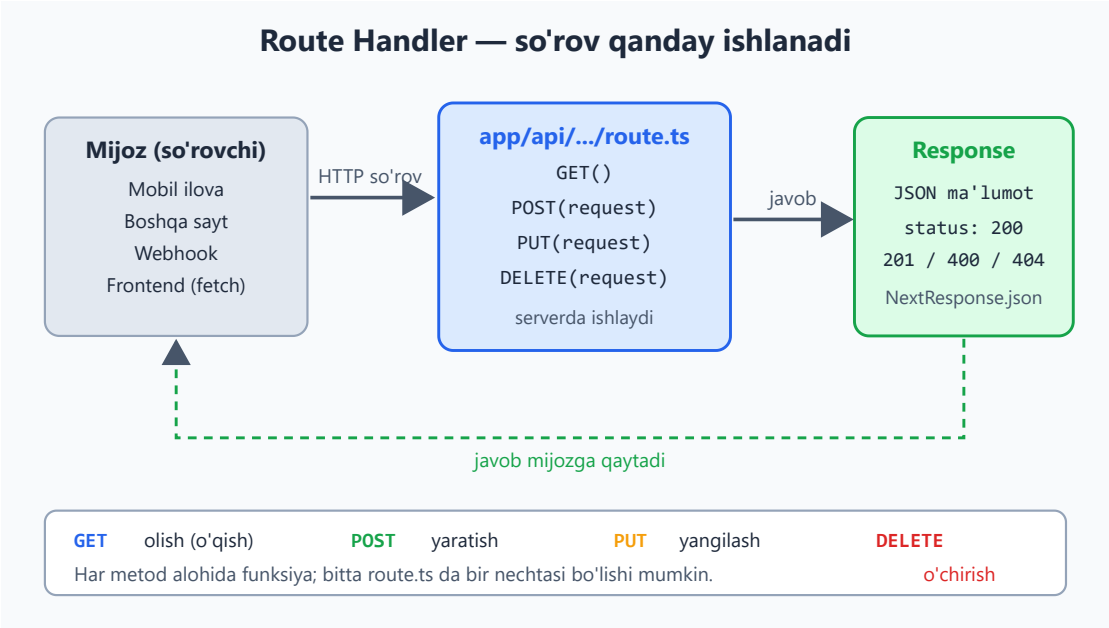
export async function GET() {
 return NextResponse.json({ xabar: "Salom, dunyo!" });
}
```

Endi brauzerda `localhost:3000/api/salom` ni ochsangiz, JSON ko'rinishidagi javob chiqadi:

```
{ "xabar": "Salom, dunyo!" }
```

**Tushuntirish:** - `route.ts` — bu API yo'naki (sahifa emas, ma'lumot qaytaradi). - `export async function GET()` — **GET** so'roviga javob beradigan funksiya. - `NextResponse.json(...)` — JSON ko'rinishidagi javob qaytaradi.

Umumiy oqim quyidagicha: HTTP so'rov kelib, `route.ts` ichidagi mos handlerga (GET/POST/PUT/DELETE) tushadi va u Response qaytaradi:



## 9.4. HTTP metodlar — GET, POST, PUT, DELETE

Internetda so'rovlarning turli **metodlari** bor. Har birining o'z vazifasi:

Metod	Vazifasi	Misol
<b>GET</b>	Ma'lumot <b>olish</b> (o'qish)	Foydalanuvchilar ro'yxatini olish
<b>POST</b>	Yangi ma'lumot <b>yaratish</b>	Yangi foydalanuvchi qo'shish
<b>PUT / PATCH</b>	Ma'lumotni <b>yangilash</b>	Profilni tahrirlash
<b>DELETE</b>	Ma'lumotni <b>o'chirish</b>	Hisobni o'chirish

Bitta `route.ts` da bir nechta metod bo'lishi mumkin:

```
// app/api/users/route.ts
import { NextResponse } from "next/server";

// GET - foydalanuvchilarni olish
export async function GET() {
 const users = [
 { id: 1, ism: "Ali" },
 { id: 2, ism: "Vali" },
];
}
```

```

 return NextResponse.json(users);
 }

 // POST – yangi foydalanuvchi qo'shish
 export async function POST(request: Request) {
 const malumot = await request.json(); // tanadagi (body) ma'lumotni o'qish
 console.log("Yangi foydalanuvchi:", malumot);
 return NextResponse.json({ muvaffaqiyat: true }, { status: 201 });
 }

```

- `request.json()` — so'rov **tanadagi** (body) yuborilgan ma'lumotni o'qiydi.
- `{ status: 201 }` — HTTP holat kodi (201 = "yaratildi"). 200 = OK, 404 = topilmadi, 500 = server xatosi.

## 9.5. Dinamik API yo'naklari — `[id]`

Dinamik sahifalar (`[id]`) kabi, API yo'naklarida ham dinamik segment bo'lishi mumkin:

```
app/api/users/[id]/route.ts
```

⚠ **Next.js 16 muhim:** Bu yerda ham `params` **asinxron** (Promise). Sahifalardagi kabi, uni `await` qilish kerak.

```

// app/api/users/[id]/route.ts
import { NextResponse } from "next/server";

export async function GET(
 request: Request,
 { params }: { params: Promise<{ id: string }> }
) {
 const { id } = await params; // Next.js 16 – async params

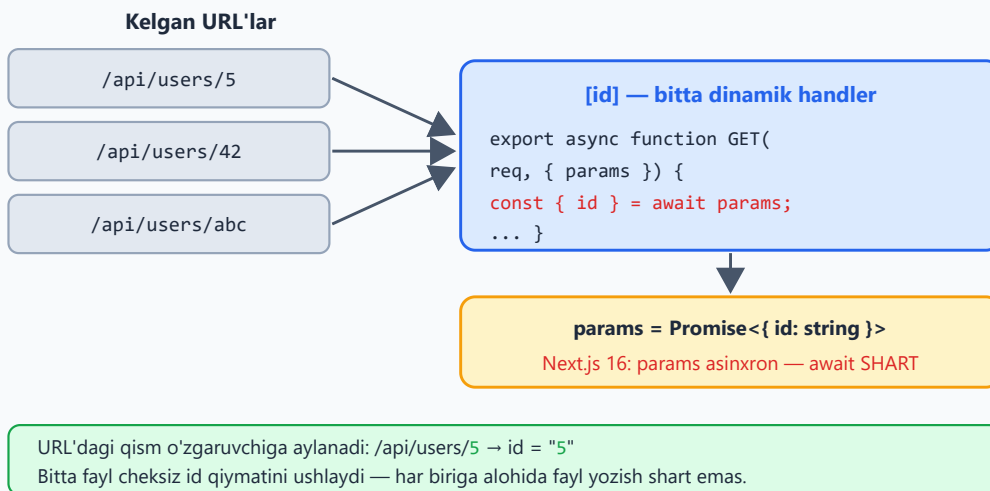
 return NextResponse.json({ id, ism: `Foydalanuvchi #${id}` });
}

```

`/api/users/5` ga GET so'rov yuborsangiz — `{ "id": "5", "ism": "Foydalanuvchi #5" }`.

Quyidagi sxema bitta `[id]` handler turli URL qiymatlarini qanday ushlashini ko'rsatadi (Next.js 16'da `params` — Promise, `await` shart):

## Dinamik segment — app/api/users/[id]/route.ts



## 9.6. So'rov parametrlarini o'qish (query params)

URL'dagi `?` dan keyingi parametrlarni o'qish (masalan, `/api/search?q=nextjs`):

```
// app/api/search/route.ts
import { NextResponse } from "next/server";

export async function GET(request: Request) {
 const { searchParams } = new URL(request.url);
 const qidiruv = searchParams.get("q"); // ?q=... qiymatini oladi

 return NextResponse.json({ qidiruv, natijalar: [] });
}
```

`/api/search?q=nextjs` → `{ "qidiruv": "nextjs", "natijalar": [] }`.

## 9.7. ⚠ Next.js 16'da GET keshlanmaydi

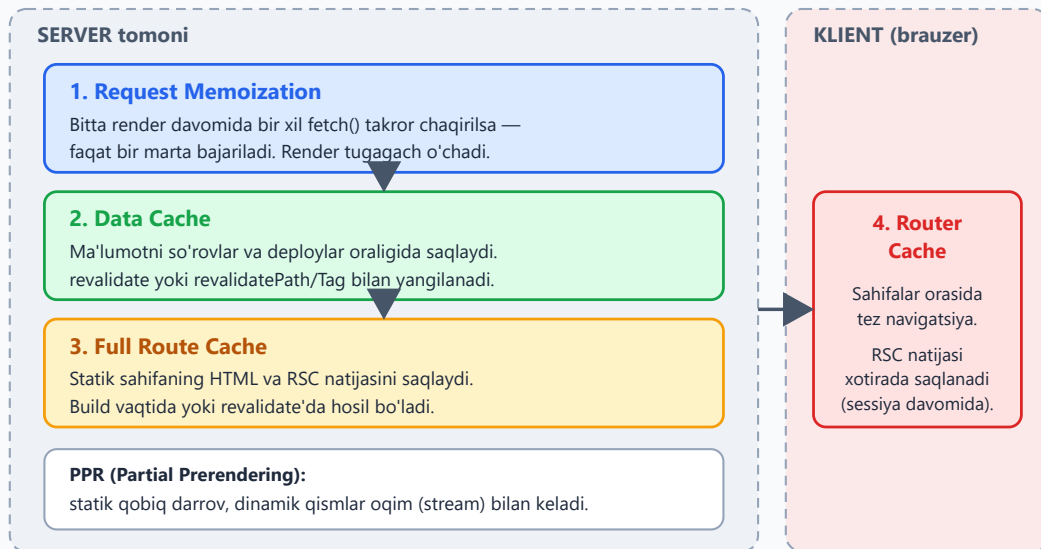
**Next.js 16 o'zgarishi:** 2-bobda `fetch` haqida aytganimizdek, Route Handler'dagi **GET** ham endi sukut bo'yicha **keshlanmaydi** (har safar yangi ishlaydi). Eski versiyalarda GET avtomatik keshlanardi.

Agar GET natijasini keshlash kerak bo'lsa, 2-qismdagi (8-bob) `"use cache"` mexanizmidan foydalanasiz yoki maxsus sozlama qo'shasiz. Hozircha: GET har safar yangi ma'lumot beradi.

Umuman, App Router'da ma'lumot bir nechta kesh qatlamidan o'tadi — uchta serverda, bittasi brauzerda. Quyidagi sxema bu qatlamlarni tartib bilan ko'rsatadi:

## Next.js caching qatlamlari (App Router)

So'rov 4 qatlamdan o'tadi: 3 tasi serverda, 1 tasi brauzerda



Next.js 16: `fetch` va `GET` sukut bo'yicha keshlanmaydi; keshlash uchun `"use cache"` yoki `revalidate` kerak.

## 9.8. To'liq misol: oddiy API

```
// app/api/posts/route.ts
import { NextResponse } from "next/server";

// Vaqtincha "baza" (haqiqiy baza 10-bobda)
let posts = [
 { id: 1, sarlavha: "Birinchi maqola" },
 { id: 2, sarlavha: "Ikkinchi maqola" },
];

// Hamma maqolalarni olish
export async function GET() {
 return NextResponse.json(posts);
}

// Yangi maqola qo'shish
export async function POST(request: Request) {
 const { sarlavha } = await request.json();

 if (!sarlavha) {
 return NextResponse.json(
 { xato: "Sarlavha kerak" },
 { status: 400 } // 400 = noto'g'ri so'rov
);
 }

 const yangi = { id: posts.length + 1, sarlavha };
 posts.push(yangi);
}
```

```
return NextResponse.json(yangi, { status: 201 });
}
```

Bu API'ni mobil ilova ham, boshqa sayt ham ishlatishi mumkin. Tabriklaymiz — birinchi API'ngizni yasadingiz!

**Sinash:** GET'ni brauzerda ochsangiz bo'ladi. POST'ni sinash uchun **Postman**, **Thunder Client** (VS Code kengaytmasi) yoki `curl` kabi vositalar kerak (brauzerda oddiy ochib bo'lmaydi).

## 9-BOB MASALALARI (20 ta)

**Tushunish (1–6):** 1. API nima? Restoran menyusi misolida tushuntiring. 2. Server Actions va Route Handlers farqi nima? Qaysi birini qachon ishlatasiz? 3. GET, POST, PUT, DELETE — har biri nima qiladi? 4. HTTP holat kodlari nima: 200, 201, 400, 404, 500? 5. `route.ts` va `page.tsx` farqi nima? 6. Qachon API kerak bo'ladi? 3 ta misol ayting.

**Asosiy Route Handlers (7–14):** 7. `app/api/salom/route.ts` yaratib, GET so'roviga `{ xabar: "Salom" }` qaytaring. 8. Brauzerda `/api/salom` ni ochib, javobni ko'ring. 9. `/api/vaqt` yaratib, hozirgi vaqtni (`new Date()`) qaytaring. 10. `/api/users` da GET — 3 ta foydalanuvchi massivini qaytaring. 11. Shu route'ga POST qo'shing — `request.json()` bilan ma'lumot oling va `console.log` qiling. 12. POST javobida `status: 201` qaytaring. 13. POST'da sarlavha bo'sh bo'lsa, `status: 400` va xato xabari qaytaring. 14. Thunder Client (VS Code) yoki Postman bilan POST so'rovini sinab ko'ring.

**Dinamik va parametrlar (15–20):** 15. `/api/users/[id]/route.ts` yaratib, `id` ni qaytaring (async params!). 16. `/api/users/5` ga kirib, to'g'ri javob kelishini tasdiqlang. 17. `/api/posts/[id]` yaratib, bitta maqola obyektni qaytaring. 18. `/api/search` yaratib, `?q=...` parametrini o'qing va qaytaring. 19. `/api/search?q=nextjs` ni ochib, qidiruv qiymatini ko'ring. 20. `/api/calc?a=5&b=3` yaratib, ikki sonni qo'shgan natijani qaytaring (`Number(...)` bilan).

**Qo'shimcha challenge:** To'liq **"vazifalar" (todo) API** yarating: GET (hammani olish), POST (qo'shish, validatsiya bilan), va `/api/todos/[id]` uchun GET (bittani olish). Vaqtincha massivda saqlang (haqiqiy baza keyingi bobda). Thunder Client bilan hammani sinab ko'ring.

---

# 10-BOB. Ma'lumotlar bazasi (Prisma)

Mana eng muhim qadam: ma'lumotni **doimiy saqlash**. Hozirgacha `console.log` qilgan ma'lumotlar server o'chsa yo'qolardi. Endi ularni **ma'lumotlar bazasiga** saqlaymiz — u ma'lumotni abadiy eslab qoladi.

## 10.1. Ma'lumotlar bazasi nima?

**Ma'lumotlar bazasi (database)** — bu ma'lumotni **tartibli saqlaydigan** va server o'chsa ham **yo'qotmaydigan** maxsus tizim.

**Misol:** `console.log` — bu bo'r bilan doskaga yozishga o'xshaydi: doska o'chirilsa, hammasi yo'qoladi. **Baza** — bu daftarga ruchka bilan yozish: yopib qo'ysangiz ham, qaytib ochsangiz — hammasi joyida.

**Baza nima saqlaydi?** Foydalanuvchilar, maqolalar, izohlar, buyurtmalar — har qanday ma'lumotni. U ma'lumotni **jadvallar (tables)**da saqlaydi. Jadval — Excel jadvaliga o'xshaydi: ustunlar (ism, email, yosh) va qatorlar (har bir foydalanuvchi).

**Baza turlari:** - **SQL bazalari** (PostgreSQL, MySQL, SQLite) — eng keng tarqalgan, tartibli, ishonchli. Biz shularni ishlatamiz. - **NoSQL bazalari** (MongoDB) — boshqacha tuzilishda. Hozir kerak emas.

**Boshlash uchun:** Biz **SQLitedan** boshlaymiz — u hech narsa o'rnatishni talab qilmaydi, oddiygina fayl (`dev.db`). O'rganish uchun ideal. Keyincha haqiqiy loyihada PostgreSQL'ga (Neon, Supabase yoki Prisma Postgres) o'tasiz.

## 10.2. ORM va Prisma nima?

Baza o'zining tiliga ega — **SQL** (`SELECT * FROM users WHERE age > 18`). Buni o'rganish kerak, lekin biroz qiyin.

**ORM** (Object-Relational Mapping) — bu **tarjimon**. Siz oddiy JavaScript yozasiz, ORM uni SQL'ga tarjima qiladi.

Siz `prisma.user.findMany()` deysiz → Prisma uni `SELECT * FROM users` ga aylantiradi. Siz SQL bilmasangiz ham, baza bilan ishlay olasiz!

**Prisma** — eng mashhur va boshlovchiga qulay ORM. Type-safe (xatolarni oldindan tutadi), zo'r hujjatli. Biz shuni ishlatamiz.

## 10.3. Prisma'ni o'rnatish

Loyihangiz papkasida terminal ochib:

```
1. Prisma'ni o'rnatamiz
npm install prisma --save-dev
npm install @prisma/client

2. Prisma'ni ishga tushiramiz (SQLite bilan)
npx prisma init --datasource-provider sqlite
```

Bu 2 ta narsa yaratadi: - `prisma/schema.prisma` — baza "**chizmasi**" (qanday ma'lumot saqlanishini belgilaydi). - `.env` — maxfiy sozlamalar (baza manzili).

`.env` faylida shunday qator bo'ladi:

```
DATABASE_URL="file:./dev.db"
```

(SQLite uchun — bu oddiy fayl, hech narsa sozlash kerak emas.)

## 10.4. Schema — baza chizmasini yozish

`prisma/schema.prisma` faylini oching. `generator` va `datasource` bloklari avtomatik yozilgan bo'ladi — ularga tegmang. Siz faqat **modellar** qo'shasiz.

**Model** — bu jadval ta'rifi. Masalan, foydalanuvchi modeli:

```
// prisma/schema.prisma

generator client {
 provider = "prisma-client-js"
}

datasource db {
 provider = "sqlite"
 url = env("DATABASE_URL")
}

// Bizning birinchi modelimiz:
model User {
 id Int @id @default(autoincrement())
 email String @unique
 name String?
 createdAt DateTime @default(now())
}
```

**Tushuntirish:** - `model User` — `User` nomli jadval yaratadi. - `id Int @id` `@default(autoincrement())` — `id` ustuni, raqamli, asosiy kalit (`@id`), avtomatik oshadi (1, 2, 3...). - `email String @unique` — matn, **takrorlanmas** (`@unique` — ikki bir xil email bo'lmaydi). - `name String?` — matn, **ixtiyoriy** (`?` — bo'sh bo'lishi mumkin). - `createdAt DateTime @default(now())` — yaratilgan vaqt, avtomatik hozirgi vaqt.

## 10.5. Migration — chizmani bazaga qo'llash

Modelni yozdingiz, lekin bazada hali jadval yo'q. **Migration** — chizmani haqiqiy bazaga aylantirishdir.

Migration — uy chizmasini olib, haqiqiy uy qurishga o'xshaydi. Chizma o'zgarsa (yangi model), qaytadan "qurasiz".

```
npx prisma migrate dev --name init
```

Bu buyruq: 1. Bazada (`dev.db`) `User` jadvalini yaratadi. 2. Prisma Client'ni generatsiya qiladi (JavaScript'dan bazaga murojaat qilishingiz uchun).

**Schemani har o'zgartirganingizda** (yangi model/ustun qo'shganda), shu buyruqni qayta ishga tushiring (`--name` keyinga mazmunli nom bilan, masalan `--name add-posts`).

## 10.6. Prisma Client — bazaga ulanish

JavaScript'dan bazaga murojaat qilish uchun **Prisma Client** ishlatamiz. Avval uni bir joyda sozlaymiz (`lib/prisma.ts`):

```
// lib/prisma.ts
import { PrismaClient } from "@prisma/client";

const globalForPrisma = globalThis as unknown as { prisma?: PrismaClient };

export const prisma = globalForPrisma.prisma ?? new PrismaClient();

if (process.env.NODE_ENV !== "production") {
 globalForPrisma.prisma = prisma;
}
```

**Nega bunaqa murakkab?** Next.js ishlab chiqishda kodni tez-tez qayta yuklaydi (hot-reload). Agar har safar yangi `PrismaClient` yaratsak, bazaga juda ko'p ulanish ochilib ketadi. Bu kod faqat **bitta** Client bo'lishini ta'minlaydi. Hozir tushunmasangiz ham, shu kodni nusxalab ishlating — bu standart amaliyot.

⚠ **Eslatma:** Yangi Prisma versiyalarida Client boshqa joyga (masalan `app/generated/prisma`) generatsiya qilinishi mumkin. Agar `@prisma/client` dan `import xato` bersa, `schema.prisma` dagi `generator` blokidagi `output` yo'lini tekshiring va `import` yo'lini o'shanga moslang (yoki Prisma'ning rasmiy Next.js qo'llanmasini ko'ring: [prisma.io/docs](https://prisma.io/docs)).

## 10.7. CRUD — baza bilan 4 ta asosiy amal

**CRUD** = **Create** (yaratish), **Read** (o'qish), **Update** (yangilash), **Delete** (o'chirish). Baza bilan ishlashning asosi.

```
import { prisma } from "@lib/prisma";

// CREATE — yangi foydalanuvchi yaratish
const yangiUser = await prisma.user.create({
 data: { email: "ali@example.com", name: "Ali" },
});

// READ — hammani o'qish
const hammaUserlar = await prisma.user.findMany();

// READ — bittani o'qish (id bo'yicha)
const bittaUser = await prisma.user.findUnique({
 where: { id: 1 },
});

// UPDATE — yangilash
const yangilangan = await prisma.user.update({
 where: { id: 1 },
 data: { name: "Ali Valiyev" },
});

// DELETE — o'chirish
await prisma.user.delete({
 where: { id: 1 },
});
```

Ko'ryapsizmi? SQL umuman yozmadik. Oddiy JavaScript — Prisma qolganini o'zi bajaradi.

## 10.8. Bazani Server Component'da o'qish (haqiqiy ma'lumot!)

2-qism (6-bob)da `fetch` bilan tashqi API'dan o'qigan edik. Endi **o'z bazamizdan** o'qiymiz — xuddi shunchalik oddiy:

```
// app/users/page.tsx – Server Component
import { prisma } from "@/lib/prisma";

export default async function UsersPage() {
 const users = await prisma.user.findMany(); // ← bevosita bazadan!

 return (
 <ul className="p-8">
 {users.map((user) => (
 <li key={user.id}>
 {user.name} – {user.email}

))}

);
}
```

`fetch` ham, API ham kerak emas! Server Component bevosita bazaga murojaat qildi. Bu — Next.js'ning eng kuchli tomoni (5-bobni eslang: Server Component serverda ishlaydi, shuning uchun bazaga to'g'ridan-to'g'ri kira oladi).

## 10.9. Bazaga Server Action bilan saqlash (haqiqiy!)

2-qism (7-bob)da Server Action bilan ma'lumotni `console.log` qilgan edik. Endi uni **haqiqatan saqlaymiz**:

```
// app/actions.ts
"use server";

import { prisma } from "@/lib/prisma";
import { revalidatePath } from "next/cache";

export async function userQoshish(formData: FormData) {
 const email = formData.get("email") as string;
 const name = formData.get("name") as string;

 // HAQIQATAN bazaga saqlash:
 await prisma.user.create({
 data: { email, name },
 });

 revalidatePath("/users"); // ro'yxatni yangila – yangi user darrov ko'rinadi
}
```

Endi forma orqali qo'shilgan foydalanuvchi **haqiqiy bazaga** saqlanadi va sahifani yangilasangiz ham — joyida turadi! Tabriklaymiz — endi siz haqiqiy full-stack dasturchisiz!

## 10.10. Prisma Studio — bazani ko'z bilan ko'rish

Bazada nima borligini ko'rish uchun Prisma maxsus vosita beradi:

```
npx prisma studio
```

Bu brauzerda bazangizni **jadval ko'rinishida** ochadi — ma'lumotni ko'rish, qo'shish, o'chirish mumkin. Juda foydali.

## 10.11. Bog'langan ma'lumotlar (Relations)

Real loyihada ma'lumotlar **bir-biriga bog'langan** bo'ladi. Masalan, har bir maqola bir foydalanuvchiga tegishli. Buni schemada ko'rsatamiz:

```
model User {
 id Int @id @default(autoincrement())
 email String @unique
 name String?
 posts Post[] // ← bir userning ko'p maqolasi bor
}

model Post {
 id Int @id @default(autoincrement())
 title String
 author User @relation(fields: [authorId], references: [id])
 authorId Int // ← qaysi userga tegishli
}
```

Endi maqolani muallifi bilan birga olishingiz mumkin:

```
const posts = await prisma.post.findMany({
 include: { author: true }, // ← muallifni ham qo'shib ber
});
// Endi har bir post.author.name mavjud
```

Bu — ilg'orroq mavzu. Hozir asosning (CRUD) ustasi bo'ling, relationlarni keyincha chuqurroq o'rganasiz.

## 10-BOB MASALALARI (22 ta)

**Tushunish (1–6):** 1. Ma'lumotlar bazasi nima? `console.log` dan farqi (doska vs daftar) nima? 2. ORM nima? Prisma nima qiladi (tarjimon analogiyasi)? 3. SQLite nega o'rganish uchun qulay? 4. Schema, model, migration — har biri nima? 5. CRUD nima?

To'liq oching (4 ta amal). 6. Nega Server Component bevosita bazaga kira oladi, lekin Client Component yo'q? (5-bobni eslang.)

**O'rnatish va schema (7–12):** 7. Prisma'ni o'rnatish (`npm install + npx prisma init --datasource-provider sqlite`). 8. `.env` da `DATABASE_URL` borligini tekshiring. 9. `schema.prisma` da `User` modeli yarating (`id, email, name, createdAt`). 10. `npx prisma migrate dev --name init` bilan migration qiling. 11. `lib/prisma.ts` faylini yarating (yuqoridagi singleton kod bilan). 12. `npx prisma studio` bilan bazani ochib, bo'sh `User` jadvalini ko'ring.

**CRUD (13–18):** 13. Server Action yarating — `prisma.user.create` bilan yangi user qo'shsin. 14. Forma yarating ( `email`, `name` ), Server Action'ga ulang. 15. `/users` Server Component'da `prisma.user.findMany()` bilan hammani ko'rsating. 16. Forma orqali qo'shgan user ro'yxatda ko'rinishini tasdiqlang ( `revalidatePath` bilan). 17. `/users/[id]` sahifasida `findUnique` bilan bitta user'ni ko'rsating. 18. "O'chirish" tugmasi qo'shing — Server Action `prisma.user.delete` qilsin.

**Ilg'or (19–22):** 19. `User` ga yangi ustun qo'shing (masalan `bio String?`) va qaytadan migration qiling. 20. `prisma.user.update` bilan userning ismini yangilaydigan Server Action yozing. 21. `Post` modelini qo'shing va `User` bilan bog'lang (relation). 22. `include: { author: true }` bilan maqolalarni muallifi bilan birga oling va ko'rsating.

**Qo'shimcha challenge:** 2-qismdagi **guestbook** loyihangizni haqiqiy bazaga ulang: `Message` modeli (`id`, `name`, `text`, `createdAt`). Forma → Server Action → `prisma.message.create`. `Xabarlar / sahifasida` `findMany` bilan (eng yangisi yuqorida — `orderBy: { createdAt: "desc" }`) ko'rinsin. Endi xabarlar server o'chsa ham yo'qolmaydi!

---

## 11-BOB. Validatsiya (Zod)

Bazaga ma'lumot saqlashni o'rgandik. Lekin **bir muammo** bor: foydalanuvchi xohlagan narsani yuborishi mumkin — bo'sh email, 1000 belgili ism, yoki hatto zararli ma'lumot. **Validatsiya** — kelgan ma'lumotni **tekshirib**, faqat to'g'risini qabul qilishdir.

## 11.1. Nega validatsiya muhim?

**Oltin qoida: Hech qachon foydalanuvchiga ishonmang.** Forma frontend'da "to'g'ri" ko'rinsa ham, kimdir uni chetlab o'tib, serverga istalgan narsani yuborishi mumkin.

Validatsiyasiz nima bo'ladi: - ❌ Bo'sh yoki noto'g'ri email bazaga tushadi - ❌ Juda uzun matn bazani buzadi - ❌ Xavfsizlik teshiklari ochiladi (xakerlar foydalanadi) - ❌ Ilova kutilmaganda "qulab" tushadi

**Validatsiya 2 joyda qilinadi:** 1. **Frontend'da** — foydalanuvchiga tez xato ko'rsatish uchun (qulaylik). 2. **Server'da** — HAQIQiy himoya (bu **majburiy**, chetlab o'tib bo'lmaydi).

## 11.2. Zod nima?

**Zod** — ma'lumotni tekshirishning eng mashhur vositasi. Siz "ma'lumot shunday bo'lishi kerak" deb **qoida (schema)** yozasiz, Zod tekshirib beradi.

Zod — bu chegaradagi pasport nazoratchisiga o'xshaydi. "Pasport amal qilishi kerak, yoshi 18+, viza borlig'i shart" — qoidalar. Nazoratchi har kimni shu qoidalar bo'yicha tekshiradi.

O'rnatish:

```
npm install zod
```

## 11.3. Birinchi Zod schema

```
import { z } from "zod";

const userSchema = z.object({
 email: z.string().email("To'g'ri email kiriting"),
 name: z.string().min(2, "Ism kamida 2 belgi bo'lsin"),
 age: z.number().min(18, "18 yoshdan katta bo'lishingiz kerak"),
});
```

**Tushuntirish:** - `z.object({...})` — obyekt qoidasi. - `z.string()` — matn bo'lishi shart. - `.email(...)` — to'g'ri email formati. - `.min(2, ...)` — kamida 2 belgi (xato xabari bilan). - `z.number().min(18)` — son, kamida 18.

**Boshqa foydali qoidalar:**

```
z.string().min(5).max(100) // 5-100 belgi
z.string().url() // to'g'ri havola (URL)
z.number().positive() // musbat son
z.boolean() // true/false
z.string().optional() // ixtiyoriy (bo'lmasa ham mayli)
z.enum(["admin", "user"]) // faqat shu qiymatlardan biri
```

## 11.4. Tekshirish — `safeParse`

Ma'lumotni tekshirish uchun `safeParse` ishlatamiz. U "muvaffaqiyatli" yoki "xato" deb javob beradi (ilova qulamaydi):

```
const natija = userSchema.safeParse({
 email: "notogri",
 name: "A",
 age: 15,
});

if (!natija.success) {
 console.log(natija.error); // xatolar ro'yxati
} else {
 console.log(natija.data); // tekshirilgan, toza ma'lumot
}
```

- `natija.success` — `true` (to'g'ri) yoki `false` (xato).
- `natija.data` — agar to'g'ri bo'lsa, tozalangan ma'lumot.
- `natija.error` — agar xato bo'lsa, xatolar.

## 11.5. Zod'ni Server Action bilan birlashtirish (haqiqiy himoya)

Mana eng muhim qo'llanilishi: Server Action ichida ma'lumotni bazaga saqlashdan **oldin** tekshirish:

```
// app/actions.ts
"use server";

import { z } from "zod";
import { prisma } from "@/lib/prisma";
import { revalidatePath } from "next/cache";

const userSchema = z.object({
 email: z.string().email("To'g'ri email kiriting"),
 name: z.string().min(2, "Ism kamida 2 belgi bo'lsin"),
});

export async function userQoshish(oldingi: any, formData: FormData) {
 // 1. Ma'lumotni yig'amiz
 const xom = {
 email: formData.get("email"),
 name: formData.get("name"),
 };

 // 2. Tekshiramiz
```

```

const natija = userSchema.safeParse(xom);

if (!natija.success) {
 // Xatolarni qaytaramiz (forma ularni ko'rsatadi)
 return {
 xatolar: natija.error.flatten().fieldErrors,
 };
}

// 3. FAQAT to'g'ri bo'lsa – bazaga saqlaymiz
await prisma.user.create({
 data: natija.data, // toza, tekshirilgan ma'lumot
});

revalidatePath("/users");
return { muvaffaqiyat: true };
}

```

`natija.error.flatten().fieldErrors` — bu har bir maydon uchun xatolarni qulay ko'rinishda beradi: `{ email: ["To'g'ri email kiriting"], name: [...] }.`

## 11.6. Xatolarni formada ko'rsatish

2-qism (7-bob)dagi `useActionState` bilan xatolarni ko'rsatamiz:

```

// app/UserFormasi.tsx
"use client";

import { useActionState } from "react";
import { userQoshish } from "../actions";

export default function UserFormasi() {
 const [holat, action, yuborilmoqda] = useActionState(userQoshish, null);

 return (
 <form action={action} className="max-w-md p-8 space-y-3">
 <div>
 <input name="email" placeholder="Email" className="w-full border p-2 rounded" />
 {holat?.xatolar?.email && (
 <p className="text-red-600 text-sm">{holat.xatolar.email[0]}
 </p>
)}
 </div>

 <div>
 <input name="name" placeholder="Ism" className="w-full border p-2 rounded" />
 {holat?.xatolar?.name && (
 <p className="text-red-600 text-sm">{holat.xatolar.name[0]}

```

```

 </p>
 })
 </div>

 <button
 type="submit"
 disabled={yuborilmoqda}
 className="px-4 py-2 bg-blue-600 text-white rounded
disabled:opacity-50"
 >
 {yuborilmoqda ? "Saqlanmoqda..." : "Saqlash"}
 </button>

 {holat?.muvaffaqiyat && <p className="text-green-600">✓ Saqlandi!
</p>}
</form>
);
}

```

Endi: noto'g'ri email yozsangiz, o'sha maydon ostida qizil xato chiqadi. Hammasi to'g'ri bo'lsa — bazaga saqlanadi. Bu — professional, xavfsiz forma!

## 11.7. TypeScript bilan bonus

Zod schemadan TypeScript turini avtomatik olish mumkin (kodni xavfsizroq qiladi):

```

const userSchema = z.object({
 email: z.string().email(),
 name: z.string(),
});

type User = z.infer<typeof userSchema>;
// type User = { email: string; name: string }

```

`z.infer` — schemadan turni avtomatik yasaydi. Endi ikkala joyda (validatsiya + tur) bir manbadan foydalanasiz.

## 11-BOB MASALALARI (20 ta)

**Tushunish (1–6):** 1. Validatsiya nima va nega muhim? "Foydalanuvchiga ishonma" qoidasini izohlang. 2. Nega validatsiya server'da **majburiy** (frontend yetmaydi)? 3. Zod nima? Pasport nazorati analogiyasi bilan tushuntiring. 4. `safeParse` nima qaytaradi (`success`, `data`, `error`)? 5. Validatsiyasiz qanday muammolar chiqadi? 3 ta ayting. 6. Validatsiya frontend va backend'da qanday farq qiladi?

**Zod schema (7–14):** 7. Zod'ni o'rnatish (`npm install zod`). 8. `email` (email format) va `name` (min 2) uchun schema yozing. 9. Schemaga `age` qo'shing (son, min 18). 10. Noto'g'ri ma'lumotni `safeParse` qilib, `natija.success` ni `console.log` qiling. 11. To'g'ri ma'lumot bilan sinab, `natija.data` ni ko'ring. 12. `password` maydoni qo'shing: min 8 belgi. 13. `website` maydoni qo'shing: `url()` va `optional()`. 14. `role` maydoni qo'shing: `enum(["admin", "user"])`.

**Server Action bilan (15–20):** 15. Server Action ichida schema yarating va `safeParse` bilan tekshiring. 16. Xato bo'lsa `fieldErrors` ni qaytaring, to'g'ri bo'lsa bazaga saqlang. 17. `useActionState` bilan formada har maydon ostida xatoni ko'rsating. 18. Noto'g'ri email yozib, qizil xato chiqishini tasdiqlang. 19. Hammasini to'g'ri to'ldirib, bazaga saqlanishini tasdiqlang. 20. `z.infer` bilan schemadan TypeScript tur yasang va ishlating.

**Qo'shimcha challenge:** 10-bobdagi guestbook loyihangizga to'liq validatsiya qo'shing: ism (2–50 belgi), xabar (5–500 belgi), ixtiyoriy email (`optional + email`). Server Action'da Zod bilan tekshiring, formada har maydon xatosini ko'rsating. Endi bo'sh yoki noto'g'ri ma'lumot bazaga tusha olmaydi.

---

## 12-BOB. Autentifikatsiya (Better Auth) va xavfsizlik

Mana yakuniy va eng muhim mavzu: **autentifikatsiya** — ya'ni foydalanuvchilar ro'yxatdan o'tishi, tizimga kirishi va himoyalangan sahifalar. Buni to'g'ri qilish — saytingizning xavfsizlig'i uchun hayotiy muhim.

### 12.1. Autentifikatsiya nima? Asosiy tushunchalar

**Autentifikatsiya (auth)** — "siz kimsiz?" degan savolga javob berish jarayoni (login).

**Avtorizatsiya** — "sizning ruxsatingiz bormi?" (masalan, faqat adminlar kira oladigan sahifa).

**3 ta muhim tushuncha (oddiy tilda):**

**1) Parol (password) va uni hashlash**

**OLTIN QOIDA:** Parolni **hech qachon ochiq saqlamang**. Buning o'rniga uni "**maydalab**" (hash qilib) saqlaymiz.

Hashlash — bu **bir tomonlama maydalagich**. Parolni maydalab bo'ladi, lekin maydalangandan orqaga parolni **tiklab bo'lmaydi**. Login paytida: kiritilgan parolni ham maydalaymiz va ikki maydani solishtiramiz. Shunday qilib, baza o'g'irlansa ham, parollar himoyalangan qoladi.

## 2) Sessiya (session) va cookie

Har safar sahifa ochganda parol so'rashi noqulay. Buning o'rniga: bir marta kirganda foydalanuvchiga **"bilaguzuk" (cookie)** beriladi.

**Cookie — bu konsertdagi bilaguzukka o'xshaydi.** Kiraverishda (login) bilaguzuk taqasiz. Keyin VIP zonaga (himoyalangan sahifa) har borganda qorovul bilaguzungizni tekshiradi — pasport qayta so'ramaydi. Chiqib ketganda (logout) bilaguzuk kesiladi.

## 3) Auth kutubxonasi

Bularning hammasini (hashlash, sessiya, cookie, xavfsizlik) qo'lda yozish juda murakkab va **xatarli** (bitta xato — butun sayt buziladi). Shuning uchun **tayyor, sinalgan kutubxona** ishlatamiz.

## 12.2. Qaysi auth kutubxonasini tanlash? (2026)

2026-yilda asosiy variantlar:

Kutubxona	Kim uchun
<b>Better Auth</b>	Self-hosted, ma'lumot o'zingizda, type-safe. <b>Yangi loyihalar uchun eng yaxshi standart.</b>
<b>Clerk</b>	Eng tez (tayyor UI), lekin pulli (foydalanuvchi soniga qarab) va ma'lumot ularda
<b>Auth.js (NextAuth)</b>	Eski, mashhur, lekin asosan eski loyihalar uchun

Biz **Better Auth**'ni o'rganamiz: bepul, ma'lumotingiz o'zingizda (vendor lock-in yo'q), Next.js 16 va Prisma bilan zo'r ishlaydi. Tezroq boshlamoqchi bo'lsangiz — **Clerk** ham yaxshi variant (3 qadamda tayyor UI).

## 12.3. Better Auth'ni o'rnatish



```
npm install better-auth
```

`.env` fayliga maxfiy kalit qo'shing (uni hech kimga bermang, GitHub'ga yuklamang!):

```
BETTER_AUTH_SECRET=tasodifiy-uzun-maxfiy-satr-bu-yerga
BETTER_AUTH_URL=http://localhost:3000
```

Maxfiy kalitni tasodifiy yarating (terminalda: `openssl rand -base64 32`).

## 12.4. Better Auth'ni sozlash (3 ta fayl)

### 1) Server sozlamasi — `lib/auth.ts`:

```
// lib/auth.ts
import { betterAuth } from "better-auth";
import { prismaAdapter } from "better-auth/adapters/prisma";
import { prisma } from "@/lib/prisma";

export const auth = betterAuth({
 database: prismaAdapter(prisma, { provider: "sqlite" }),
 emailAndPassword: {
 enabled: true, // email + parol bilan kirishni yoqamiz
 },
});
```

### 2) Client sozlamasi — `lib/auth-client.ts`:

```
// lib/auth-client.ts
import { createAuthClient } from "better-auth/react";

export const authClient = createAuthClient();
```

### 3) API yo'naki — `app/api/auth/[...all]/route.ts`:

```
// app/api/auth/[...all]/route.ts
import { auth } from "@/lib/auth";
import { toNextJsHandler } from "better-auth/next-js";

export const { GET, POST } = toNextJsHandler(auth);
```

Bu yo'naki barcha auth so'rovlarini (login, register, logout) avtomatik boshqaradi.

**4) Bazaga auth jadvallarini qo'shish:** Better Auth o'z jadvallarini (`User`, `Session`, `Account`) talab qiladi. Ularni schemaga qo'shish uchun:

```
npx @better-auth/cli generate # schema.prisma'ga modellarni qo'shadi
npx prisma migrate dev --name add-auth # bazaga qo'llaydi
```

## 12.5. Ro'yxatdan o'tish va kirish (formalar)

Better Auth client funksiyalarini ishlatamiz (bular **Client Component**da):

```
// app/register/page.tsx
"use client";

import { useState } from "react";
import { authClient } from "@/lib/auth-client";
import { useRouter } from "next/navigation";

export default function RegisterPage() {
 const router = useRouter();
 const [xato, setXato] = useState("");

 async function royxatdanOtish(formData: FormData) {
 const { error } = await authClient.signUp.email({
 email: formData.get("email") as string,
 password: formData.get("password") as string,
 name: formData.get("name") as string,
 });

 if (error) {
 setXato(error.message || "Xato yuz berdi");
 } else {
 router.push("/dashboard"); // muvaffaqiyatdan keyin
 }
 }

 return (
 <form action={royxatdanOtish} className="max-w-md p-8 space-y-3">
 <input name="name" placeholder="Ism" className="w-full border p-2 rounded" />
 <input name="email" type="email" placeholder="Email"
 className="w-full border p-2 rounded" />
 <input name="password" type="password" placeholder="Parol"
 className="w-full border p-2 rounded" />
 <button type="submit" className="px-4 py-2 bg-blue-600 text-white rounded">
 Ro'yxatdan o'tish
 </button>
 {xato && <p className="text-red-600">{xato}</p>}
 </form>
);
}
```

**Kirish (login)** ham xuddi shunday, faqat `signUp` o'rniga `signIn`:

```
const { error } = await authClient.signIn.email({
 email: formData.get("email") as string,
 password: formData.get("password") as string,
});
```

### Chiqish (logout):

```
await authClient.signOut();
```

Parolni hashlash, cookie yaratish — bularning hammasini Better Auth **avtomatik** qiladi. Siz faqat `signUp` / `signIn` chaqirasiz!

## 12.6. Himoyalangan sahifalar — sessiyani tekshirish

Faqat kirgan foydalanuvchi ko'radigan sahifa (masalan, dashboard). Server Component'da sessiyani tekshiramiz:

```
// app/dashboard/page.tsx - Server Component
import { auth } from "@/lib/auth";
import { headers } from "next/headers";
import { redirect } from "next/navigation";

export default async function DashboardPage() {
 const session = await auth.api.getSession({
 headers: await headers(), // Next.js 16 - headers() async!
 });

 if (!session) {
 redirect("/login"); // kirmagan bo'lsa - login'ga yubor
 }

 return (
 <div className="p-8">
 <h1 className="text-2xl font-bold">Boshqaruv paneli</h1>
 <p>Xush kelibsiz, {session.user.name}!</p>
 </div>
);
}
```

**Tushuntirish:** - `auth.api.getSession(...)` — joriy foydalanuvchi sessiyasini oladi. - `headers: await headers()` — Next.js 16'da `headers()` asinxron (`await` shart). - `if (!session) redirect(...)` — kirmagan bo'lsa, login sahifasiga yuboradi.

## 12.7. ⚠️ ENG MUHIM XAVFSIZLIK DARSİ: middleware'ga ishonma

Bu — minglab dasturchi qiladigan **xatarli xato**. Diqqat bilan o'qing.

2025-yilda jiddiy xavfsizlik teshigi topildi (CVE-2025-29927): **Next.js middleware'dagi himoya chetlab o'tilishi mumkin**. Ya'ni, agar siz faqat middleware (Next.js 16'da `proxy.ts`) ichida "kirmagan bo'lsa, kiritma" deb yozsangiz — xaker uni aldab o'tib ketishi mumkin.

**Bilaguzuk analogiyasi:** Agar qorovul faqat eshik oldida turib, ichkarida hech kim tekshirmasa — kimdir devordan oshib kirsas, hech kim ushlamaydi. Har VIP xonada ham tekshiruv bo'lishi kerak.

**To'g'ri yondashuv:** -  **Har bir himoyalangan sahifa/action ichida** sessiyani tekshiring (`auth.api.getSession` — yuqoridagi kabi). Bu — HAQIQIY himoya. -  `proxy.ts` (middleware)'ni faqat **qulaylik** uchun ishlating (masalan, kirmaganlari tez login'ga yo'naltirish). Lekin u **yagona himoya bo'lmasin**.

**Next.js 16 o'zgarishi:** Next.js 16'da `middleware.ts` fayli `proxy.ts` ga, `middleware` funksiyasi esa `proxy` ga o'zgardi. Eski darsliklarda `middleware` ko'rsangiz — endi u `proxy`.

`proxy.ts` (ixtiyoriy, tez yo'naltirish uchun):

```
// proxy.ts
import { NextRequest, NextResponse } from "next/server";
import { getSessionCookie } from "better-auth/cookies";

export async function proxy(request: NextRequest) {
 const cookie = getSessionCookie(request);
 // BU XAVFSIZ EMAS — faqat tez yo'naltirish uchun!
 if (!cookie) {
 return NextResponse.redirect(new URL("/login", request.url));
 }
 return NextResponse.next();
}

export const config = {
 matcher: ["/dashboard/:path*"], // qaysi yo'llarga qo'llanadi
};
```

**Yana bir bor:** `proxy.ts` — qulaylik. Haqiqiy tekshiruv — har sahifa/action ichida.

## 12.8. Server Action'da foydalanuvchini tekshirish

Ma'lumotni o'zgartiradigan har bir Server Action ichida ham foydalanuvchini tekshiring:

```
// app/actions.ts
"use server";
```

```
import { auth } from "@/lib/auth";
import { headers } from "next/headers";
import { prisma } from "@/lib/prisma";

export async function maqolaQoshish(formData: FormData) {
 // 1. Avval foydalanuvchini tekshiramiz
 const session = await auth.api.getSession({ headers: await headers() });
 if (!session) {
 return { xato: "Avval tizimga kiring" };
 }

 // 2. Endi xavfsiz – uning maqolasini saqlaymiz
 await prisma.post.create({
 data: {
 title: formData.get("title") as string,
 authorId: session.user.id, // kim yaratgani
 },
 });

 return { muvaffaqiyat: true };
}
```

## 12.9. Xavfsizlik bo'yicha asosiy qoidalar (xulosa)

Yodlab oling — bular professional dasturchi bo'lishning asosi:

1. **Parolni hech qachon ochiq saqlamang** — auth kutubxonasi avtomatik hashlaydi.
2. **Maxfiy kalitlarni .env da saqlang** va ularni GitHub'ga **yuklamang** ( `.gitignore` da `.env` borligini tekshiring).
3. **Hech qachon foydalanuvchiga ishonmang** — har doim server'da validatsiya (Zod) qiling.
4. **Har himoyalangan sahifa/action ichida** sessiyani tekshiring — middleware'ga **yagona himoya** sifatida ishonmang.
5. **HTTPS ishlating** (production'da) — ma'lumot shifrlangan holatda yuborilsin.
6. **Auth'ni o'zingiz yozmang** — sinalgan kutubxona ishlating.



## 12-BOB MASALALARI (22 ta)

**Tushunish (1–8):** 1. Autentifikatsiya va avtorizatsiya farqi nima? 2. Parolni nega ochiq saqlash mumkin emas? Hashlash nima (bir tomonlama maydalagich)? 3. Cookie va sessiya nima? Konsert bilaguzugi analogiyasi bilan tushuntiring. 4. Nega auth'ni o'zimiz

yozmay, kutubxona ishlatamiz? 5. Better Auth, Clerk, Auth.js — har biri kim uchun? 6. CVE-2025-29927 nima haqida edi? Nega middleware'ga ishonish xavfli? 7. Next.js 16'da `middleware.ts` nimaga o'zgardi? 8. Himoyalangan sahifani qaerda tekshirish kerak — faqat middleware'dami yoki har sahifadami? Nega?

**O'rnatish va sozlash (9–14):** 9. Better Auth'ni o'rnatish (`npm install better-auth`). 10. `.env` ga `BETTER_AUTH_SECRET` va `BETTER_AUTH_URL` qo'shing. 11. `lib/auth.ts` yarating (Prisma adapter bilan). 12. `lib/auth-client.ts` yarating. 13. `app/api/auth/[...all]/route.ts` yarating. 14. Auth modellarini generatsiya qilib, migration qiling.

**Forma va sessiya (15–20):** 15. Ro'yxatdan o'tish (register) formasi yarating (`authClient.signUp.email`). 16. Kirish (login) formasi yarating (`authClient.signIn.email`). 17. Xato bo'lganda (masalan, mavjud email) xato xabarini ko'rsating. 18. Himoyalangan `/dashboard` sahifasi yarating (`getSession + redirect`). 19. Dashboard'da foydalanuvchi ismini ko'rsating (`session.user.name`). 20. "Chiqish" (logout) tugmasini qo'shing (`authClient.signOut`).

**Xavfsizlik (21–22):** 21. Server Action yarating — ichida `getSession` bilan tekshirsin, kirmagan bo'lsa rad etsin. 22. `.gitignore` da `.env` borlig'ini tekshiring (maxfiy kalitlar himoyada).

**Qo'shimcha challenge:** To'liq **himoyalangan blog** yasang: ro'yxatdan o'tish + kirish + chiqish. Faqat kirgan foydalanuvchi maqola qo'sha olsun (Server Action'da `getSession` tekshiruvi). Har maqola muallifiga bog'lansin (`authorId`). Bosh sahifa — hammaga ochiq (maqolalar ro'yxati), `/new` — faqat kirganlarga. Zod validatsiyasi ham bo'lsin. Bu — haqiqiy, xavfsiz, full-stack ilova!

---

## 3-QISM YAKUNI

Ajoyib! Endi siz **haqiqiy full-stack dasturchisiz**. Mana nimani o'rgandingiz:

✅ **Route Handlers** bilan tashqi dunyoga ochiq API yaratasiz    ✅ **Prisma + ma'lumotlar bazasi** bilan ma'lumotni doimiy saqlaysiz (CRUD)    ✅ **Zod** bilan kelgan ma'lumotni tekshirib, xavfsiz qabul qilasiz    ✅ **Better Auth** bilan to'liq login/register tizimi va himoyalangan sahifalar qurasiz    ✅ **Xavfsizlik** asoslarini bilasiz — hashlash, sessiya, "middleware'ga ishonma" darsi

Endi sizda haqiqiy, foydalanuvchili, ma'lumot saqlaydigan, xavfsiz veb-ilova qurish uchun **hamma asosiy ko'nikma** bor. Bu — ko'pchilik dasturchi yetmaydigan darajada katta yutuq!

## Keyingi qadam: 4-QISM (yakuniy) nimani o'z ichiga oladi?

- **Tezlik (Performance)** — saytni "uchadigan" qilish, optimizatsiya texnikalari
- **Test yozish** — kodningizni avtomatik tekshirish (xatolarni oldindan tutish)
- **Deployment** — saytni internetga chiqarish (Vercel, VPS), domen ulash
- **Monitoring va ilg'or patternlar** — real loyihani boshqarish, professional amaliyotlar
- **Yakuniy loyiha** — hamma bilimni birlashtirgan to'liq projekt

**Maslahat:** 4-qismga o'tishdan oldin, bu qismdagi "himoyalangan blog" challenge'ini albatta bajaring. Auth + DB + validatsiya + Server Actions — hammasini birga ishlatib ko'rsangiz, full-stack mantig'i miyangizda mustahkamlanadi.

---

*Bu — "Next.js 0 dan Expert darajagacha" qo'llanmasining 3-qismi.*

---

 [README](#) · [← 2-qism — Server & Actions](#) · **Keyingi: 4-qism — Production** →

# Next.js — 0 dan Expert darajagacha (4-QISM, YAKUNIY)

 [README](#) · [← 3-qism — Full-stack](#) · [Keyingi: BONUS boblar →](#)

**Eslatma:** Bu — qo'llanmaning **yakuniy** qismi. 1–3-qismlar (JS, React, routing, Server/Client Components, data fetching, Server Actions, caching, API, ma'lumotlar bazasi, validatsiya, auth)ni o'rgangan bo'lishingiz kerak.

**Versiya:** Next.js **16**, React 19. Node.js **20.9+**.

## Bu qismda nimalarni o'rganamiz

Endi sizda ishlaydigan full-stack ilova bor. Bu qism — uni **professional, tez va internetga chiqarilgan** holatga keltirish haqida. Bu — "dasturchi" va "**expert** dasturchi" o'rtasidagi farq.

- **13-bob. Performance (Tezlik)** — saytni "uchadigan" qilish texnikalari
- **14-bob. Test yozish** — kodni avtomatik tekshirish, xatolarni oldindan tutish (Vitest + Playwright)
- **15-bob. Deployment** — saytni internetga chiqarish (Vercel va VPS)
- **16-bob. Monitoring, ilg'or patternlar va yakuniy loyiha**

**Maslahat:** Bu qismdagi bilimlar real loyihalarda, vaqt o'tishi bilan chuqurlashadi. Hozir asoslarni tushunsangiz va bir marta sinab ko'rsangiz — yetadi. Expert bo'lish — bu uzluksiz amaliyot.

---

## 13-BOB. Performance (Tezlik)

Tez sayt — yaxshi sayt. Foydalanuvchilar 2–3 soniya kutsa, ketib qoladi; Google ham tez saytlarni yuqoriroq ko'rsatadi. Yaxshi xabar: **Next.js ko'p optimizatsiyani sizning o'rningizga qiladi**. Bu bobda qolganini ham o'rganamiz.

## 13.1. Tezlik nima va nega muhim?

**Misol:** Tez restoran — taomning bir qismini oldindan tayyorlab qo'yadi (kesh), tayyor bo'lgan taomni darrov chiqaradi, hammasini kutib o'tirmaysiz (streaming), va keraksiz yukni tashimaydi. Sekin restoran — hamma narsani noldan, sizning oldingizda qiladi va siz 40 daqiqa kutasiz.

Tezlik ko'rsatkichlari (**Core Web Vitals** — Google o'lchaydigan): - **LCP** — eng katta element qancha tez ko'rinadi (sahifa "tayyor" bo'lish hissi). - **CLS** — sahifa "sakrab" turmasligi (matn/rasm joyini o'zgartirmasligi). - **INP** — tugmaga bosganda qancha tez javob beradi.

## 13.2. Next.js sizning o'rningizga qiladigan optimizatsiyalar

Bularni **bepulga** olasiz (alohida hech narsa qilmaysiz): - ☒ **Server Components** — komponent kodi brauzerga yuborilmaydi → kamroq JavaScript → tezroq (5-bobni eslang). - ☒ **Avtomatik kod bo'lish (code splitting)** — har sahifa faqat o'ziga kerak kodni yuklaydi. - ☒ **Turbopack** — juda tez build (Next.js 16'da standart).

Demak, agar siz "ko'pchilik komponentni Server qoldirib, faqat interaktivni Client qilish" qoidasiga amal qilsangiz (5-bob) — allaqachon tez saytdasiz.

## 13.3. Rasmlarni optimallashtirish — `next/image`

Rasmlar — sahifani sekinlashtiruvchi eng katta sabab. 1-qismda ko'rgan `<Image>` komponenti buni hal qiladi:

```
import Image from "next/image";

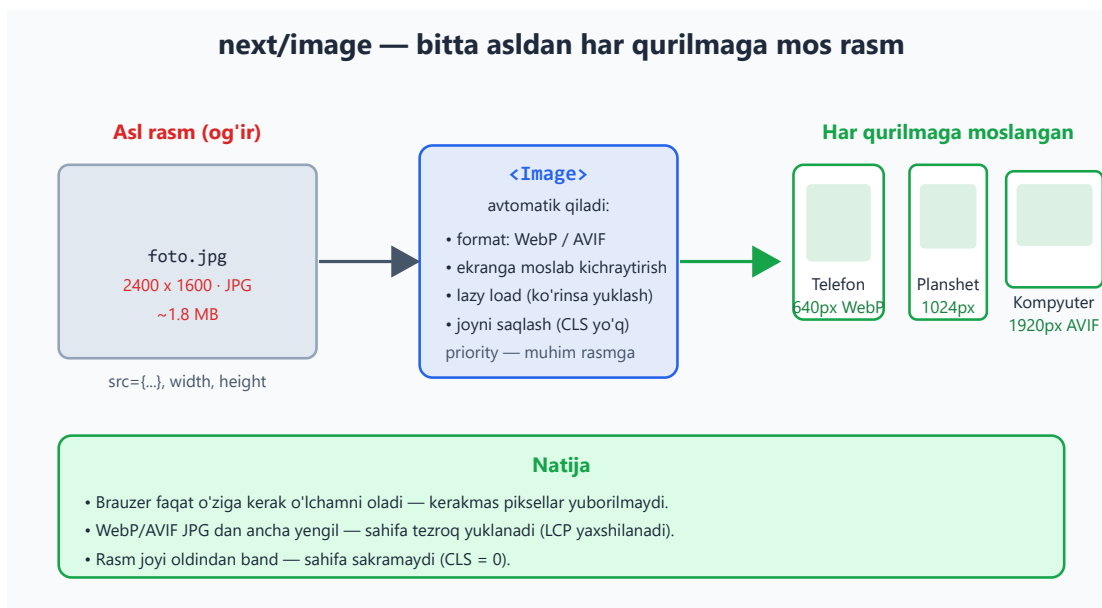
export default function Rasm() {
 return (
 <Image
 src="/katta-rasm.jpg"
 alt="Tavsif"
 width={800}
 height={600}
 priority // ← faqat ekranda darrov ko'rinadigan muhim rasm uchun
 />
);
}
```

`<Image>` avtomatik: - Rasmni kichikroq formatga (WebP/AVIF) o'giradi. - Ekran o'lchamiga moslab kichraytiradi (telefonga kichik, kompyuterga katta). - Faqat

ko'rinadigan rasmlarni yuklaydi ("lazy loading"). - Rasm yuklanguncha joyini saqlab turadi (CLS yo'q — sakramaydi).

Oddiy `<img>` o'rniga **doim** `<Image>` ishlatish. Bu — bepul tezlik.

Quyidagi diagramma bitta asl rasmdan har bir qurilmaga mos o'lcham va format qanday tayyorlanishini ko'rsatadi:



## 13.4. Shriftlarni optimallashtirish — `next/font`

Shriftlar ham sekinlashtiradi. `next/font` ularni avtomatik optimallashtiradi:

```
// app/layout.tsx
import { Inter } from "next/font/google";

const inter = Inter({ subsets: ["latin"] });

export default function RootLayout({ children }: { children:
React.ReactNode }) {
 return (
 <html lang="uz" className={inter.className}>
 <body>{children}</body>
 </html>
);
}
```

Bu shriftni saytingiz bilan birga "joylashtiradi" (tashqi serverdan kutmaydi) va matn "sakramasligini" ta'minlaydi.

## 13.5. Streaming — kontentni bosqichma-bosqich ko'rsatish

2-qism (6-bob)dagi `loading.tsx` ni eslang. U aslida **streaming** texnikasi: sekin yuklanadigan qism kutilayotganda, sahifaning qolgan qismini darrov ko'rsatadi.

Restoran: salat tayyor bo'lishini kutib, ovqatni stol ustida ushlab turmaydi. Salatni darrov chiqaradi, asosiy taom keyin keladi. Foydalanuvchi bo'sh o'tirmaydi.

Aniqroq nazorat uchun `<Suspense>` ishlatamiz — sahifaning **bir qismini** kutib, qolganini darrov ko'rsatish:

```
import { Suspense } from "react";

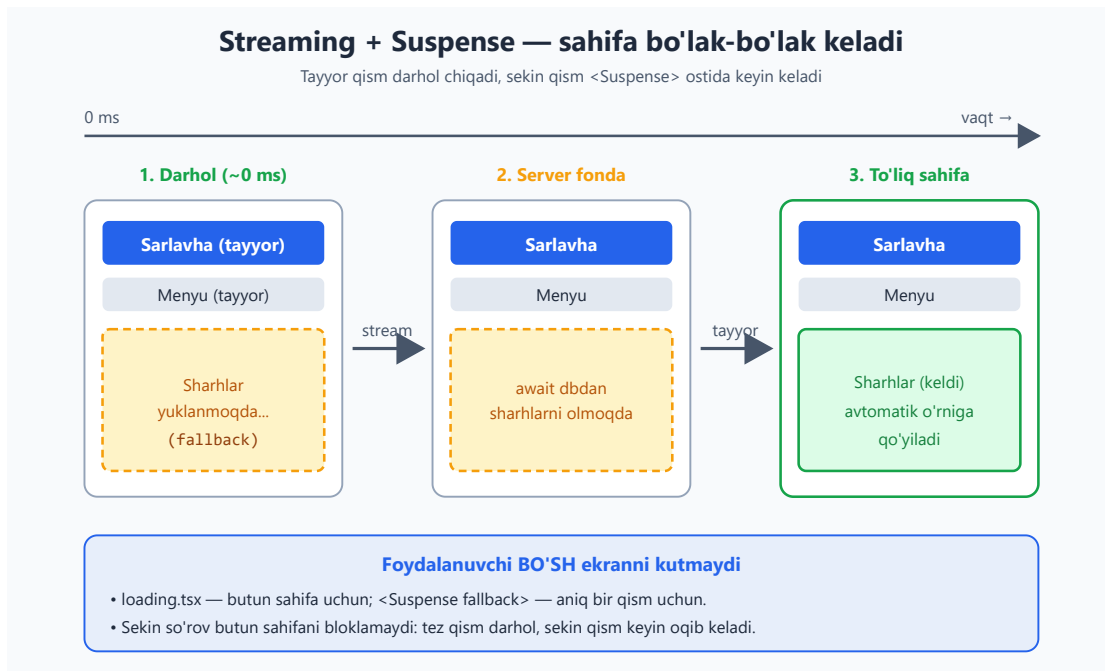
export default function Page() {
 return (
 <div>
 <h1>Bosh sahifa</h1>
 {/* Bu darrov ko'rinadi */}

 <Suspense fallback={<p>Sharhlar yuklanmoqda...</p>}>
 <SeKinSharhlar /> {/* Bu sekin, alohida kutiladi */}
 </Suspense>
 </div>
);
}

async function SeKinSharhlar() {
 const sharhlar = await sekinMalumotOl(); // vaqt oladi
 return <div>{/* sharhlarni ko'rsatish */}</div>;
}
```

Natija: sarlavha darrov ko'rinadi, sharhlar "Yuklanmoqda..." bilan kelguncha foydalanuvchi kutib o'tirmaydi.

Quyidagi diagramma sahifaning bo'lak-bo'lak qanday oqib kelishini bosqichma-bosqich ko'rsatadi:



## 13.6. Static shell + dinamik teshik (Partial Prerendering)

Next.js 16'ning kuchli imkoniyati: sahifaning **statik qismini** oldindan tayyorlab (juda tez), faqat **dinamik qismini** (masalan, foydalanuvchi savati) jonli yuklash. Bu 2-qism (8-bob)dagi `"use cache"` modeli bilan ishlaydi.

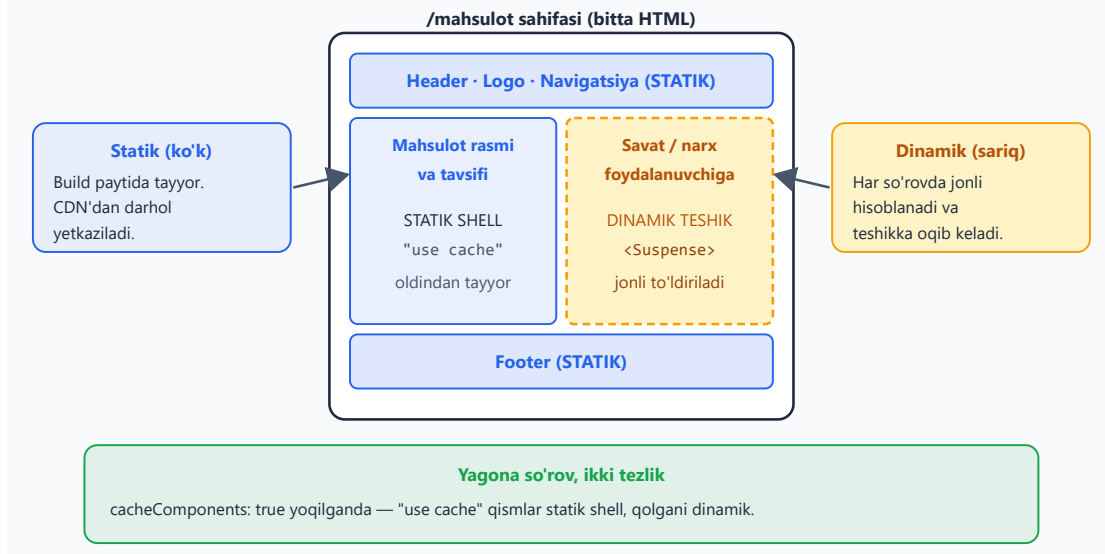
Restoran: menyu, devor bezagi — oldindan tayyor (statik). Faqat sizning buyurtmangiz — jonli tayyorlanadi (dinamik). Hamma narsani noldan qilmaydi.

Buni `cacheComponents: true` (8-bob) yoqilganda, `"use cache"` bilan keshlangan qismlar statik bo'lib, qolgani dinamik qoladi. Amalda: keshlanishi mumkin bo'lgan qismlarni keshlang, dinamik qismni `<Suspense>` ga o'rang.

Quyidagi diagramma bitta sahifada statik shell va dinamik teshik qanday birga yashashini ko'rsatadi:

## Partial Prerendering — statik shell + dinamik teshik

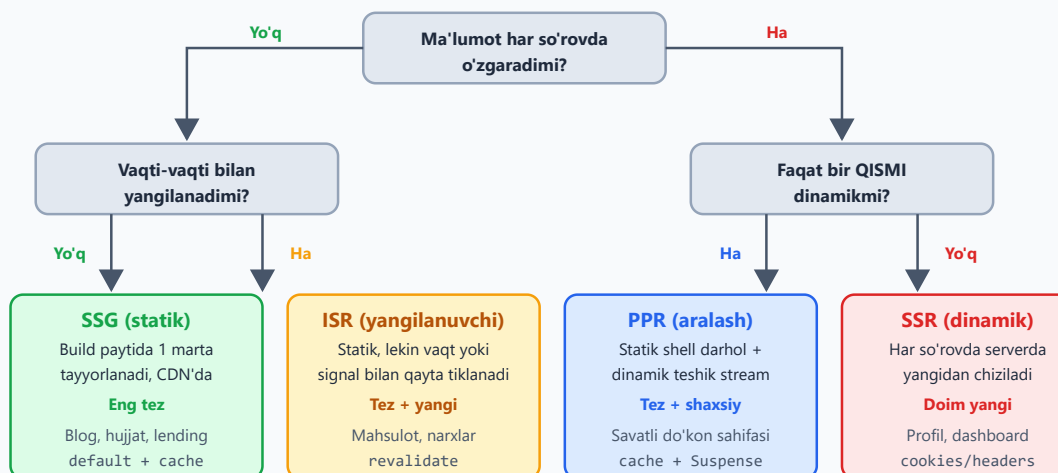
Bir sahifa: statik qism oldindan (juda tez), dinamik qism <Suspense> ostida jonli



Bu yerda ko'rgan to'rt yondashuv (statik, vaqtli yangilanish, aralash va to'liq dinamik) qachon qaysi biri kerakligini quyidagi yo'l-xarita umumlashtiradi:

## Rendering strategiyalari — qachon qaysi?

App Router'da bu sahifa kodidan avtomatik tanlanadi (cache va dinamik API'larga qarab)



### App Router'da nima farqni hal qiladi:

- cookies(), headers(), searchParams ishlatilgani — sahifa DINAMIK (SSR) bo'ladi.
- "use cache" + revalidate — statik (SSG) yoki vaqtli yangilanuvchi (ISR) qiladi.
- Statik shellni dinamik teshik bilan birlashtirsangiz (Suspense) — bu PPR.

## 13.7. Og'ir Client komponentlarni keyin yuklash — next/dynamic

Ba'zi komponentlar og'ir (masalan, katta grafik, xarita). Ularni faqat **kerak bo'lganda** yuklash mumkin:

```
import dynamic from "next/dynamic";

// Bu komponent faqat kerak bo'lganda yuklanadi (boshida emas)
const OgirGrafik = dynamic(() => import("./OgirGrafik"), {
 loading: () => <p>Grafik yuklanmoqda...</p>,
});

export default function Page() {
 return (
 <div>
 <h1>Hisobot</h1>
 <OgirGrafik />
 </div>
);
}
```

Bu — "lazy loading": og'ir kod boshlang'ich yuklanishni sekinlashtirmaydi.

## 13.8. Bundle Analyzer — nima og'irligini topish

Sayingiz sekin bo'lsa, sababini topish kerak. `@next/bundle-analyzer` qaysi kutubxona ko'p joy egallayotganini ko'rsatadi:

```
npm install @next/bundle-analyzer
```

U "xarita" chizadi — qaysi paket katta ekanini ko'rasiz va keraksizlarini olib tashlaysiz.

**Maslahat:** Tezlik muammosini **o'lchamasdan** hal qilmang. Avval Chrome'dagi **Lighthouse** (F12 → Lighthouse) yoki bundle analyzer bilan **sababni toping**, keyin tuzating.

## 13.9. Tezlik bo'yicha amaliy qoidalar

1. Ko'pchilik komponentni **Server** qoldiring, `"use client"` ni faqat kichik interaktiv qismlarga bering.
2. Rasmlar uchun **doim** `<Image>`, shriftlar uchun `next/font`.
3. Sekin qismlarni `<Suspense>` yoki `loading.tsx` bilan o'rang.
4. Tez-tez o'zgarmaydigan ma'lumotni **keshlang** (`"use cache"`, 8-bob).
5. Og'ir client komponentlarni `next/dynamic` bilan keyin yuklang.
6. Muammoni **avval o'lchang** (Lighthouse), keyin tuzating.

## 13-BOB MASALALARI (20 ta)

**Tushunish (1–7):** 1. Nega tezlik muhim? 2 ta sabab ayting. 2. Core Web Vitals nima: LCP, CLS, INP — har biri nimani o'lchaydi? 3. Next.js sizning o'rningizga qiladigan 3 ta optimizatsiyani ayting. 4. Nega Server Component avtomatik tezroq (brauzerga nima yuborilmaydi)? 5. Streaming nima? Restoran misolida tushuntiring. 6. Partial Prerendering (statik shell + dinamik teshik) nima? 7. `next/dynamic` (lazy loading) qachon foydali?

**Amaliy (8–16):** 8. Loyihangizdagi oddiy `<img>` ni `<Image>` ga o'zgartiring. 9. Muhim rasmga `priority` qo'shing, sababini izohlang. 10. `next/font` bilan loyihaga shrift qo'shing (`layout.tsx`). 11. Sekin yuklanadigan komponent yarating va uni `<Suspense fallback={...}>` ga o'rang. 12. Bir sahifada tez qism (sarlavha) darrov, sekin qism `Suspense` bilan kelishini ko'rsating. 13. Og'ir komponentni `next/dynamic` bilan yuklang (`loading` bilan). 14. Chrome'da Lighthouse'ni ishga tushiring (F12 → Lighthouse) va sahifangizni baholang. 15. Lighthouse natijasidan bitta muammoni toping va tuzatishga harakat qiling. 16. `@next/bundle-analyzer` ni o'rnatib, qaysi paket katta ekanini ko'ring.

**Strategiya (17–20):** 17. Bir sahifani oling: qaysi qismlar Server, qaysilari Client bo'lishi kerak — qayta ko'rib chiqing. 18. Qaysi ma'lumotni keshlash mumkin, qaysini yo'q — bitta loyihangiz uchun rejalashtiring. 19. "Avval o'lcha, keyin tuzat" qoidasini o'z so'zingiz bilan tushuntiring. 20. Tezlik bo'yicha 6 ta qoidani o'z loyihangizga qanday qo'llashingizni yozing.

**Qo'shimcha challenge:** 3-qismdagi blog loyihangizni tezlashtiring: rasmlarni `<Image>` ga o'tkazing, shriftni `next/font` qiling, maqolalar ro'yxatini "use cache" bilan keshlang, sekin qismlarni `<Suspense>` ga o'rang. Lighthouse'da oldin va keyin ballarni solishtiring.

---

## 14-BOB. Test yozish

Kod yozdingiz, qo'lda sinab ko'rdingiz, ishladi. Lekin keyin boshqa joyni o'zgartirsangiz, eskisi buzilmadimi? Har safar hammasini qo'lda tekshirib bo'lmaydi. **Test** — kodni **avtomatik** tekshiradigan kod.

### 14.1. Test nima va nega kerak?

**Misol:** Test — bu avtomobilning xavfsizlik tekshiruviga yoki oshpazning taomni mijozga berishdan oldin tatib ko'rishiga o'xshaydi. Muammoni **mijoz sezishdan oldin** topasiz.

Testsiz: har o'zgarishdan keyin qo'lda hamma narsani bosib chiqasiz (zerikarli, ishonchsiz). Test bilan: bir buyruq be" – kod o'zi tekshiradi, xato bo'lsa darrov aytadi.

**Foydalari:** - 🐛 Xatolarni erta tutadi (foydalanuvchi sezmasdan). - 😊 O'zgarish kiritishda qo'rqmaysiz (test buzilsa, ko'rasiz). - 📖 Test kodning qanday ishlashini hujjatlaydi.

## 14.2. Test turlari va vositalar (2026)

Tur	Nima tekshiradi	Vosita
Unit test	Bitta funksiya/komponent alohida	Vitest + React Testing Library
E2E test	Butun jarayon (foydalanuvchi kabi)	Playwright

**Misol:** Unit test — bitta ingredientni tatib ko'rish (tuz yetarli mi?). E2E test — butun taomni mijoz kabi yeb ko'rish (boshidan oxirigacha).

**2026 holatiga ko'ra:** - **Vitest** — unit testlar uchun standart (eski Jest o'rniga; tezroq, API'si bir xil). - **Playwright** — E2E uchun standart (haqiqiy brauzerda sinaydi).

⚠️ **Muhim cheklov:** Vitest **async** **Server Component**larni sinashni qo'llab-quvvatlamaydi (React'da bu hali barqaror emas). Shuning uchun: **sinxron komponentlar**, **Server Actions (oddiy funksiya sifatida)**, **Zod schemalar** — Vitest bilan; **async** **Server Componentlar va auth jarayonlari** — Playwright (E2E) bilan.

## 14.3. Vitest'ni o'rnatish

```
npm install -D vitest @vitejs/plugin-react jsdom @testing-library/react @testing-library/dom vite-tsconfig-paths
```

Loyiha ildizida `vitest.config.mts` yarating:

```
// vitest.config.mts
import { defineConfig } from "vitest/config";
import react from "@vitejs/plugin-react";
import tsconfigPaths from "vite-tsconfig-paths";

export default defineConfig({
```

```
plugins: [tsconfigPaths(), react()],
test: {
 environment: "jsdom", // brauzer muhitini taqlid qiladi
},
});
```

package.json ga test buyrug'ini qo'shing:

```
{
 "scripts": {
 "test": "vitest"
 }
}
```

**Tez yo'l:** `npx create-next-app --example with-vitest with-vitest-app` — tayyor sozlangan loyiha beradi.

## 14.4. Birinchi unit test — oddiy funksiya

Avval testlash oson bo'lgan oddiy funksiya:

```
// lib/math.ts
export function qoshish(a: number, b: number) {
 return a + b;
}
```

```
// lib/math.test.ts
import { describe, it, expect } from "vitest";
import { qoshish } from "../math";

describe("qoshish funksiyasi", () => {
 it("ikki sonni to'g'ri qo'shadi", () => {
 expect(qoshish(2, 3)).toBe(5);
 });

 it("manfiy sonlar bilan ishlaydi", () => {
 expect(qoshish(-1, 1)).toBe(0);
 });
});
```

`npm test` bilan ishga tushiring. Yashil ✓ — test o'tdi.

**Tushuntirish:** - `describe(...)` — testlarni guruhlaydi. - `it(...)` — bitta test ("u shunday qilishi kerak"). - `expect(natija).toBe(kutilgan)` — "natija kutilganga teng bo'lsin".

## 14.5. Zod schemani sinash

3-qism (11-bob)dagi Zod schemalar — sinash uchun ideal (oddiy funksiya):

```
// schema.test.ts
import { describe, it, expect } from "vitest";
import { z } from "zod";

const userSchema = z.object({
 email: z.string().email(),
 name: z.string().min(2),
});

describe("userSchema", () => {
 it("to'g'ri ma'lumotni qabul qiladi", () => {
 const natija = userSchema.safeParse({ email: "a@b.com", name: "Ali" });
 expect(natija.success).toBe(true);
 });

 it("noto'g'ri emailni rad etadi", () => {
 const natija = userSchema.safeParse({ email: "xato", name: "Ali" });
 expect(natija.success).toBe(false);
 });
});
```

## 14.6. Komponentni sinash (React Testing Library)

Sinxron (oddiy, `async` bo'lmagan) komponentlarni sinash:

```
// Salom.tsx
export default function Salom({ ism }: { ism: string }) {
 return <h1>Salom, {ism}!</h1>;
}
```

```
// Salom.test.tsx
import { describe, it, expect } from "vitest";
import { render, screen } from "@testing-library/react";
import Salom from "./Salom";

describe("Salom komponenti", () => {
 it("ismni ko'rsatadi", () => {
 render(<Salom ism="Oqil" />);
 expect(screen.getByText("Salom, Oqil!")).toBeDefined();
 });
});
```

`render(...)` — komponentni "chizadi", `screen.getByText(...)` — matnni topadi.

## 14.7. E2E test — Playwright

`async` Server Componentlar, auth jarayonlari, butun foydalanuvchi yo'li uchun Playwright:

```
npm init playwright@latest
```

(Yoki tayyor: `npx create-next-app --example with-playwright with-playwright-app`)

Oddiy E2E test:

```
// e2e/home.spec.ts
import { test, expect } from "@playwright/test";

test("bosh sahifa sarlavhasini ko'rsatadi", async ({ page }) => {
 await page.goto("http://localhost:3000");
 await expect(page.getByRole("heading", { level: 1 })).toBeVisible();
});

test("about sahifasiga o'tadi", async ({ page }) => {
 await page.goto("http://localhost:3000");
 await page.getByRole("link", { name: "Biz haqimizda" }).click();
 await expect(page).toHaveURL(/about/);
});
```

Playwright **haqiqiy brauzer** ochib, foydalanuvchi kabi bosadi, yozadi, tekshiradi. Auth, formalar, navigatsiya — hammasini real holatda sinaydi.

## 14.8. Nimani sinash kerak (va nimani yo'q)

- ✔ **Sinang:** muhim biznes mantiq, Zod schemalar, Server Actions, kritik foydalanuvchi yo'llari (login, to'lov, ro'yxatdan o'tish).
- ✗ **Sinamang:** tashqi kutubxona allaqachon sinagan narsalar, "tugma tugma ekanini" tekshiruvchi ma'nosiz testlar.

**Sifat > miqdor.** 30 ta muhim test — 200 ta ma'nosiz testdan yaxshiroq. Eng muhim yo'llarga (login, asosiy amallar) e'tibor bering.



14-BOB MASALALARI (20 ta)

**Tushunish (1–6):** 1. Test nima va nega kerak? Avtomobil tekshiruvi/taom tatib ko'rish misolida ayting. 2. Unit test va E2E test farqi nima? 3. 2026'da nega Jest emas, Vitest ishlatiladi? 4. Vitest nega `async` Server Componentlarni sinay olmaydi? Nima qilish kerak? 5. Qaysi narsalarni Vitest bilan, qaysilarini Playwright bilan sinaymiz? 6. "Sifat > miqdor" — testda buni izohlang.

**Vitest (7–14):** 7. Vitest'ni o'rnatish va `vitest.config.mts` yaratish. 8. `package.json` ga `"test": "vitest"` qo'shish. 9. `qoshish(a, b)` funksiyasi uchun unit test yozish. 10. Unga manfiy sonlar va nol uchun testlar qo'shish. 11. `npm test` bilan testlarni ishga tushirish, yashil ✓ ni ko'ring. 12. Bir Zod schema uchun "to'g'ri qabul qiladi" testi yozish. 13. Shu schema uchun " noto'g'rini rad etadi" testi yozish. 14. Sinxron komponent ( `Salom` ) uchun RTL bilan test yozish.

**Playwright (15–20):** 15. Playwright'ni o'rnatish ( `npm init playwright@latest` yoki `with-playwright` namunasi). 16. Bosh sahifa sarlavhasini tekshiradigan E2E test yozish. 17. Bir sahifadan boshqasiga o'tishni tekshiradigan test yozish. 18. Playwright testini ishga tushirish ( `npx playwright test` ). 19. Login formasini tekshiradigan E2E test yozish (email/parol kiritish, kirish). 20. Sinash kerak bo'lgan 5 ta kritik yo'lni loyihangiz uchun ro'yxatlash.

**Qo'shimcha challenge:** 3-qismdagi blog loyihangiz uchun: Zod schemalariga unit testlar (Vitest) + "foydalanuvchi ro'yxatdan o'tadi, maqola qo'shadi, ko'radi" jarayoniga E2E test (Playwright) yozish.

---

## 15-BOB. Deployment — saytni internetga chiqarish

Mana eng hayajonli qadam! Hozirgacha saytingiz faqat **sizning kompyuteringizda** ( `localhost:3000` ) ishlardi. **Deployment** — uni internetga chiqarish, ya'ni butun dunyo ko'ra oladigan qilish.

### 15.1. Deployment nima?

**Misol:** Hozirgacha sizning restoraningiz uy oshxonasi edi — faqat siz ovqatlanardingiz. Deployment — ko'chada haqiqiy restoran ochish, har kim kelib ovqatlanishi mumkin bo'lgan joy. **Domen** (masalan, `mening-saytim.uz` ) — restoraning manzili va peshtaxtasi.

Next.js saytini chiqarishning 2 ta asosiy yo'li bor: 1. **Vercel** — eng oson (Next.js'ni yaratgan kompaniya). Boshlovchi uchun ideal. 2. **VPS** (o'z serveringiz) — to'liq nazorat, arzonroq (katta loyihalar uchun).

## 15.2. 1-yo'l: Vercel (eng oson)

Vercel — Next.js'ni yaratgan kompaniya, shuning uchun ularning platformasi Next.js uchun mukammal sozlangan. Kichik/o'rta loyihalar uchun bepul.

**Qadamlar:** 1. Kodingizni **GitHub**ga yuklang (agar bilmasangiz — `git` va GitHub'ni alohida o'rganing, bu zarur ko'nikma). 2. [vercel.com](https://vercel.com)ga kiring (GitHub bilan). 3. "New Project" → GitHub reponi tanlang. 4. Maxfiy sozlamalarni ( `.env` dagi `DATABASE_URL`, `BETTER_AUTH_SECRET` va h.k.) Vercel sozlamalariga qo'shing (**Environment Variables**). 5. "Deploy" bosing.

Bir necha daqiqada saytingiz `sizning-loyihangiz.vercel.app` manzilida internetda! Har safar GitHub'ga yangi kod yuklasangiz, Vercel avtomatik yangilaydi.

**Domen ulash:** Vercel'da "Domains" bo'limidan o'z domeningizni ( `mening-saytim.uz` ) ulashingiz mumkin.

## 15.3. 2-yo'l: VPS (o'z serveringiz)

VPS — bu internetdagi ijaraga olingan kompyuter (Linux). To'liq nazorat, arzon (oyiga \$5–10), vendor lock-in yo'q. Next.js 16 self-hosting'ni ancha osonlashtirgan.

Bu yo'l ko'proq qadamli va Linux/terminal bilimini talab qiladi. Boshida Vercel bilan boshlang, keyin tajriba bilan VPS'ga o'ting.

**Asosiy g'oya — `standalone` rejim:** Next.js 16 loyihangizni **mustaqil, yengil paketga** jamlaydi (faqat kerakli fayllar, ulkan `node_modules` siz).

`next.config.ts` ga qo'shing:

```
// next.config.ts
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
 output: "standalone", // ← mustaqil paket yaratadi
};

export default nextConfig;
```

**VPS'da deployment qadamlari (qisqacha):**

```
1. VPS'da Node.js 22 va PM2 o'rnatish
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs nginx
sudo npm install -g pm2

2. Loyihani build qilish (lokalda yoki serverda)
npm run build

3. standalone papkaga static va public fayllarni nusxalash
(bular avtomatik kiritilmaydi – qo'lda nusxalanadi)
cp -r .next/static .next/standalone/.next/
cp -r public .next/standalone/

4. PM2 bilan ishga tushirish (24/7 ishlab turadi)
pm2 start .next/standalone/server.js --name myapp
pm2 save # ro'yxatni saqlaydi
pm2 startup # server qayta yuklansa ham avtomatik ishga tushadi
```

**Nginx – reverse proxy** (saytni 80/443 portga ulash):

```
/etc/nginx/sites-available/myapp
server {
 listen 80;
 server_name mening-saytim.uz;

 location / {
 proxy_pass http://localhost:3000;
 proxy_http_version 1.1;
 proxy_set_header Host $host;
 proxy_set_header X-Real-IP $remote_addr;
 }
}
```

**SSL (HTTPS) – Let's Encrypt (Certbot) bilan bepul:**

```
sudo apt install certbot python3-certbot-nginx
sudo certbot --nginx -d mening-saytim.uz
```

**Maxfiy sozlamalar (env):** `DATABASE_URL`, kalitlar kabi server sozlamalarini VPS'da (PM2 config yoki `.env` da) o'rnatish – kodga "qotirib" yozmang. `NEXT_PUBLIC_` bilan boshlanadigan o'zgaruvchilar build paytida kiritiladi (o'zgartirsangiz – qayta build kerak).

## 15.4. Vercel vs VPS – qaysi birini tanlash?

	Vercel	VPS
Qiyinlik	★ Juda oson	★★★ Murakkabroq
Narx	Boshida bepul, o'sganda qimmat	\$5–10/oy, barqaror
Nazorat	Cheklangan	To'liq
Boshlovchi uchun	✓ Ideal	Tajriba kerak

**Maslahat:** Birinchi loyihangizni **Vercel** da chiqaring (5 daqiqa). Keyinroq, server boshqarishni o'rganib, **VPS**'ni sinang. Ikkalasi ham foydali ko'nikma.

## 15.5. Deploymentdan oldin tekshiruv ro'yxati

- ✓ `npm run build` xatosiz o'tadimi? (Lokalda sinang.)
- ✓ Barcha maxfiy sozlamalar ( `.env` ) serverda o'rnatilganmi?
- ✓ `.env` GitHub'ga yuklanmaganmi? ( `.gitignore` da bo'lsin.)
- ✓ Ma'lumotlar bazasi production'ga ulanganmi? (SQLite emas — PostgreSQL: Neon/Supabase.)
- ✓ `npm run build && npm run start` bilan production rejimini lokalda sinab ko'rdingizmi? (8-bobdagi kesh shu yerda ishlaydi.)

⚠ **Eslatma:** O'rganishda ishlatgan **SQLite** — production uchun emas. Haqiqiy saytda **PostgreSQL** (Neon, Supabase, yoki Prisma Postgres) ishlatilg. Prisma schemada `provider` ni `postgresql` ga o'zgartirib, `DATABASE_URL` ni yangilang.

## 15-BOB MASALALARI (20 ta)

**Tushunish (1–7):** 1. Deployment nima? Uy oshxonasi vs restoran misolida ayting. 2. Domen nima? Nima vazifani bajaradi? 3. Vercel va VPS farqi nima? Har biri kim uchun? 4. `output: "standalone"` nima qiladi? 5. PM2 nima uchun kerak ( `pm2 startup / pm2 save` nima qiladi)? 6. Nginx (reverse proxy) nima vazifani bajaradi? 7. Nega o'rganishdagi SQLite production uchun yaramaydi?

**Vercel (8–13):** 8. Kodingizni GitHub'ga yuklang (git bilmasangiz — avval o'rganing). 9. Vercel'ga GitHub bilan kiring. 10. "New Project" bilan repongizni ulang. 11. Environment Variables'ga `.env` qiymatlarini qo'shing. 12. "Deploy" bosib, saytni internetga chiqaring. 13. Sayt manzilini ( `...vercel.app` ) brauzerda ochib, ishlashini tasdiqlang.

**VPS va sozlamalar (14–20):** 14. `next.config.ts` ga `output: "standalone"` qo'shing. 15. `npm run build` qilib, `.next/standalone` papkasi yaratilganini ko'ring. 16. (Agar VPS'ingiz bo'lsa) Node.js + PM2 o'rnatib, loyihani ishga tushiring. 17. Oddiy Nginx reverse proxy konfiguratsiyasini yozing (port 3000'ga). 18. Maxfiy va `NEXT_PUBLIC` o'zgaruvchilar farqini izohlang. 19. Deployment oldidan tekshiruv ro'yxatini (5 band) o'z loyihangizga qo'llang. 20. Production bazasini (Neon/Supabase) sozlash bo'yicha reja tuzing (SQLite → PostgreSQL).

**Qo'shimcha challenge:** 3-qismdagi blog loyihangizni haqiqatan internetga chiqaring: bazani PostgreSQL'ga (Neon/Supabase) o'tkazing, Vercel'ga deploy qiling, env'larni sozlang. Do'stlaringizga manzilni yuboring — ular ham foydalansin!

---

## 16-BOB. Monitoring, ilg'or patternlar va yakuniy loyiha

Yakuniy bob. Saytingiz internetda. Endi uni **kuzatish**, professional **patternlarni** bilish va hamma bilimni **bitta loyihada** birlashtirish vaqti.

### 16.1. Monitoring — saytni kuzatish

Sayt chiqdi, lekin foydalanuvchilar xato ko'rsa-chi? Sekin bo'lsa-chi? Buni bilishingiz kerak. **Monitoring** — saytni kuzatish.

**Misol:** Restoran ochildi, lekin oshxonada nima bo'layotganini bilmasangiz — mijozlar shikoyat qilganda kech bo'ladi. Monitoring — oshxonadagi kameralar va sensorlar: muammo bo'lishi bilan bilasiz.

**Asosiy vositalar:** - **Xato kuzatish (error tracking):** [Sentry](#) — saytda xato bo'lsa, sizga darrov xabar beradi (qaysi qator, qaysi foydalanuvchi). - **Analitika:** kim, qayerdan, qaysi sahifaga kirayotganini ko'rish (Vercel Analytics, yoki maxfiylikka do'st Plausible/Umami). - **Tezlik kuzatuv:** Vercel Speed Insights — real foydalanuvchilarda tezlikni o'lchaydi.

Sentry'ni o'rnatish juda oson (ularning qo'llanmasi bor) va boshlovchi loyihalar uchun bepul rejasi yetadi.

### 16.2. `instrumentation.ts` — ilg'or kuzatuv

Next.js maxsus `instrumentation.ts` fayli beradi — u ilova ishga tushganda bajariladigan kuzatuv kodini joylash uchun (masalan, Sentry'ni ulash, OpenTelemetry). Bu — ilg'or mavzu; hozir borligini bilib qo'ying, kerak bo'lganda Sentry hujjatlaridan foydalanasiz.

## 16.3. Ilg'or pattern: Data Access Layer (xavfsizlik uchun)

3-qism (12-bob)da "har action/sahifada sessiyani tekshir" degandik. Buni **markazlashtirish** — professional yondashuv. Sessiya tekshiruvini bitta joyda yozasiz, hamma joyda shuni chaqirasiz:

```
// lib/auth-utils.ts
import { auth } from "@/lib/auth";
import { headers } from "next/headers";
import { redirect } from "next/navigation";

// Markazlashtirilgan tekshiruv
export async function joriyFoydalanuvchi() {
 const session = await auth.api.getSession({ headers: await headers() });
 if (!session) {
 redirect("/login");
 }
 return session.user;
}
```

Endi har sahifada qaytarmaysiz, shunchaki chaqirasiz:

```
export default async function DashboardPage() {
 const user = await joriyFoydalanuvchi(); // bitta qator!
 return <h1>Salom, {user.name}!</h1>;
}
```

Bu "Data Access Layer (DAL)" deyiladi — xavfsizlik mantig'ini bir joyda saqlash. Kod takrorlanmaydi va xato qilish ehtimoli kamayadi.

## 16.4. Ilg'or pattern: Optimistik UI ( `useOptimistic` )

Foydalanuvchi "Yoqdi" tugmasini bossa, server javobini kutmasdan **darrov** yurakcha to'lsa — yaxshi tajriba. React'ning `useOptimistic` hook'i shuni qiladi. Bu — ilg'or mavzu; loyihangiz o'sganda o'rganasiz.

## 16.5. Boshqa ilg'or mavzular (kelajakda o'rganiladigan)

Expert bo'lish yo'lida quyidagilarni sekin-asta o'rganasiz: - **Parallel va Intercepting Routes** — murakkab navigatsiya (modal oynalar, yonma-yon sahifalar). - **Webhooklar** — boshqa xizmatlar (to'lov, email) bilan integratsiya. - **Background jobs / queues** — uzoq ishlarni fonda bajarish. - **i18n** — ko'p tillik sayt. - **CI/CD** — GitHub Actions bilan avtomatik test va deploy. - **Caching strategiyalar** — chuqurroq kesh boshqaruvi (8-bob ustida).

Bularni **bir vaqtda** o'rganmang. Loyiha ishlab, ehtiyoj tug'ilganda — o'sha mavzuni o'rganasiz. Bu — tabiiy o'sish yo'li.

## 16.6. 🏆 YAKUNIY LOYIHA — hamma bilimni birlashtirish

Endi to'rt qismda o'rgangan hamma narsani **bitta haqiqiy loyihada** birlashtiring.

Tavsiya: **mini ijtimoiy/blog platforma** yoki **vazifalar boshqaruvi (task manager)**.

**Talablar (har biri qaysi bobda o'rganilgan):**

1. **Sahifalar va routing** (1-qism, 4-bob): bosh sahifa, profil, har bir element uchun dinamik sahifa.
2. **Server/Client Components** (2-qism, 5-bob): ma'lumot — Server'da, interaktiv tugmalar — Client'da.
3. **Ma'lumot olish** (2-qism, 6-bob): elementlar ro'yxatini bazadan o'qish, `loading / error` bilan.
4. **Server Actions** (2-qism, 7-bob): qo'shish, tahrirlash, o'chirish — formalar orqali.
5. **Keshlash** (2-qism, 8-bob): ro'yxatni `"use cache"` bilan tezlashtirish.
6. **API** (3-qism, 9-bob): mobil ilova uchun kamida bitta Route Handler.
7. **Ma'lumotlar bazasi** (3-qism, 10-bob): Prisma + modellar (foydalanuvchi, element, bog'lanishlar).
8. **Validatsiya** (3-qism, 11-bob): Zod bilan barcha formalarni tekshirish.
9. **Auth** (3-qism, 12-bob): ro'yxatdan o'tish, kirish, faqat egasi tahrirlay olishi.
10. **Tezlik** (4-qism, 13-bob): `<Image>`, `next/font`, `<Suspense>`.
11. **Test** (4-qism, 14-bob): kamida bir nechta unit + E2E test.
12. **Deployment** (4-qism, 15-bob): Vercel'ga chiqarish, PostgreSQL bilan.
13. **Monitoring** (4-qism, 16-bob): Sentry ulash.

Buni tugatsangiz — siz **portfolioga qo'ysa bo'ladigan, internetda ishlab turgan, professional Next.js loyihasi** yaratgan bo'lasiz. Bu — ishga kirish yoki freelance uchun kuchli dalil!

---

## 16-BOB MASALALARI (20 ta)

**Tushunish (1–7):** 1. Monitoring nima va nega kerak? Restoran kamerasi misolida ayting. 2. Sentry nima qiladi? 3. Analitika nimani ko'rsatadi? 4. Data Access Layer (DAL) nima? Nega xavfsizlik uchun foydali? 5. Optimistik UI ( `useOptimistic` ) qanday yaxshi tajriba beradi? 6. Nega ilg'or mavzularni "bir vaqtda" emas, "ehtiyojga qarab" o'rganish kerak? 7. `instrumentation.ts` nima uchun?

**Patternlar (8–12):** 8. `lib/auth-utils.ts` da markazlashtirilgan `joriyFoydalanuvchi()` funksiyasini yozing. 9. Uni bir himoyalangan sahifada chaqirib, kodni soddalashtiring. 10. Sentry hisobi ochib, loyihangizga ulang (ularning qo'llanmasi bilan). 11. Ataylab xato yarating va Sentry uni tutishini tasdiqlang. 12. (Ilg'or) `useOptimistic` haqida o'qib, "Yoqdi" tugmasi misolini tushuning.

**Yakuniy loyiha (13–20):** 13. Yakuniy loyiha mavzusini tanlang (blog, task manager, yoki o'zingizniki). 14. Prisma modellarini loyihalang (foydalanuvchi, asosiy element, bog'lanishlar). 15. Auth qo'shing (register, login, logout). 16. Asosiy CRUD'ni Server Actions + Zod validatsiya bilan qiling. 17. Ro'yxat sahifasini `loading / error / keshlash` bilan yarating. 18. Kamida bir nechta unit (Vitest) va bir E2E (Playwright) test yozing. 19. Tezlikni optimallang ( `<Image>` , `next/font` , `<Suspense>` ). 20. Loyihani Vercel'ga (PostgreSQL bilan) deploy qiling va Sentry ulang.

**Yakuniy challenge:** Yuqoridagi to'liq loyihani boshidan oxirigacha yakunlang va internetga chiqaring. Tugatgach — uni portfoliongizga qo'shing, GitHub'ga yuklang, do'stlaringizga ulashing. **Tabriklaymiz — siz endi Next.js dasturchisiz!**

---

## BUTUN QO'LLANMA YAKUNI

To'rt qismni tugatdingiz. Keling, qancha yo'l bosganingizni ko'rib chiqaylik:

**1-QISM** — Poydevor: JavaScript, React, Next.js loyihasi, routing. **2-QISM** — Dinamik ilova: Server/Client Components, data fetching, Server Actions, keshlash. **3-QISM** — Full-stack: API, ma'lumotlar bazasi, validatsiya, autentifikatsiya. **4-QISM** — Professional: tezlik, test, deployment, monitoring, yakuniy loyiha.

Agar masalalarni yechgan va loyihalarni qurgan bo'lsangiz — siz endi **haqiqiy Next.js dasturchisiz**. Bu — kichik yutuq emas. Ko'pchilik bu yo'lni yakunlamaydi.

Bundan keyin nima qilish kerak?

1. **Loyiha quring.** Bilim — amaliyotda mustahkamlanadi. O'zingizga qiziq narsani (sayt, ilova) yasang. Xato qiling, tuzating, o'rganing.
2. **Boshqa loyihalarni o'qing.** GitHub'dagi ochiq Next.js loyihalarini ko'ring — boshqalar qanday yozganini o'rganing.
3. **Hujjatlarni do'st tuting.** [nextjs.org/docs](https://nextjs.org/docs) — eng ishonchli manba. Yangi versiyalar chiqqanda shu yerda o'qing.
4. **Hamjamiyatga qo'shiling.** Savol bering, javob bering, boshqalar bilan o'rganing.
5. **To'xtamang.** Texnologiya o'zgaradi (masalan, Next.js har yilda yangilanadi). Expert — bu "hammasini bilgan" emas, "doim o'rganishda davom etgan" odam.

**Eng muhim maslahat:** Mukammallikni kutmang. Bugun bilgan narsangiz bilan **bir narsa yasang**. Eng yaxshi o'rganish — qilib ko'rish. Omad!

---

*Bu — "Next.js 0 dan Expert darajagacha" qo'llanmasining yakuniy 4-qismi. To'rt qism birga — JavaScript asoslaridan to professional, internetda ishlab turgan ilovagacha bo'lgan to'liq yo'l.*

---

 [README](#) · [← 3-qism](#) — Full-stack · Keyingi: [BONUS boblar](#) →

# Next.js — 0 dan Expert: BONUS BOBLAR

 [README](#) · [← 4-qism](#) — [Production](#)

**Eslatma:** Bu — asosiy 4 qismga **qo'shimcha (bonus)** boblar. Ular **ilg'or** mavzular bo'lib, 1–4-qismlardagi hamma narsani (ayniqsa Server/Client Components, Server Actions, Route Handlers, auth) bilishni talab qiladi. Agar asosiy qismlarni tugatmagan bo'lsangiz — avval ularga qayting.

**Versiya:** Next.js **16**, React 19. Node.js **20.9+**.

## Bonus boblar

- **17-bob. Ko'p tillik (i18n)** — saytni bir nechta tilda qilish (next-intl)
- **18-bob. To'lov qabul qilish (Stripe + mahalliy provayderlar)** — pul to'lash, obuna, webhook
- **19-bob. Real-time (jonli yangilanish / WebSocket)** — chat, bildirishnoma, jonli ma'lumot

**Maslahat:** Bularning har biri katta mavzu. Hozir asosiy g'oyani va patternni tushunsangiz yetadi — chuqurroq tafsilotlarni real loyihada ehtiyoj tug'ilganda o'rganasiz.

---

## 17-BOB. Ko'p tillik (i18n)

Saytingizni o'zbek, rus va ingliz tillarida ko'rsatmoqchimisiz? Bu — **i18n** (internationalization — "internationalizatsiya"; i va n orasida 18 ta harf bor, shuning uchun "i18n"). O'zbekiston bozori uchun ayniqsa muhim — ko'p saytlar uz/ru/en tillarini talab qiladi.

### 17.1. i18n nima va qanday ishlaydi?

**Misol:** Muzeyni tasavvur qiling — har bir eksponat oldida bir nechta tilda yozuv bor. Eksponat bitta, lekin tashrif buyuruvchi o'z tilidagi yozuvni o'qiydi. URL'dagi til kodi (`/uz`, `/ru`, `/en`) — "qaysi tildagi yozuvlarni ko'rsataylik" degani.

i18n ikki qismdan iborat: 1. **Routing (yo'naltirish)**: URL'da til ko'rsatiladi — `/uz/about` , `/ru/about` , `/en/about` . 2. **Tarjimalar**: matnlar har til uchun alohida faylda saqlanadi (lug'at kabi).

## 17.2. ⚠ App Router'da i18n: kutubxona kerak

**Muhim:** 1-qismda o'rgangan App Router'da Next.js'ning **o'zining ichki i18n sozlamasi yo'q** (u faqat eski Pages Router uchun edi). Shuning uchun kutubxona ishlatamiz.

2026-yilda App Router uchun eng yaxshi tanlov — **next-intl**: - Juda yengil (~2KB) - Server Components'da to'g'ridan-to'g'ri ishlaydi (qo'shimcha JavaScript yo'q) - TypeScript bilan zo'r - Next.js 16 bilan to'liq mos

Boshqa variantlar ham bor ( `react-i18next` , `next-i18next` v16), lekin biz next-intl'ni o'rganamiz — App Router uchun standart.

```
npm install next-intl
```

## 17.3. Sozlash (4 ta fayl)

### 1) Tillarni belgilash — `i18n/routing.ts`:

```
// i18n/routing.ts
import { defineRouting } from "next-intl/routing";

export const routing = defineRouting({
 locales: ["uz", "ru", "en"], // qo'llab-quvvatlanadigan tillar
 defaultLocale: "uz", // standart til
});
```

### 2) Yo'naltirish — `proxy.ts`:

⚠ **Next.js 16 eslatma:** Bu fayl ilgari (15-versiyagacha) `middleware.ts` deb atalardi. Next.js 16'da u **proxy.ts** ga o'zgardi (3-qism, 12-bobni eslang). Eski darsliklarda `middleware` ko'rsangiz — endi u `proxy`.

```
// proxy.ts (loyiha ildizida)
import createMiddleware from "next-intl/middleware";
import { routing } from "../i18n/routing";

export default createMiddleware(routing);

export const config = {
 // API, statik fayllar va Next ichki yo'llaridan tashqari hammasiga
 qo'llanadi
}
```

```
matcher: ["/((?!api|_next|.*\\.\\.*)).*"/],
};
```

Bu foydalanuvchining tilini aniqlaydi (cookie → brauzer tili → standart) va to'g'ri til manziliga yo'naltiradi.

### 3) Til-sezgir navigatsiya — `i18n/navigation.ts`:

```
// i18n/navigation.ts
import { createNavigation } from "next-intl/navigation";
import { routing } from "../routing";

export const { Link, redirect, usePathname, useRouter } =
 createNavigation(routing);
```

**Muhim:** Endi Next.js'ning oddiy `Link` / `useRouter` ini emas, **shularni** ishlatish — ular til prefiksini (`/uz`, `/ru`) avtomatik qo'shadi.

### 4) Tarjima fayllari — `messages/` papkasida:

```
// messages/uz.json
{
 "salom": "Salom dunyo",
 "haqida": "Biz haqimizda"
}
```

```
// messages/ru.json
{
 "salom": "Привет мир",
 "haqida": "О нас"
}
```

```
// messages/en.json
{
 "salom": "Hello world",
 "haqida": "About us"
}
```

## 17.4. `[locale]` papka strukturasi

Endi barcha sahifalar `app/[locale]/` ichiga ko'chadi (1-qismdagi dinamik sahifalarni eslang):

```
app/
└─ [locale]/
 └─ layout.tsx
```

```
└─ page.tsx
 └─ about/
 └─ page.tsx
```

**Layout** — tarjimalarni yuklab beradi:

```
// app/[locale]/layout.tsx
import { NextIntlClientProvider } from "next-intl";
import { setRequestLocale } from "next-intl/server";
import { routing } from "@i18n/routing";

// Barcha tillar uchun sahifalarni oldindan tayyorlash (statik)
export function generateStaticParams() {
 return routing.locales.map((locale) => ({ locale }));
}

export default async function LocaleLayout({
 children,
 params,
}: {
 children: React.ReactNode;
 params: Promise<{ locale: string }>;
}) {
 const { locale } = await params; // Next.js 16 - async params
 setRequestLocale(locale); // joriy tilni o'rnatamiz

 return (
 <html lang={locale}>
 <body>
 <NextIntlClientProvider>{children}</NextIntlClientProvider>
 </body>
 </html>
);
}
```

## 17.5. Tarjimani ishlatish — useTranslations

Endi sahifalarda tarjimalardan foydalanamiz:

```
// app/[locale]/page.tsx
import { useTranslations } from "next-intl";
import { setRequestLocale } from "next-intl/server";

export default function Home({ params }: { params: Promise<{ locale: string }> }) {
 // (server component'da setRequestLocale uchun locale kerak bo'lsa,
 // await qiling)
 const t = useTranslations(); // tarjima funksiyasi

 return (
```

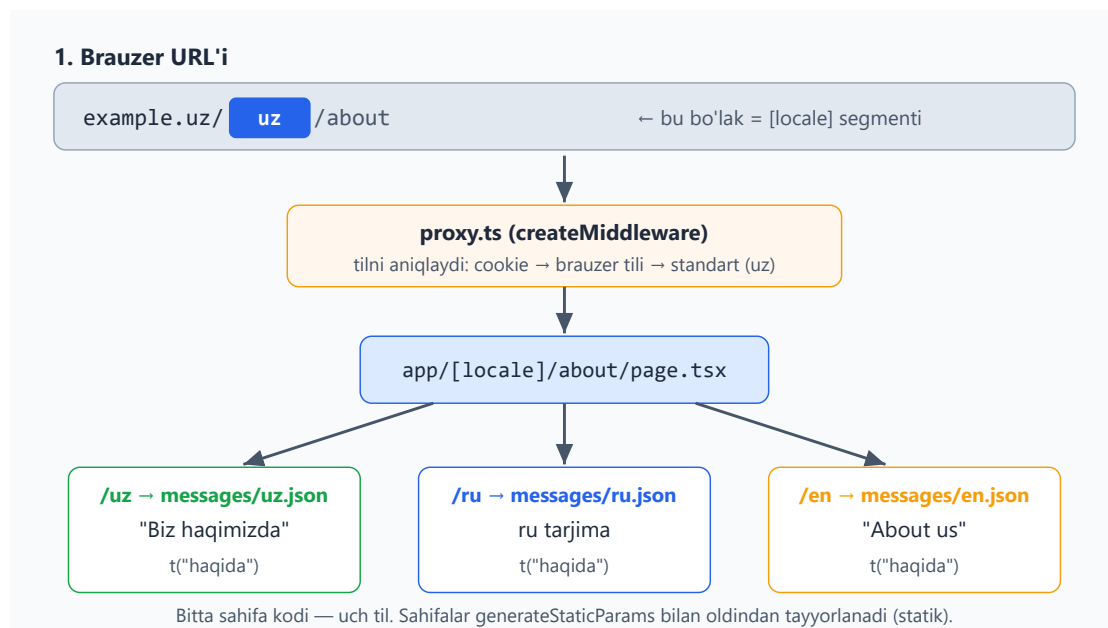
```

<div>
 <h1>{t("salom")}</h1>
 <p>{t("haqida")}</p>
</div>
);
}

```

/uz ga kirsangiz "Salom dunyo", /ru ga "Привет мир", /en ga "Hello world" chiqadi!  
Bitta kod — uch til.

Quyidagi diagrammada URL'dagi [locale] segmenti qanday qilib tilni aniqlashi va tegishli tarjima faylidan mos kontentni ko'rsatishi tasvirlangan:



**Diqqat:** `t("salom")` — `messages` faylidagi kalitni oladi. Kalit bo'lmasa, xato chiqadi.

## 17.6. Til almashtirish tugmasi

Foydalanuvchi tilni o'zgartira olishi uchun (next-intl navigatsiyasidan foydalanib):

```

"use client";
import { useRouter, usePathname } from "@i18n/navigation";

export default function TilAlmashtir() {
 const router = useRouter();
 const pathname = usePathname();

 return (
 <select onChange={(e) => router.replace(pathname, { locale:
e.target.value }}}>
 <option value="uz">O'zbekcha</option>
 <option value="ru">Русский</option>
 </select>
);
}

```

```
 <option value="en">English</option>
 </select>
);
}
```

Tanlangan til avtomatik saqlanadi (cookie) va keyingi tashriflarda eslab qolinadi.

## 17.7. Maslahatlar

- **Kalitlarni mavzu bo'yicha guruhlang** (namespace): `messages/uz.json` ichida `{ "home": {...}, "about": {...} }` — har sahifa o'z bo'limini oladi.
- **ICU sintaksisi**: son/sana/ko'plik formatlarini `next-intl` avtomatik to'g'rilaydi (masalan, "1 ta xabar" vs "5 ta xabar").
- **SEO**: har til o'z URL'iga ega bo'lgani uchun Google har tilni alohida indekslaydi — zo'r.

## 17-BOB MASALALARI (20 ta)

**Tushunish (1–7):** 1. `i18n` nima? Muzey yozuvlari misolida tushuntiring. 2. `i18n`'ning ikki qismi (routing va tarjimalar) nima? 3. Nega App Router'da kutubxona kerak (Next.js o'zida yo'q)? 4. 2026'da App Router uchun nega `next-intl` tavsiya etiladi? 5. `[locale]` papka nima qiladi? URL'da qanday ko'rinadi? 6. Nega oddiy `Link` emas, `next-intl`'ning `Link` ini ishlatamiz? 7. Next.js 16'da `middleware.ts` nimaga o'zgardi (`i18n` routing'da qaysi fayl)?

**Sozlash (8–14):** 8. `next-intl`'ni o'rnating. 9. `i18n/routing.ts` da `uz`, `ru`, `en` tillarini belgilang (default: `uz`). 10. `proxy.ts` yarating (`createMiddleware`). 11. `i18n/navigation.ts` yarating. 12. `messages/uz.json`, `ru.json`, `en.json` fayllarini yarating (kamida 3 ta kalit). 13. Sahifalarni `app/[locale]/` ichiga ko'chiring. 14. `app/[locale]/layout.tsx` ni sozlang (`generateStaticParams` + `setRequestLocale`).

**Ishlatish (15–20):** 15. Bosh sahifada `useTranslations` bilan tarjimalarni ko'rsating. 16. `/uz`, `/ru`, `/en` ga kirib, matn o'zgarishini tasdiqlang. 17. `about` sahifasi yaratib, unga ham tarjima qo'shing. 18. Til almashtirish tugmasini (`<select>`) yarating. 19. Tilni almashtiring va sahifa o'zgarishini ko'ring. 20. `messages` faylini namespace bilan guruhlang (`home`, `about`) va ishlatang.

**Qo'shimcha challenge:** 3-qismdagi blog loyihangizni 3 tilda qiling: menyu, tugmalar, sahifa sarlavhalari uchun tarjimalar. Til almashtirish tugmasi har sahifada ishlasin. (Maqola matnlari bazadan kelsa — har til uchun alohida maydon yoki tarjima jadvali kerak bo'ladi, buni o'ylab ko'ring.)

---

## 18-BOB. To'lov qabul qilish (Stripe va mahalliy provayderlar)

Sayingizdan pul to'plamoqchimisiz? Obuna, mahsulot sotish, xizmat haqi? Bu bob — to'lovlarni **xavfsiz** qabul qilish haqida. Biz **Stripe** misolida o'rganamiz, lekin pattern hamma joyda (jumladan Payme/Click) bir xil.

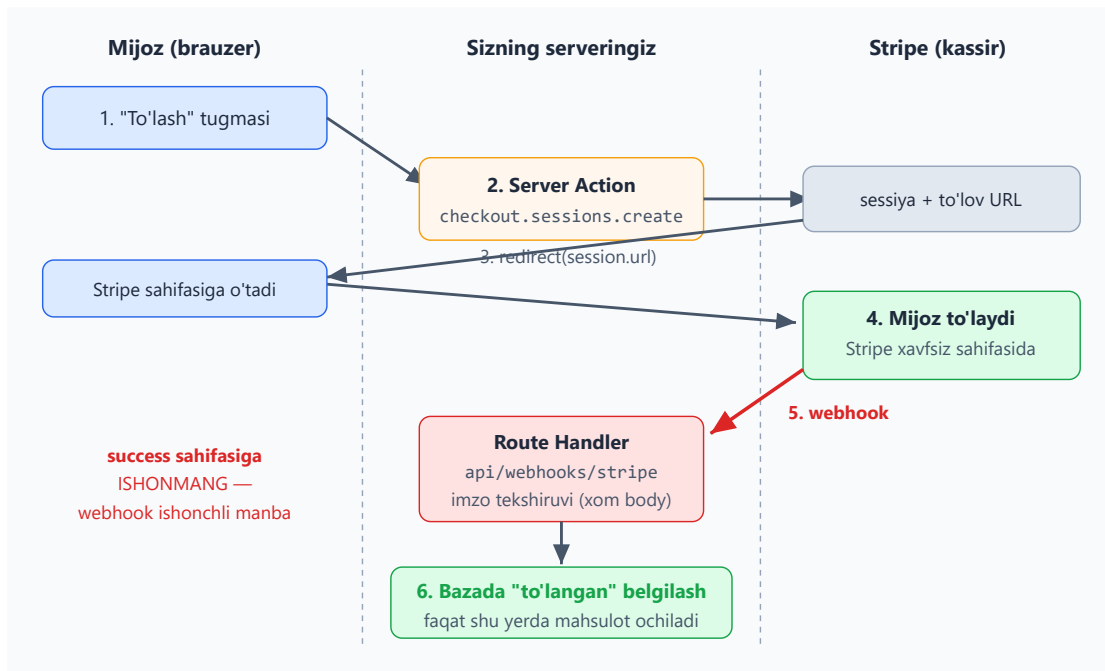
### 18.1. To'lovning asosiy pattern'i (eng muhim g'oya)

**Misol:** Siz pulni **o'zingiz** sanab olmaysiz. Mijozni ishonchli **kassirga** (Stripe, Payme) yuborasiz. Kassir pulni oladi va sizga **tasdiqnoma** (webhook) yuboradi: "to'landi". Siz mahsulotni faqat tasdiqnoma kelgach ochasiz — mijoz "to'ladim" degani uchun emas.

To'lov jarayoni doim shunday (qaysi provayder bo'lishidan qat'i nazar): 1. Mijoz "To'lash" tugmasini bosadi. 2. Sayt server'da **to'lov sessiyasi** yaratadi. 3. Mijoz to'lov sahifasiga (Stripe/Payme) **yo'naltiriladi**. 4. Mijoz to'laydi. 5. Provayder sizga **webhook** (tasdiqnoma) yuboradi. 6. Siz tasdiqnoma kelgach mahsulotni/xizmatni ochasiz.

⚠ **Eng muhim:** "Ochishni" 5-qadamdagi **webhook** asosida qiling, 3-qadamdagi muvaffaqiyat sahifasi asosida emas! Mijoz to'lov sahifasini yopib qo'ysa yoki aldasa — webhook'gina ishonchli manba.

Quyidagi diagrammada shu oltita qadam — tugmadan checkout sessiyasiga, Stripe to'lov sahifasiga, so'ng webhook orqali bazada tasdiqlashgacha — yaxlit ko'rsatilgan:



## 18.2. ⚠ O'zbekiston konteksti: Payme, Click, Uzum

**Muhim: Stripe O'zbekistonda ishlamaydi** (Stripe bu yerda mavjud emas).  
O'zbekiston bozori uchun **Payme, Click, Uzum** kabi mahalliy provayderlar ishlatiladi.

**Yaxshi xabar:** yuqoridagi pattern (sessiya yaratish → yo'naltirish → webhook bilan tasdiqlash) **bir xil**. Payme/Click'ning ham o'z API va webhook'lari bor. Stripe'ni o'rganib, patternni tushunsangiz — mahalliy provayderlarga ham oson o'tasiz (faqat API hujjatlari boshqacha). Quyida Stripe misolida pattern'ni o'rganamiz, keyin uni Payme/Click'ga moslaysiz.

Halqaro bozorga (masalan, xorijiy mijozlar) sotsangiz — Stripe to'g'ri keladi. Mahalliy bozor uchun — Payme/Click/Uzum.

## 18.3. Stripe'ni o'rnatish

```
npm install stripe @stripe/stripe-js
```

- `stripe` — server uchun (maxfiy, **hech qachon Client Component'da ishlatmang!**).
- `@stripe/stripe-js` — brauzer uchun (yengil yordamchi).

`.env` ga kalitlarni qo'shing (Stripe Dashboard'dan olasiz):

```
STRIPE_SECRET_KEY=sk_test_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

```
STRIPE_WEBHOOK_SECRET=whsec_...
NEXT_PUBLIC_APP_URL=http://localhost:3000
```

Stripe'ni bir joyda sozlang:

```
// lib/stripe.ts
import Stripe from "stripe";

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!);
```

## 18.4. To'lov sessiyasini yaratish (Server Action)

3-qism (7-bob)dagi Server Actions'ni eslang. To'lov sessiyasini server'da yaratamiz:

```
// app/actions.ts
"use server";

import { stripe } from "@/lib/stripe";
import { redirect } from "next/navigation";

export async function tolovBoshlash(priceId: string) {
 // To'lov sessiyasini yaratamiz
 const session = await stripe.checkout.sessions.create({
 mode: "payment", // bir martalik to'lov (obuna uchun
 "subscription")
 line_items: [{ price: priceId, quantity: 1 }],
 success_url: `${process.env.NEXT_PUBLIC_APP_URL}/success`,
 cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/cancel`,
 });

 // Mijozni Stripe to'lov sahifasiga yo'naltiramiz
 redirect(session.url!);
}
```

Mijoz tugmani bosganda, u Stripe'ning xavfsiz to'lov sahifasiga o'tadi. Siz parol/karta ma'lumotlarini **umuman ko'rmaysiz** — Stripe hammasini o'zi qiladi (xavfsiz).

## 18.5. ⚠ Webhook — to'lovni tasdiqlash (MAJBURIY)

Mana eng muhim qism. Mijoz to'lagach, Stripe sizga **webhook** yuboradi. Bu — **Route Handler** bo'lishi **shart** (Server Action emas), chunki Stripe'ga "ping" qilish uchun doimiy URL kerak.

```
// app/api/webhooks/stripe/route.ts
import { stripe } from "@/lib/stripe";
import { headers } from "next/headers";
```

```
import { NextResponse } from "next/server";

export async function POST(req: Request) {
 // 1. XOM (raw) body - JSON.parse QILMANG, imzoni buzadi!
 const body = await req.text();
 const signature = (await headers()).get("stripe-signature")!;

 // 2. Imzoni tekshiramiz (haqiqatan Stripe'danmi?)
 let event;
 try {
 event = stripe.webhooks.constructEvent(
 body,
 signature,
 process.env.STRIPE_WEBHOOK_SECRET!
);
 } catch (err) {
 return new NextResponse("Webhook imzosi noto'g'ri", { status: 400 });
 }

 // 3. To'lov tugaganini tekshiramiz
 if (event.type === "checkout.session.completed") {
 const session = event.data.object;
 // ENDI mahsulotni/xizmatni ochamiz (bazada belgilaymiz)
 console.log("To'lov tasdiqlandi:", session.id);
 // await prisma.order.update(...) - buyurtmani "to'langan" qilamiz
 }

 return new NextResponse(null, { status: 200 });
}
```

**Tushuntirish:** - `req.text()` — xom body (juda muhim: `req.json()` ishlatmang — imzo tekshiruvini buzadi). - `constructEvent(...)` — webhook haqiqatan Stripe'danmi yoki firibgarmi — imzo bilan tekshiradi. - `checkout.session.completed` — to'lov muvaffaqiyatli bo'lganini bildiradi. - Faqat shu yerda mahsulotni ochasiz — bu ishonchli manba.

## 18.6. ⚠ Webhook'lar bir necha marta kelishi mumkin (idempotency)

**Muhim:** Stripe webhook'ni bir necha marta yuborishi mumkin (uzilish bo'lsa, 3 kun davomida qayta urinadi). Shuning uchun bir to'lovni **ikki marta** ishlamasligingiz kerak (masalan, mijozga ikki marta mahsulot bermang).

Yechim — qaysi webhook'larni ishlaganingizni eslab qolish:

```
// Ishlashdan oldin tekshiring:
const oldin = await prisma.processedWebhook.findUnique({
 where: { eventId: event.id },
```



```
});
if (oldin) return new NextResponse(null, { status: 200 }); //
allaqachon ishlangan

// ... ishni bajaring ...

// Keyin belgilang:
await prisma.processedWebhook.create({ data: { eventId: event.id } });
```

## 18.7. Lokal sinash — Stripe CLI

Webhook'ni lokalda ( `localhost` ) sinash uchun Stripe CLI ishlatiladi:

```
Stripe CLI'ni o'rnatish (https://stripe.com/docs/stripe-cli)
stripe login

Webhook'larni lokal serveringizga yo'naltirish:
stripe listen --forward-to localhost:3000/api/webhooks/stripe

Test to'lov hodisasini yuborish:
stripe trigger checkout.session.completed
```

Bu `STRIPE_WEBHOOK_SECRET` ni beradi (uni `.env` ga qo'ying) va to'lovlarni soxta sinash imkonini beradi (haqiqiy pul ishlatmasdan).

## 18.8. Obunalar (subscriptions)

Bir martalik emas, **oylik obuna** uchun (masalan, EduCore \$29/oy): - `mode: "subscription"` ishlatish ( `"payment"` o'rniga). - Stripe Dashboard'da "recurring" (takrorlanuvchi) narx yarating. - Webhook'da `customer.subscription.created`, `invoice.payment_succeeded` kabi hodisalarni tinglang.

Obunalar murakkabroq (bekor qilish, yangilash, to'lov o'tmaganda). Boshida bir martalik to'lovni o'rganing, keyin obunaga o'ting.

## 18-BOB MASALALARI (20 ta)

**Tushunish (1–8):** 1. To'lovning asosiy pattern'ini (6 qadam) ayting. Kassir misolida tushuntiring. 2. Nega mahsulotni "success sahifasi" emas, "webhook" asosida ochamiz? 3. O'zbekistonda qaysi to'lov provayderlari ishlatiladi? Stripe ishlaydimi? 4. Pattern Payme/Click uchun ham bir xilmi? Nega Stripe'ni o'rganish foydali? 5. Nega `stripe` (server SDK) Client Component'da ishlatib bo'lmaydi? 6. Webhook nega Server

Action emas, Route Handler bo'lishi kerak? 7. Webhook'da nega `req.text()` (xom body) ishlatiladi, `req.json()` emas? 8. Idempotency nima? Nega webhook'ni ikki marta ishlamaslik kerak?

**Sozlash va sessiya (9–14):** 9. Stripe'ni o'rnatish (`stripe + @stripe/stripe-js`). 10. `.env` ga 3 ta Stripe kalitini qo'shing. 11. `lib/stripe.ts` yarating. 12. Stripe Dashboard'da test mahsulot va narx yarating (`priceId` oling). 13. `tolovBoshlash(priceId)` Server Action yozing (sessiya yaratish + redirect). 14. "To'lash" tugmasini yarating va Server Action'ga ulang.

**Webhook (15–20):** 15. `app/api/webhooks/stripe/route.ts` yarating (imzo tekshiruv bilan). 16. `checkout.session.completed` hodisasini tutib, `console.log` qiling. 17. Stripe CLI'ni o'rnatib, `stripe listen` bilan webhook'ni ulang. 18. `stripe trigger` bilan test to'lov yuborib, webhook ishlashini ko'ring. 19. Idempotency qo'shing (`processedWebhook` jadvali bilan). 20. To'lovdan keyin bazada buyurtmani "to'langan" deb belgilang (Prisma bilan).

**Qo'shimcha challenge:** Oddiy "mahsulot sotish" sahifasi yasang: mahsulotlar ro'yxati (har biriga narx), "Sotib olish" tugmasi → Stripe checkout → webhook → bazada buyurtmani saqlash. To'lov tugaganlar `/orders` sahifasida ko'rinsin. (Mahalliy bozor uchun: keyinroq shu pattern'ni Payme/Click hujjatlari bo'yicha moslang.)

---

## 19-BOB. Real-time (jonli yangilanish / WebSocket)

Chat, jonli bildirishnoma, "kim onlayn", jonli hisoblagich, hamkorlikdagi tahrirlash — bularning hammasi **real-time** (jonli) imkoniyatlardir. Bu bobda Next.js'da ularni qanday qilishni o'rganamiz. Diqqat: bu Next.js'da **alohida e'tibor** talab qiladigan mavzu.

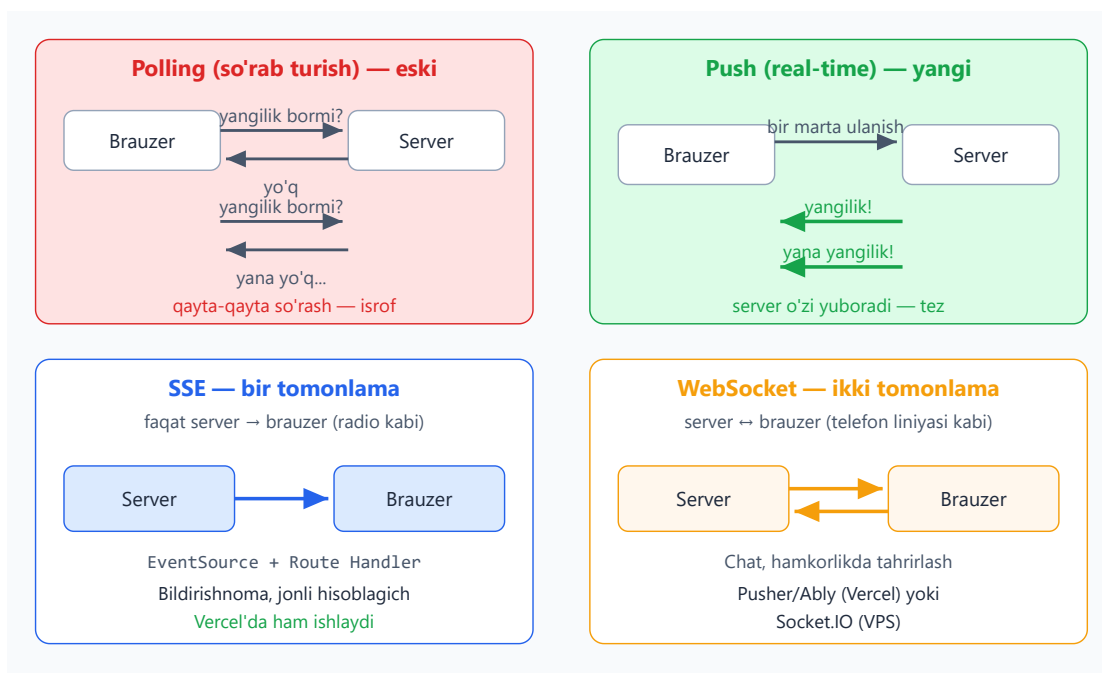
### 19.1. Real-time nima va muammosi nimada?

**Misol:** Oddiy web — siz har safar eshikni ochib "yangilik bormi?" deb so'rashingiz kerak (so'rab turish — "polling"). Real-time — yangilik **o'zi sizga keladi**, sodir bo'lishi bilanoq (push) — eshikni qayta-qayta tekshirish o'rniga qo'ng'iroq eshitganga o'xshaydi.

Texnik jihatdan: - **WebSocket** — server va brauzer o'rtasida **ikki tomonlama, doimiy ochiq** aloqa liniyasi (telefon liniyasiga o'xshaydi — ikkalasi ham gaplashishi mumkin). -

**SSE (Server-Sent Events) — bir tomonlama** (faqat server → brauzer). Radio eshittirishiga o'xshaydi (stansiya → siz).

Quyidagi diagrammada avval polling (so'rab turish) va push (real-time) farqi, so'ng SSE'ning bir tomonlama va WebSocket'ning ikki tomonlama aloqasi yonma-yon ko'rsatilgan:



## 19.2. ⚠ Serverless muammosi (eng muhim tushuncha)

**Muhim:** Vercel, Netlify kabi **serverless** platformalar **doimiy ochiq aloqani** (WebSocket) qo'llab-quvvatlamaydi! Sizning ilovangiz "cloud funksiya" sifatida ishlaydi — so'rov tugagach, funksiya o'chiriladi. Doimiy ochiq liniyani qanday saqlaysiz?

**Telefon analogiyasi:** Serverless'da operator har gapdan keyin telefonni o'chirib qo'yadi — liniyani ochiq saqlab bo'lmaydi.

Bu muammo yechimlarini ikkiga bo'ladi: 1. **Tashqi xizmat** (Pusher, Ably, Supabase Realtime) — WebSocket'ni **ular** boshqaradi, siz ulanasiz. Vercel'da **yagona yo'l**. 2. **O'z serveringiz** (VPS) — doimiy ishlaydigan Node.js server. Bu yerda **Socket.IO** ishlaydi.

## 19.3. Uchta yondashuv — qaysi birini qachon?

Yondashuv	Qanday ishlaydi	Qachon
<b>SSE</b> (Server-Sent Events)	Bir tomonlama push, Route Handler orqali	Bildirishnoma, jonli hisoblagich (faqat server → mijoz)
<b>Tashqi xizmat</b> (Pusher/Ably)	WebSocket'ni xizmat boshqaradi	Vercel'da chat, ikki tomonlama, tez boshlash
<b>Socket.IO</b> (VPS'da)	O'z serveringizda doimiy WebSocket	VPS bor, to'liq nazorat, bepul (server haqi bor)

**Maslahat siz uchun:** Agar **Vercel**'da bo'lsangiz → SSE (bir tomonlama uchun) yoki Pusher/Ably (chat uchun). Agar **VPS**'ingiz bo'lsa → Socket.IO ham yaxshi variant (bepul, to'liq nazorat).

## 19.4. SSE — eng oddiy yo'l (bir tomonlama push)

SSE Route Handler orqali ishlaydi va **hamma joyda** (jumladan Vercel'da) ishlaydi. Bildirishnoma, jonli yangiliklar uchun ideal.

**Server tomoni (Route Handler):**

```
// app/api/notifications/route.ts
export async function GET() {
 const encoder = new TextEncoder();

 const stream = new ReadableStream({
 start(controller) {
 // Har 3 soniyada mijozga xabar yuboramiz
 const interval = setInterval(() => {
 const data = JSON.stringify({ vaqt: new Date().toISOString() });
 controller.enqueue(encoder.encode(`data: ${data}\n\n`));
 }, 3000);

 // Aloqa uzilganda tozalaymiz
 // (real loyihada request.signal bilan)
 },
 });

 return new Response(stream, {
 headers: {
 "Content-Type": "text/event-stream",
 "Cache-Control": "no-cache",
 Connection: "keep-alive",
 },
 });
}
```

```
});
}
```

### Brauzer tomoni (Client Component):

```
"use client";
import { useEffect, useState } from "react";

export default function Bildirishnomalar() {
 const [xabar, setXabar] = useState("");

 useEffect(() => {
 const eventSource = new EventSource("/api/notifications");

 eventSource.onmessage = (event) => {
 const data = JSON.parse(event.data);
 setXabar(data.vaqt); // har 3 soniyada yangilanadi
 };

 return () => eventSource.close(); // tozalash
 }, []);

 return <p>Oxirgi yangilanish: {xabar}</p>;
}
```

`EventSource` — brauzerning SSE uchun tayyor vositasi. Server xabar yuborganda `onmessage` ishlaydi. So'rab turish (polling) kerak emas!

## 19.5. Tashqi xizmat — Pusher (Vercel'da chat uchun)

Ikki tomonlama (chat) va Vercel uchun — Pusher kabi xizmat. G'oya: 1. Pusher'da bepul hisob ochasiz (kalitlar olasiz). 2. Xabar yuborilganda, server (Server Action/Route Handler) Pusher'ga "e'lon qil" deydi. 3. Pusher uni barcha ulangan brauzerlarga tarqatadi.

```
// Soddalashtirilgan g'oya (server tomoni)
// import Pusher from "pusher";
// pusher.trigger("chat", "yangi-xabar", { matn: "Salom!" });
```

```
// Brauzer tomoni
// pusher.subscribe("chat").bind("yangi-xabar", (data) => {
// // yangi xabarni ko'rsatish
// });
```

Pusher WebSocket'ni o'zi boshqaradi — siz faqat "e'lon qil" va "tingla" deysiz. Vercel'da ishlaydi. (Abyl ham shunga o'xshash, kattaroq miqyos uchun.)

**Narx eslatmasi:** Bu xizmatlar xabar miqdoriga qarab pul oladi (Pusher'da ~200 ulanish bepul). Ko'p foydalanuvchili chatda qimmatga tushishi mumkin — shunda VPS + Socket.IO arzonroq bo'ladi.

## 19.6. Socket.IO — VPS'da to'liq nazorat

Agar o'z **VPS**'ingiz bo'lsa (3-qism, 15-bobni eslang), **Socket.IO** bilan to'liq, ikki tomonlama, bepul real-time qura olasiz. Socket.IO — bu sohaning standarti (rooms, namespaces, avtomatik qayta ulanish).

**Diqqat:** Socket.IO doimiy ishlaydigan server talab qiladi. Buni odatda Next.js'dan alohida **Node.js server** sifatida (yoki maxsus server sozlamasi bilan) ishlatiladi, VPS'da PM2 bilan boshqariladi. Bu ilg'or sozlash — Socket.IO va Next.js hujjatlarini ko'ring.

VPS'ingiz borligi uchun bu yo'l siz uchun tabiiy va arzon (xizmat haqi yo'q, faqat server haqi).

## 19.7. Qaysi birini tanlash — qisqacha

- **Faqat server** → **mijoz push** (bildirishnoma, jonli son) → **SSE** (oddiy, bepul, hamma joyda).
- **Chat/ikki tomonlama + Vercel** → **Pusher/Ably** (tez, lekin pulli).
- **Chat/ikki tomonlama + VPS** → **Socket.IO** (bepul, nazorat, biroz ko'proq sozlash).
- **Allaqachon Supabase'da** → **Supabase Realtime** (baza o'zgarishi → jonli yangilanish).

**Maslahat:** Boshlovchi uchun **SSE**'dan boshlang — eng oddiy va ko'p holatda yetarli. Chat kerak bo'lganda tashqi xizmat yoki Socket.IO'ga o'ting.



## 19-BOB MASALALARI (20 ta)

**Tushunish (1–9):** 1. Real-time nima? "So'rab turish (polling)" va "push" farqi nima? Qo'ng'iroq misolida ayting. 2. WebSocket va SSE farqi nima (ikki tomonlama vs bir tomonlama)? 3. Serverless'da (Vercel) WebSocket nega muammo? Telefon analogiyasi bilan tushuntiring. 4. Real-time uchun 2 ta asosiy yechim turkumi (tashqi xizmat va o'z server) nima? 5. SSE qachon to'g'ri keladi? Misol ayting. 6. Pusher/Ably qachon ishlatiladi? Kamchiligi nima (narx)? 7. Socket.IO qachon yaxshi (qanday server kerak)?

8. Vercel'da chat uchun nima ishlatasiz? VPS'da-chi? 9. Supabase Realtime qachon mantiqiy?

**SSE amaliyot (10–16):** 10. `app/api/notifications/route.ts` yaratib, SSE stream qaytaring. 11. Har 3 soniyada joriy vaqtni yuboradigan qiling. 12. Client Component'da `EventSource` bilan ulang. 13. Kelgan xabarni ekranda ko'rsating (har 3 soniyada yangilansin). 14. `onmessage` va `eventSource.close()` nima qilishini izohlang. 15. Yuborish oralig'ini 1 soniyaga o'zgartiring va kuzating. 16. Server yuboradigan ma'lumotni o'zgartiring (masalan, tasodifiy son).

**Konseptual va tanlov (17–20):** 17. Pusher bilan chat qanday ishlashini (3 qadam) o'z so'zingiz bilan yozing. 18. Socket.IO uchun nega doimiy server kerakligini izohlang. 19. Quyidagilar uchun qaysi yechimni tanlaysiz: (a) Vercel'da bildirishnoma, (b) VPS'da chat, (c) jonli "kim onlayn" hisoblagichi? Izohlang. 20. O'z loyihangiz uchun bitta real-time imkoniyat o'ylab toping va qaysi yondashuv mosligini ayting.

**Qo'shimcha challenge:** SSE bilan **jonli bildirishnoma** tizimi yasang: server yangi voqea bo'lganda (masalan, kimdir izoh qoldirganda) ulangan brauzerlarga xabar yuborsin, ekranda "🔔 Yangi bildirishnoma" chiqsin. (Ilg'or: Pusher yoki Socket.IO bilan oddiy 2 kishilik chat yasashga harakat qiling.)

---



## BONUS BOBLAR YAKUNI

Tabriklaymiz — endi sizda **professional, real loyihalarda kerak bo'ladigan** qo'shimcha ko'nikmalar ham bor:

✅ **i18n** — saytni o'zbek/rus/ingliz (yoki istalgan) tillarida (next-intl) ✅ **To'lov** — Stripe (va shu pattern bilan Payme/Click/Uzum) orqali xavfsiz pul qabul qilish — webhook bilan ✅ **Real-time** — chat, bildirishnoma, jonli yangilanish (SSE, Pusher, Socket.IO) — serverless muammosini bilgan holda

Bu uchta mavzu — ko'p SaaS va biznes loyihalarida talab qilinadi. Ularni asosiy 4 qism bilan birga ishlatasangiz, deyarli har qanday zamonaviy veb-ilovani qura olasiz.

**Eslatma:** Bularning hammasi — chuqur mavzular. Bu boblar sizga **pattern va yo'nalishni** berdi. Haqiqiy ustalik — real loyihada ishlash, hujjatlarni o'qish va tajriba orttirish bilan keladi. Omad! 🚀

---

*Bu — "Next.js 0 dan Expert darajagacha" qo'llanmasining bonus boblari. Asosiy 4 qism + shu 3 bonus bob birga — boshlang'ich darajadan to ilg'or, professional SaaS*

*imkoniyatlarigacha bo'lgan to'liq yo'l.*

---

 [README / Mundarija](#) · [← 4-qism](#) — Production